STRATO v1.0.0: Final Deliverable

Spherical TRISO Analysis and Transport Operations

2026-11-06

Contents

1 Scope and model summary **3**

2 Notation **4**

3 Finite-volume solver (Code B) **6**

3.1 Cell faces, uniform width per layer, last face set to R_ℓ . . . 8
3.2 Cell volume . . . 9
3.3 Volume centroid . . . 10
3.4 Face area . . . 11
3.5 Half-cell distances to the face . . . 11
3.6 Internal face conductance (series resistors) . . . 12
3.7 Outer-boundary conductance (half cell to $c = 0$) . . . 13
3.8 Centre condition: no flow through $r = 0$ . . . 13
3.9 Initial condition: kernel full, coatings empty . . . 14
3.10 Inventory (also M^0) . . . 15
3.11 Time step actually used . . . 16
3.12 Backward Euler coefficients . . . 16
3.13 Tridiagonal solve, forward sweep without subtraction . . . 18
3.14 Back substitution . . . 19
3.15 Release rate . . . 20
3.16 Cumulative release with Kahan compensation . . . 21
3.17 Balance residual . . . 21
3.18 Release fraction . . . 22
3.19 Output rows and minimum concentration . . . 23
3.20 Explicit-step bound (diagnostic, printed once) . . . 24
3.21 Forward Euler scheme and its L^2 limit . . . 24
3.22 Source term, decay, $D(T)$, Robin outer face . . . 25

4 Analytical solver (Code C) **26**

4.1 Separation of variables, shape equation . . . 28
4.2 Transformation $u = r\phi$ and wavenumber . . . 28
4.3 Layer transfer of the state (u, q) . . . 29
4.4 Interface rule . . . 30
4.5 Centre start and shooting, characteristic function . . . 31
4.6 Phase form and zero count (Sturm count) . . . 32
4.7 Mode cap . . . 33
4.8 Lower bracket . . . 34
4.9 Root isolation by bisection on the count; stopping rule . . . 35

4.10 Input check of the layer list . . . 36
4.11 Eigenfunction in each layer . . . 37
4.12 Scale of the eigenfunction . . . 38
4.13 Quadrature . . . 39
4.14 Norm and moments . . . 40
4.15 Modal weight . . . 41
4.16 Time evolution and release fraction . . . 42
4.17 Truncation and validity time . . . 42
4.18 Flux ratio and accuracy floor . . . 43
4.19 Robin root equation (Case B reference) . . . 44
4.20 Orthogonality, expansion coefficient A_n, series for $c(r,t)$, inventory series, closed-form integrals . . . 45
4.21 Completeness proof, tail bound, clamp of negative F, partition coefficients, Robin release for the layered particle . . . 46

5 Program flow and comparison (Code A) 47
5.1 Read input, reject unfilled placeholders . . . 49
5.2 Validate input (five regions, radii, D_ℓ, cell counts, times, c_0, T) . . . 50
5.3 Layer list handed to Code C . . . 51
5.4 Main run: load, mesh, run, write `timeseries.csv` and `final_profile.csv` 52
5.5 Console report of the FV run . . . 53
5.6 FV against series comparison . . . 54
5.7 Floor warning . . . 54
5.8 Richardson time extrapolation . . . 55
5.9 Spatial and temporal refinement study . . . 56
5.10 Observed order . . . 57
5.11 Resolution test . . . 58
5.12 Study cost and guard . . . 59
5.13 One exact value (helper for tests and tools) . . . 60
5.14 Output files and figures (`comparison`, `convergence_study`, SVG plots) . . 60
5.15 Temporal order against a same-mesh reference; relative error; check that $t_{\text{end}}/\Delta t_{\text{in}}$ is an integer . . . 62

6 Numerical properties 62

7 Known inconsistencies 67

8 References 73

Purpose. This document is the short form of the mathematics behind STRATO v1.0.0, the Rust program that computes the fractional release of a fission product from one TRISO fuel particle. For every operation of the program it puts three descriptions of one computation side by side: the final mathematical formulation, a short pseudocode and the actual Rust excerpt with a link to the tagged source. The three describe the same logic, step for step. No derivation is repeated here.

Version. 1.0.0, repository tag `v1.0.0` (commit `8c5d8a8`) of https://github.com/weewzr/STRATO. **Date.** 2026-11-06.

Companion document. Every equation number in this document is the number of the same equation in STRATO_Full_Mathematical_Derivation.md (called the Full Derivation below). A reference such as Eq. (5.14) means the equation tagged (5.14) there, and the Full Derivation shows where it comes from. Table 0.1 states how the two documents differ.

Table 0.1 Differences between the two documents.

Question	Full Derivation	Final Deliverable
Reader	A reader with school algebra and the basic ideas of derivatives and integrals	A reader who wants the finished formulation and its code
Content	Physical foundation, conservation law, conditions, both solutions, numerical properties, link to the code	The final equation, the pseudocode and the Rust excerpt of every operation, one table of properties, one table of inconsistencies
Style	Every transformation in its own step with the reason, every symbol with its SI unit	Tables as the main layout, short sentences, symbol meaning given in the mapping sentence under each excerpt
Proofs	Written out	Only named, with the status of each property in Table 6.1 and a pointer to the section of the Full Derivation

Contents. 1 Scope and model summary; 2 Notation; 3 Finite-volume solver (Code B); 4 Analytical solver (Code C); 5 Program flow and comparison (Code A); 6 Numerical properties; 7 Known inconsistencies; 8 References.

Conventions. The three classifications of an operation are IMPLEMENTED (a line of Rust computes exactly this expression, and a pinned link gives the line range), DERIVED NOT IMPLEMENTED (the expression follows from the mathematics but no line evaluates it) and FUTURE IMPROVEMENT (absent from v1.0.0). The three statuses of a numerical property are PROVED (the proof is in the Full Derivation under stated assumptions), DEMONSTRATED NUMERICALLY (a test, the output of `examples/verify.rs` or a check script shows it for specific cases) and CLAIMED ONLY (stated in a comment or a document, neither proved nor shown beyond one case). Links such as src/numerical/finite_volume.rs, lines 50 to 99 point into the tagged source, so they stay valid when the repository moves on. The Rust cell index `i` runs from 0 while the equations use the cell number $j = i + 1$, and the Rust layer index `mat` runs from 0 while the equations use the layer number $\ell = \texttt{mat} + 1$. Excerpts are copied unchanged from the tag. A line `// ...` marks omitted lines.

1 Scope and model summary

STRATO v1.0.0 models one intact, perfectly spherical particle of five concentric layers: kernel, buffer, inner pyrolytic carbon (IPyC), silicon carbide (SiC) and outer pyrolytic carbon (OPyC). Each layer has one constant diffusivity D_ℓ. At time zero the kernel holds a uniform concentration c_0 and the coatings are empty. Substance diffuses outward and disappears at the outer surface. The program reports the fractional release $F(t)$ in two independent ways, and compares them. Code B is a conservative finite-volume scheme with Backward Euler time stepping. Code C is a Sturm-Liouville eigenfunction series. Code A is the driver, the configuration reader, the comparison and the refinement study.

Table 1.1 lists the equations that define the problem. All numbers refer to the Full Derivation.

Table 1.1 The problem solved by STRATO v1.0.0.

Item	Final formulation	Eq.
Layer equation	$\partial c_\ell/\partial t = \frac{D_\ell}{r^2}\,\partial_r(r^2\,\partial_r c_\ell)$ for $R_{\ell-1} < r < R_\ell$	(2.7)
Centre	$\partial_r c_1(0,t) = 0$ and $c_1(0,t)$ finite	(3.1)
Interfaces	$c_\ell(R_\ell^-,t) = c_{\ell+1}(R_\ell^+,t)$ and $D_\ell\,\partial_r c_\ell(R_\ell^-,t) = D_{\ell+1}\,\partial_r c_{\ell+1}(R_\ell^+,t)$	(3.2), (3.3)
Outer surface	$c_5(R,t) = 0$	(3.4)
Initial state	$c(r,0) = c_0$ for $0 \le r < R_1$, and 0 for $R_1 < r < R$	(3.5)
Initial inventory	$M^0 = \frac{4\pi}{3} R_1^3 c_0$	(3.6)
Release rate	$\Phi_R(t) = -4\pi R^2 D_5\,\partial_r c_5(R,t)$	(3.7)
Fractional release	$F(t) = M_{\text{rel}}(t)/M^0 = 1 - M(t)/M^0$	(3.9)

Table 1.2 lists the eight assumptions of v1.0.0. The “Where in the code” column gives the lines that realise or enforce the assumption.

Table 1.2 Assumptions of v1.0.0. A3 is a simplification, because measured partition coefficients differ from 1 (IAEA, 1997).

ID	Assumption	Equation	Where in the code
A1	Spherical symmetry: the concentration depends on r and t only	(2.7)	The mesh and the series use the radius only
A2	Fickian diffusion with one constant, isotropic $D_\ell > 0$ per layer, independent of c, t and temperature	(1.4), (2.7)	Positive D_ℓ is checked at src/config.rs, lines 89 to 99
A3	Perfect contact at the four interfaces: equal concentration and equal flow, partition factor $\gamma_\ell = 1$	(3.2), (3.3)	src/numerical/finite_volume.rs, line 86 and src/analytical/sturm_liouville.rs, lines 98 to 100
A4	No source and no radioactive decay after $t = 0$	(2.6) with $S = 0$	`rhs = v * old` at src/numerical/finite_volume.rs, line 179
A5	Absorbing outer surface: anything that reaches $r = R$ is removed at once	(3.4)	src/numerical/finite_volume.rs, lines 88 to 91
A6	Kernel-only start: uniform c_0 in the kernel, empty coatings, recoil deposits ignored	(3.5)	src/numerical/finite_volume.rs, lines 101 to 112
A7	Fixed, intact geometry: radii do not change, no layer cracks, the kernel stays centred	(1.1)	Radii checked at src/config.rs, lines 89 to 99
A8	Temperature is read and checked, and used in no formula	none	src/config.rs, lines 86 to 88

The input `inputs/synthetic/synthetic_five_layer.toml` is a numerical benchmark. It does not represent any real TRISO particle or fission product, and every number quoted from it in this document is calculated.

2 Notation

Table 2.1 lists the symbols that occur in the equations of Sections 3 to 5. Units are SI. The word arbitrary marks the scale fixed by the shooting start $q(0) = 1$ (Eq. (4.9));

the weights w_n and the release fraction F do not depend on that scale. The full list is in Tables 0.1 to 0.4 of the Full Derivation.

Table 2.1 Notation used in the equations of Sections 3 to 5.

Symbol	Meaning	Unit	Rust name
r, t	radial distance from the particle centre; time	m; s	`r`; `t`
ℓ	layer index: 1 Kernel, 2 Buffer, 3 IPyC, 4 SiC, 5 OPyC	1	`mat` ($= \ell - 1$)
j	cell index, $j = 1, \dots, N_c$; the face between cells j and $j+1$ is $j + 1/2$	1	`i` ($= j - 1$)
R_ℓ, R	outer radius of layer ℓ with $R_0 = 0$; particle radius $R = R_5$	m	`outer_radius_m`
L_ℓ	layer thickness $R_\ell - R_{\ell-1}$	m	`length`
D_ℓ	diffusivity of layer ℓ	$\mathrm{m^2\ s^{-1}}$	`diffusivity_m2_s`
c, c_0	concentration; initial kernel concentration	$\mathrm{mol\ m^{-3}}$	`c`; `kernel_concentration_mol_m3`
M, M^0, M_{rel}	inventory; initial inventory; cumulative released amount	mol	`inventory`; `initial`; `released`
F	fractional release M_{rel}/M^0	1	`release_fraction`
Φ_R	release rate through the outer surface	$\mathrm{mol\ s^{-1}}$	`release_rate`
n_ℓ, N_c	number of cells in layer ℓ; total number of cells	1	`cells`; `mesh.cells.len()`
w_ℓ	uniform cell width in layer ℓ, L_ℓ/n_ℓ	m	`width`
$r_{j-1/2}$, $r_{j+1/2}$, r_j	inner face, outer face and volume centroid of cell j	m	`left`, `right`, `centroid`
V_j, $A_{j+1/2}$	cell volume; face area	$\mathrm{m^3}$; $\mathrm{m^2}$	`volume`; `area`
$\delta^L_{j+1/2}$, $\delta^R_{j+1/2}$	distances from the two neighbouring centroids to the face	m	`dl`, `dr`
$G_{j+1/2}$, G_R	internal face conductance; outer-boundary conductance	$\mathrm{m^3\ s^{-1}}$	`conductance[j-1]`; `outer_conductance`
g^W_j, g^E_j	inner (west) and outer (east) conductance of cell j	$\mathrm{m^3\ s^{-1}}$	`gw[i]`, `ge[i]`
v_j	$V_j/\Delta t$	$\mathrm{m^3\ s^{-1}}$	`v`
C^n_j	concentration of cell j at time level n	$\mathrm{mol\ m^{-3}}$	`c`
Δt_{in}, t_{end}, Δt, N_t	requested step; end time; step actually used; number of steps	s; s; s; 1	`dt_s`; `end_time_s`; `dt`; `steps`
ε^n	balance residual $M^n + M^n_{\mathrm{rel}} - M^0$	mol	`balance_residual`
Λ_n	n-th eigenvalue (decay constant), ascending	$\mathrm{s^{-1}}$	`eigenvalues`
$u = r\phi$, $q = D\,du/dr$	transformed eigenfunction and flux-like state	arbitrary $\times$ m; arbitrary $\times\ \mathrm{m^2\ s^{-1}}$	`u`, `q`
k_ℓ	wavenumber $\sqrt{\Lambda/D_\ell}$	$\mathrm{m^{-1}}$	`k`
$\mathcal{N}(\Lambda)$	Sturm count, number of eigenvalues $\le \Lambda$	1	`eigenvalue_count`

Symbol	Meaning	Unit	Rust name
$I_{1,n}$, I_n, N_n	kernel moment, whole-particle moment and norm of mode n	arbitrary $\times$ m^3; arbitrary $\times$ m^3; arbitrary^2 $\times$ m^3	`i1`, `moment`, `norm`
w_n	modal weight $3I_{1,n}I_n/(R_1^3 N_n)$	1	`weights`
K, $K_{\max}$	number of modes kept; mode cap 20 000	1	`eigenvalues.len()`; `MAX_MODES`
$\Lambda_{\max}$, Λ_{cov}	requested and covered upper eigenvalue	s^{-1}	`lambda_max`; `lambda_covered`
t_{valid}	earliest time at which the series is used, $40/\Lambda_{\text{cov}}$	s	`min_valid_time()`
ρ, ε_F	flux-cancellation ratio; accuracy floor of the series on F	1; 1	`min_flux_ratio`; `accuracy_floor()`
$e(t_n)$, p_k	absolute difference between the two methods; observed order at refinement level k	1; 1	`abs_error`; `orders`

3 Finite-volume solver (Code B)

Code B (`src/numerical/finite_volume.rs`) solves the layer equation (2.7) with a conservative finite-volume scheme. The sphere is cut into spherical-shell cells, and each cell holds one concentration. Neighbouring cells exchange substance through the face between them with the series conductance of Eq. (5.9) (Patankar, 1980). Backward Euler advances the cell values in time, and a tridiagonal elimination without subtraction (Higham, 2002, for the rounding model) finds the new values at each step. The release through the outer face is summed with Kahan compensation (Kahan, 1965). Table 3.1 lists the 22 operations B.1 to B.22 in the order of the computation. Rows B.21 and B.22 are not computed in v1.0.0. They are listed so that the limits of the code are visible.

Table 3.1 Operations of Code B in the order of the computation. Row B.1 is described in Section 3.1, row B.22 in Section 3.22.

No.	Final mathematical formulation	Pseudocode	Corresponding Rust implementation
B.1	$w_\ell = L_\ell/n_\ell$, $r_{j-1/2} = R_{\ell-1} + \iota\, w_\ell$; last outer face $= R_\ell$, Eq. (5.2)	`w = (R[l] - R[l-1]) / n[l]`; outer face of the last cell of a layer is `R[l]`	`build_mesh`, src/numerical/finite_volume.rs, lines 50 to 73. IMPLEMENTED
B.2	$V(a,b) = \frac{4\pi}{3}(b^3 - a^3)$, Eq. (1.6); $V_j = V(r_{j-1/2}, r_{j+1/2})$, Eq. (5.3)	`RETURN (4 pi / 3) (b^3 - a^3); V[j] = shell_volume(inner, outer)`	`shell_volume`, src/numerical/finite_volume.rs, lines 38 to 40. IMPLEMENTED
B.3	$r_j = \frac{3}{4}\frac{r_{j+1/2}^4 - r_{j-1/2}^4}{r_{j+1/2}^3 - r_{j-1/2}^3}$, $r_1 = \frac{3}{4} r_{3/2}$, Eq. (5.4)	`IF inner = 0: RETURN 0.75 outer; ELSE RETURN 0.75 (b^4 - a^4) / (b^3 - a^3)`	`volume_centroid`, src/numerical/finite_volume.rs, lines 42 to 48. IMPLEMENTED
B.4	$A_{j+1/2} = 4\pi r_{j+1/2}^2$, Eq. (5.5)	`A_face = 4 pi rf^2` for each internal face and for the face at `R`	`build_mesh`, src/numerical/finite_volume.rs, lines 82 to 83. IMPLEMENTED

No.	Final mathematical formulation	Pseudocode	Corresponding Rust implementation
B.5	$\delta^L_{j+1/2} = r_{j+1/2} - r_j$, $\delta^R_{j+1/2} = r_{j+1} - r_{j+1/2}$, Eq. (5.7)	`deltaL = rf - r[j]; deltaR = r[j+1] - rf`	`build_mesh`, src/numerical/finite_volume.rs, lines 84 to 85. IMPLEMENTED
B.6	$G_{j+1/2} = \frac{A_{j+1/2}}{\delta^L_{j+1/2}/D_j + \delta^R_{j+1/2}/D_{j+1}}$, Eq. (5.9)	`G[j] = A_face / (deltaL/D[j] + deltaR/D[j+1])`	`build_mesh`, src/numerical/finite_volume.rs, lines 78 to 87. IMPLEMENTED
B.7	$G_R = \frac{4\pi R^2 D_5}{R - r_{N_c}}$, Eq. (5.11)	`G_R = A_R / ((R - r[Nc]) / D[5])`	`build_mesh`, src/numerical/finite_volume.rs, lines 88 to 91. IMPLEMENTED
B.8	$g^W_1 = 0$, Eq. (5.10)	store no conductance at `r = 0`; set `gW[1] = 0` when the step is assembled	`backward_euler_step`, src/numerical/finite_volume.rs, line 172. IMPLEMENTED
B.9	$C^0_j = c_0$ in layer 1, else 0; $M^0 = \sum_j V_j C^0_j$, Eq. (5.20)	`C[j] = c0` for kernel cells, `0` otherwise; `M0 = sum of V[j] C[j]`	`initial_concentration`, src/numerical/finite_volume.rs, lines 101 to 112. IMPLEMENTED
B.10	$M^n = \sum_{j=1}^{N_c} V_j C^n_j$, Eq. (5.21)	`M = 0; M = M + V[j] C[j]` for each cell; `RETURN M`	`inventory`, src/numerical/finite_volume.rs, lines 114 to 120 and 2 more ranges (Section 3.10). IMPLEMENTED
B.11	$N_t = \lceil t_{\text{end}}/\Delta t_{\text{in}} \rceil$, $\Delta t = t_{\text{end}}/N_t$, Eq. (5.16)	`Nt = ceil(t_end / dt_in); dt = t_end / Nt`	`run`, src/numerical/finite_volume.rs, lines 190 to 191. IMPLEMENTED
B.12	$-g^W_j C^{n+1}_{j-1} + (v_j + g^W_j + g^E_j) C^{n+1}_j - g^E_j C^{n+1}_{j+1} = v_j C^n_j$, Eq. (5.14)	`v = V / dt; diag = v + gW + gE; rhs = v C[j]`	`backward_euler_step`, src/numerical/finite_volume.rs, lines 162 to 182. IMPLEMENTED
B.13	$e_j = v_j + \frac{g^W_j e_{j-1}}{e_{j-1} + g^W_j}$, $d_j = e_j + g^E_j$, Eq. (5.18); $y_j = \frac{\text{rhs}_j + g^W_j y_{j-1}}{d_j}$, Eq. (5.19)	`e[j] = v[j] + gW[j] e[j-1] / (e[j-1] + gW[j]); y[j] = (rhs[j] + gW[j] y[j-1]) / d[j]`	`solve_diffusion_system`, src/numerical/finite_volume.rs, lines 132 to 160. IMPLEMENTED
B.14	$C^{n+1}_{N_c} = y_{N_c}$, $C^{n+1}_j = y_j + \frac{g^E_j}{d_j} C^{n+1}_{j+1}$, Eq. (5.19)	`Cnew[Nc] = y[Nc]`; for `j = Nc-1` down to 1: `Cnew[j] = y[j] + gE[j] / d[j] Cnew[j+1]`	`solve_diffusion_system`, src/numerical/finite_volume.rs, lines 154 to 158. IMPLEMENTED
B.15	$\Phi^{n+1}_R = G_R C^{n+1}_{N_c}$, Eq. (5.22)	`Phi_R = G_R Cnew[Nc]`	`run`, src/numerical/finite_volume.rs, line 202 and 1 more range (Section 3.15). IMPLEMENTED
B.16	$\upsilon = \Delta t\, \Phi_R - \kappa$, $\Sigma = M_{\text{rel}} + \upsilon$, $\kappa \leftarrow (\Sigma - M_{\text{rel}}) - \upsilon$, $M_{\text{rel}} \leftarrow \Sigma$, Eq. (5.23)	`upsilon = dt Phi_R - kappa; Sigma = Mrel + upsilon;` update `kappa` and `Mrel`	`run`, src/numerical/finite_volume.rs, lines 211 to 217. IMPLEMENTED

No.	Final mathematical formulation	Pseudocode	Corresponding Rust implementation
B.17	$\varepsilon^n = M^n + M^n_{\text{rel}} - M^0$, Eq. (5.24)	`eps = M + Mrel - M0`	`run`, src/numerical/finite_volume.rs, lines 219 to 220. IMPLEMENTED
B.18	$F^n = M^n_{\text{rel}}/M^0$, Eq. (5.24); equivalently $F = 1 - M/M^0$, Eq. (3.9)	`F = Mrel / M0`	`run`, src/numerical/finite_volume.rs, line 186 and 1 more range (Section 3.18). IMPLEMENTED
B.19	$n \bmod n_{\text{out}} = 0$ or $n = N_t$, $t_n = n\,\Delta t$, Eq. (5.25)	save an output row when `n mod n_out = 0` or `n = Nt`	`run`, src/numerical/finite_volume.rs, lines 196 to 206 and 1 more range (Section 3.19). IMPLEMENTED
B.20	$\Delta t_{\text{FE}} = \min_j \dfrac{V_j}{g_j^W + g_j^E}$, Eq. (6.6)	`ratio[j] = V[j] / (gW[j] + gE[j]); dt_FE = smallest ratio;` print once	`explicit_dt_bound`, src/numerical/finite_volume.rs, lines 238 to 252 and 1 more range (Section 3.20). IMPLEMENTED (diagnostic only)
B.21	$\Delta t \le 2/\mu_{\max}$, Eq. (6.7); not computed	none; the code does not form the scaled matrix	none. DERIVED NOT IMPLEMENTED
B.22	source S, Eq. (2.6), and Robin outer face, Eq. (3.10); absent from v1.0.0	none; `S = 0` and the outer face is a perfect sink	none. FUTURE IMPROVEMENT

Each operation below repeats the equation, the pseudocode and the Rust excerpt of its row in Table 3.1.

3.1 Cell faces, uniform width per layer, last face set to R_ℓ

Operation B.1. Classification: IMPLEMENTED. Equations: (1.1), (1.2), (5.2). Rust function: `build_mesh`. Pinned lines: src/numerical/finite_volume.rs, lines 50 to 73.

Final equation.

$$0 = R_0 < R_1 < R_2 < R_3 < R_4 < R_5 = R \tag{1.1}$$

$$L_\ell = R_\ell - R_{\ell-1}, \qquad \ell = 1, \dots, 5 \tag{1.2}$$

$$\begin{aligned} w_\ell &= \frac{R_\ell - R_{\ell-1}}{n_\ell} = \frac{L_\ell}{n_\ell}, \\ r_{j-1/2} &= R_{\ell-1} + \iota\, w_\ell, \\ r_{j+1/2} &= \begin{cases} R_{\ell-1} + (\iota+1)\, w_\ell, & \iota < n_\ell - 1, \\ R_\ell, & \iota = n_\ell - 1. \end{cases} \end{aligned} \tag{5.2}$$

The first equation set (1.1) and (1.2) gives the layer radii and thicknesses that feed the mesh. The right face of the last cell of a layer is set to R_ℓ itself.

Pseudocode.

```
GIVEN  radii R[1..5] that obey 0 = R[0] < R[1] < ... < R[5]  # Eq. (1.1): layers stack outward
GIVEN  cell counts n[1..5]                                   # one whole number per layer
FOR each layer l = 1 to 5:
    # Eq. (1.2): thickness of the layer
    1. L[l] = R[l] - R[l-1]
    # Eq. (5.2): all cells of a layer share one width
    2. w[l] = L[l] / n[l]
    # i counts cells inside this layer from 0
    3. FOR offset i = 0 to n[l]-1:
         3a. inner_face = R[l-1] + i * w[l]
         3b. IF i < n[l]-1:
                 outer_face = R[l-1] + (i+1) * w[l]
             ELSE:
                 # last cell ends exactly on the layer radius,
                 # so no rounding gap is left at the interface
                 outer_face = R[l]
         3c. store (inner_face, outer_face) as the next cell
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 53 to 62 and 72 to 73
    let mut inner = 0.0;
    for (mat, region) in cfg.regions.iter().enumerate() {
        let width = (region.outer_radius_m - inner) / region.cells as f64;
        for k in 0..region.cells {
            let left = inner + k as f64 * width;
            let right = if k + 1 == region.cells {
                region.outer_radius_m
            } else {
                inner + (k + 1) as f64 * width
            };
        // ...
        inner = region.outer_radius_m;
    }
```

Mapping of symbols to the excerpt: ℓ (1 to 5) is `mat` (0 to 4, so $\ell =$ `mat` $+ 1$); R_ℓ is `region.outer_radius_m`; $R_{\ell-1}$ is `inner` (starts at 0.0, line 53; set to R_ℓ at line 72); n_ℓ is `region.cells`; L_ℓ: no variable; the numerator `region.outer_radius_m - inner`; w_ℓ is `width`; ι is `k`; $r_{j-1/2}$ is `left` (field `Cell::left`); $r_{j+1/2}$ is `right` (field `Cell::right`); j is `i`, with $j = i + 1$; the pushes into `cells` happen in cell order; R is `region.outer_radius_m` of the last region.

The branch at line 58 is the exact assignment of the last right face to R_ℓ in Eq. (5.2).

3.2 Cell volume

Operation B.2. Classification: IMPLEMENTED. Equations: (1.6), (5.3). Rust function: `shell_volume`. Pinned lines: src/numerical/finite_volume.rs, lines 38 to 40.

Final equation.

$$V(a, b) = \frac{4\pi}{3}\left(b^3 - a^3\right) \tag{1.6}$$

$$V_j = \frac{4\pi}{3}\left(r_{j+1/2}^3 - r_{j-1/2}^3\right). \tag{5.3}$$

Pseudocode.

```
DEFINE volume_of_shell(a, b):
    # Eq. (1.6): big ball minus small ball
    1. RETURN (4 * pi / 3) * (b^3 - a^3)

FOR each cell j = 1 to Nc:
    # Eq. (5.3): a cell is one thin shell
    2. V[j] = volume_of_shell(inner_face of j, outer_face of j)
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 38 to 40
pub fn shell_volume(a: f64, b: f64) -> f64 {
    4.0 * PI / 3.0 * (b.powi(3) - a.powi(3))
}
```

Mapping of symbols to the excerpt: $V(a, b)$ is return value of `shell_volume`; a, b is `a`, `b`; π is `PI` (`std::f64::consts::PI`); V_j is field `Cell::volume`, set by `shell_volume(left, right)`; $r_{j-1/2}$, $r_{j+1/2}$ is `left`, `right` passed as `a`, `b`.

3.3 Volume centroid

Operation B.3. Classification: IMPLEMENTED. Equations: (5.4). Rust function: `volume_centroid`. Pinned lines: src/numerical/finite_volume.rs, lines 42 to 48.

Final equation.

$$r_j = \frac{3}{4}\,\frac{r_{j+1/2}^4 - r_{j-1/2}^4}{r_{j+1/2}^3 - r_{j-1/2}^3} \quad (j \geq 2), \qquad r_1 = \frac{3}{4}\,r_{3/2}. \tag{5.4}$$

The first cell has $r_{1/2} = 0$, which gives $r_1 = \frac{3}{4} r_{3/2}$.

Pseudocode.

```
FOR each cell j = 1 to Nc:
    IF j = 1:
        1. r[1] = (3/4) * outer_face of cell 1       # inner face is 0, so the general
                                                     # formula collapses to this
    ELSE:
        2. a = inner_face of j,  b = outer_face of j
        # Eq. (5.4): the volume-weighted mean radius
        # of the shell, used as the cell's position
        3. r[j] = (3/4) * (b^4 - a^4) / (b^3 - a^3)
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 42 to 48
pub fn volume_centroid(a: f64, b: f64) -> f64 {
    if a == 0.0 {
        0.75 * b
    } else {
        0.75 * (b.powi(4) - a.powi(4)) / (b.powi(3) - a.powi(3))
    }
}
```

Mapping of symbols to the excerpt: r_j is return value of `volume_centroid`; stored in field `Cell::centroid`; $r_{j-1/2}$, $r_{j+1/2}$ is `a`, `b`; r_1 and $r_{3/2}$ is `0.75 * b` with `b` the outer face of cell 1 (branch `a == 0.0`).

The general formula at line 46 gives the same value at $a = 0$; the branch only skips the terms in a.

3.4 Face area

Operation B.4. Classification: IMPLEMENTED. Equations: (1.5), (5.5). Rust function: `build_mesh`. Pinned lines: src/numerical/finite_volume.rs, lines 82 to 83.

Final equation.

$$A(r) = 4\pi r^2 \tag{1.5}$$

$$A_{j+1/2} = 4\pi r_{j+1/2}^2. \tag{5.5}$$

Pseudocode.

```
FOR each face between cell j and cell j+1 (j = 1 to Nc-1), and the face at R:
    1. rf = radius of the face
    2. A_face[j] = 4 * pi * rf^2  # Eqs. (1.5), (5.5): area of a sphere of radius rf
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 79 to 83
    for pair in cells.windows(2) {
        let left = &pair[0];
        let right = &pair[1];
        let face = left.right;
        let area = 4.0 * PI * face * face;
```

Mapping of symbols to the excerpt: $A_{j+1/2}$ is `area`; $r_{j+1/2}$ is `face`, which is `left.right`; cell j, cell $j+1$ is `left` (`pair[0]`), `right` (`pair[1]`); π is `PI`.

The loop variable `pair` runs over `cells.windows(2)`, one pair per internal face (line 79). The row lines are 82 to 83; lines 79 to 81 are shown so that `left` and `right` are defined.

3.5 Half-cell distances to the face

Operation B.5. Classification: IMPLEMENTED. Equations: (5.7). Rust function: `build_mesh`. Pinned lines: src/numerical/finite_volume.rs, lines 84 to 85.

Final equation.

$$\delta_{j+1/2}^{L} = r_{j+1/2} - r_j, \qquad \delta_{j+1/2}^{R} = r_{j+1} - r_{j+1/2}. \tag{5.7}$$

Pseudocode.

```
FOR each internal face j+1/2 (between cell j and cell j+1):
    1. deltaL[j] = rf - r[j]    # Eq. (5.7): from the centroid of cell j to the face
    2. deltaR[j] = r[j+1] - rf  # Eq. (5.7): from the face to the centroid of cell j+1
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 80 to 85
        let left = &pair[0];
        let right = &pair[1];
        let face = left.right;
        let area = 4.0 * PI * face * face;
        let dl = face - left.centroid;
        let dr = right.centroid - face;
```

Mapping of symbols to the excerpt: $\delta_{j+1/2}^{L}$ is `dl`; $\delta_{j+1/2}^{R}$ is `dr`; $r_{j+1/2}$ is `face`; r_j is `left.centroid`; r_{j+1} is `right.centroid`.

The row lines are 84 to 85; lines 80 to 83 are shown so that `left`, `right` and `face` are defined.

3.6 Internal face conductance (series resistors)

Operation B.6. Classification: IMPLEMENTED. Equations: (5.8), (5.9). Rust function: `build_mesh`. Pinned lines: src/numerical/finite_volume.rs, lines 78 to 87.

Final equation.

$$J_{j+1/2} = D_j \frac{C_j - C_{j+1/2}}{\delta^L_{j+1/2}} = D_{j+1} \frac{C_{j+1/2} - C_{j+1}}{\delta^R_{j+1/2}}. \tag{5.8}$$

$$A_{j+1/2} J_{j+1/2} = G_{j+1/2} \, (C_j - C_{j+1}), \qquad G_{j+1/2} = \frac{A_{j+1/2}}{\delta^L_{j+1/2}/D_j + \delta^R_{j+1/2}/D_{j+1}}. \tag{5.9}$$

Eq. (5.8) is the flux through the face written from each side; eliminating the face value $C_{j+1/2}$ gives Eq. (5.9). The code uses Eq. (5.9) at every internal face.

Pseudocode.

```
FOR each internal face j+1/2:
    # How Eq. (5.9) comes from Eq. (5.8):
    # Eq. (5.8), left side of the face
    1. J from the left  : J = D[j]   * (C[j]  - Cf)   / deltaL[j]
    # Eq. (5.8), right side of the face
    2. J from the right : J = D[j+1] * (Cf - C[j+1])  / deltaR[j]
    # what leaves cell j arrives in cell j+1
    3. Both are the same J
    4. Solve each for the drop in concentration:
         C[j] - Cf    = J * deltaL[j] / D[j]
         Cf - C[j+1]  = J * deltaR[j] / D[j+1]
    5. Add them: the unknown face value Cf cancels
         C[j] - C[j+1] = J * (deltaL[j]/D[j] + deltaR[j]/D[j+1])
    6. Multiply by the face area:  A_face * J = G * (C[j] - C[j+1])  # Eq. (5.9)

    # What the program computes (step 6 solved for G):
    # Eq. (5.9): two resistors in series
    7. G[j] = A_face[j] / ( deltaL[j]/D[j] + deltaR[j]/D[j+1] )
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 78 to 87
    let mut conductance = Vec::with_capacity(cells.len() - 1);
    for pair in cells.windows(2) {
        let left = &pair[0];
        let right = &pair[1];
        let face = left.right;
        let area = 4.0 * PI * face * face;
        let dl = face - left.centroid;
        let dr = right.centroid - face;
        conductance.push(area / (dl / left.diffusivity + dr / right.diffusivity));
    }
```

Mapping of symbols to the excerpt: $G_{j+1/2}$ is element pushed onto `conductance`, read back as `mesh.conductance[j - 1]` (0-based `conductance[i]` for the face between cells `i` and `i+1`); $A_{j+1/2}$ is `area`; $\delta^L_{j+1/2}$, $\delta^R_{j+1/2}$ is `dl`, `dr`; D_j is `left.diffusivity`; D_{j+1} is `right.diffusivity`; $J_{j+1/2}$, $C_{j+1/2}$: no variable (the face value is eliminated on paper).

There is no branch on equal diffusivities; the series-resistor form is used at every internal face, including faces inside a layer.

3.7 Outer-boundary conductance (half cell to $c = 0$)

Operation B.7. Classification: IMPLEMENTED. Equations: (3.4), (5.11). Rust function: `build_mesh`. Pinned lines: src/numerical/finite_volume.rs, lines 88 to 91.

Final equation.

$$c_5(R, t) = 0, \qquad t > 0. \tag{3.4}$$

$$G_R = \frac{A(R)}{(R - r_{N_c})/D_5 + 0} = \frac{4\pi R^2 D_5}{R - r_{N_c}}, \qquad \Phi_R \approx G_R\,(C_{N_c} - 0) = G_R\,C_{N_c}. \tag{5.11}$$

Pseudocode.

```
GIVEN  the boundary rule c5(R, t) = 0       # Eq. (3.4): the outside acts as a perfect sink
1. A_R  = 4 * pi * R^2                      # area of the particle surface
2. dist = R - r[Nc]                         # from the last centroid to the surface
# Eq. (5.11): one resistor only; the "+ 0" is the
# surface-side resistor, which a perfect sink does not have
3. G_R  = A_R / (dist / D[5] + 0)
   which is  G_R = 4 * pi * R^2 * D[5] / (R - r[Nc])
4. Later, release rate = G_R * (C[Nc] - 0)  # Eq. (5.11): concentration outside is 0
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 88 to 91
    let last = cells.last().expect("nonempty mesh");
    let outer_radius = cfg.regions.last().expect("five regions").outer_radius_m;
    let outer_area = 4.0 * PI * outer_radius * outer_radius;
    let outer_conductance = outer_area * last.diffusivity / (outer_radius - last.centroid);
```

Mapping of symbols to the excerpt: G_R is `outer_conductance` (local, then field `Mesh::outer_conductance`); R is `outer_radius`; $A(R)$ is `outer_area`; D_5 is `last.diffusivity`; r_{N_c} is `last.centroid`; π is `PI`; $\Phi_R(t)$, C_{N_c} is not in `build_mesh`; see row B.15.

3.8 Centre condition: no flow through $r = 0$

Operation B.8. Classification: IMPLEMENTED. Equations: (3.1), (5.10). Rust function: `backward_euler_step`. Pinned lines: src/numerical/finite_volume.rs, line 172.

Final equation.

$$\frac{\partial c_1}{\partial r}(0, t) = 0, \qquad \lim_{r \to 0^+} 4\pi r^2 D_1 \frac{\partial c_1}{\partial r} = 0, \qquad c_1(0, t) \text{ finite.} \tag{3.1}$$

$$g_1^W = 0. \tag{5.10}$$

The mesh stores no conductance for $r = 0$. The condition enters as $g_1^W = 0$ in the step.

Pseudocode.

```
GIVEN  the centre rule: no flow through r = 0  # Eq. (3.1): flux 4*pi*r^2*D1*dc/dr goes to 0
# the mesh holds only internal faces and the surface
1. Store no conductance for a face at r = 0
2. When the step is assembled, set gW[1] = 0   # Eq. (5.10): cell 1 has no inner neighbour
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 171 to 172
    for i in 0..n {
        gw[i] = if i == 0 { 0.0 } else { mesh.conductance[i - 1] };
```

Mapping of symbols to the excerpt: g_1^W is `gw[0]`, set to `0.0`; g_j^W, $j \geq 2$ is `gw[i]`, equal to `mesh.conductance[i - 1]`; c_1, D_1: no variable; the condition is the absence of a face at $r = 0$.

There is no ghost cell and `mesh.conductance` has no entry for $r = 0$ (`Mesh` holds $N_c - 1$ internal conductances, lines 18 to 24).

3.9 Initial condition: kernel full, coatings empty

Operation B.9. Classification: IMPLEMENTED. Equations: (3.5), (5.20). Rust function: `initial_concentration`. Pinned lines: src/numerical/finite_volume.rs, lines 101 to 112.

Final equation.

$$c(r,0) = \begin{cases} c_0, & 0 \leq r < R_1, \\ 0, & R_1 < r < R. \end{cases} \tag{3.5}$$

$$C_j^0 = \begin{cases} c_0, & \text{cell } j \text{ in layer 1}, \\ 0, & \text{otherwise}, \end{cases} \qquad M^0 = \sum_{j=1}^{N_c} V_j \, C_j^0. \tag{5.20}$$

Pseudocode.

```
FOR each cell j = 1 to Nc:
    IF cell j lies in layer 1 (the kernel):
        1. C[j] = c0               # Eq. (3.5): kernel starts at c0
    ELSE:
        2. C[j] = 0                # Eq. (3.5): all coatings start empty
3. M0 = sum over j of V[j] * C[j]  # Eq. (5.20): initial amount of material
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 101 to 112
pub fn initial_concentration(cfg: &Config, mesh: &Mesh) -> Vec<f64> {
    mesh.cells
        .iter()
        .map(|c| {
            if c.material == 0 {
                cfg.initial.kernel_concentration_mol_m3
            } else {
                0.0
            }
        })
        .collect()
}
```

Mapping of symbols to the excerpt: C_j^0 is element of the returned vector; bound to `c` in `run`; c_0 is `cfg.initial.kernel_concentration_mol_m3`; kernel (layer 1) is `c.material == 0`; coating cells is value `0.0`; M^0 is `initial`, see row B.10.

The kernel is identified by the material index, not by the radius R_1.

3.10 Inventory (also M^0)

Operation B.10. Classification: IMPLEMENTED. Equations: (1.7), (5.20), (5.21). Rust function: `inventory`. Pinned lines: src/numerical/finite_volume.rs, lines 114 to 120; src/numerical/finite_volume.rs, line 186; src/numerical/finite_volume.rs, line 219.

Final equation.

$$M(t) = \int_0^R c(r,t)\, 4\pi r^2\, dr \tag{1.7}$$

$$C_j^0 = \begin{cases} c_0, & \text{cell } j \text{ in layer 1,} \\ 0, & \text{otherwise,} \end{cases} \qquad M^0 = \sum_{j=1}^{N_c} V_j\, C_j^0. \tag{5.20}$$

$$M^n = \sum_{j=1}^{N_c} V_j\, C_j^n. \tag{5.21}$$

Pseudocode.

```
# Target (Eq. 1.7): M = integral of c(r) * 4*pi*r^2 dr, the sum of (concentration x shell
# volume)
GIVEN concentrations C[1..Nc] at time level n
1. M = 0
2. FOR each cell j = 1 to Nc:
       M = M + V[j] * C[j]  # Eq. (5.21): in each cell the integral becomes
                            # (cell concentration) x (cell volume)
3. RETURN M                 # applied to the initial values this is M0, Eq. (5.20)
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 114 to 120, 186 and 219
pub fn inventory(mesh: &Mesh, c: &[f64]) -> f64 {
    mesh.cells
        .iter()
        .zip(c)
        .map(|(cell, &x)| cell.volume * x)
        .sum()
}
    // ...
    let initial = inventory(mesh, &c);
        // ...
        let inv = inventory(mesh, &c);
```

Mapping of symbols to the excerpt: M^n is return value of `inventory`; bound to `inv`; M^0 is `initial`; V_j is `cell.volume`; C_j^n is `x` (the closure variable), element of the slice `c`; N_c is `mesh.cells.len()`.

Line 186 is inside `run`; lines 114 to 120 are the function `inventory`. The sum runs in cell order.

3.11 Time step actually used

Operation B.11. Classification: IMPLEMENTED. Equations: (5.16). Rust function: `run`. Pinned lines: src/numerical/finite_volume.rs, lines 190 to 191.

Final equation.

$$N_t = \left\lceil \frac{t_{\text{end}}}{\Delta t_{\text{in}}} \right\rceil, \qquad \Delta t = \frac{t_{\text{end}}}{N_t} \le \Delta t_{\text{in}}. \tag{5.16}$$

Pseudocode.

```
GIVEN  requested step dt_in and end time t_end
1. Nt = ceil(t_end / dt_in)       # Eq. (5.16): whole number of steps, rounded up
2. dt = t_end / Nt                # Eq. (5.16): step stretched to land exactly on t_end
3. dt is never larger than dt_in  # Eq. (5.16): dt <= dt_in
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 190 to 191
    let steps = (cfg.time.end_time_s / cfg.time.dt_s).ceil() as usize;
    let dt = cfg.time.end_time_s / steps as f64;
```

Mapping of symbols to the excerpt: N_t is `steps`; Δt_{in} is `cfg.time.dt_s`; t_{end} is `cfg.time.end_time_s`; Δt is `dt`; $\lceil \cdot \rceil$ is `.ceil()` followed by `as usize`.

3.12 Backward Euler coefficients

Operation B.12. Classification: IMPLEMENTED. Equations: (5.12), (5.13), (5.14), (5.15). Rust function: `backward_euler_step`. Pinned lines: src/numerical/finite_volume.rs, lines 162 to 182.

Final equation.

$$V_j \frac{dC_j}{dt} = g_j^W \left(C_{j-1} - C_j\right) + g_j^E \left(C_{j+1} - C_j\right), \qquad j = 1, \dots, N_c, \quad C_{N_c+1} = 0. \tag{5.12}$$

$$V_j \frac{C_j^{n+1} - C_j^n}{\Delta t} = g_j^W \left(C_{j-1}^{n+1} - C_j^{n+1}\right) + g_j^E \left(C_{j+1}^{n+1} - C_j^{n+1}\right), \qquad C_{N_c+1}^{n+1} = 0. \tag{5.13}$$

$$-g_j^W C_{j-1}^{n+1} + \left(v_j + g_j^W + g_j^E\right) C_j^{n+1} - g_j^E C_{j+1}^{n+1} = v_j C_j^n = \text{rhs}_j. \tag{5.14}$$

$$\left(\frac{\mathbf{V}}{\Delta t} + \mathbf{A}\right) \mathbf{C}^{n+1} = \frac{\mathbf{V}}{\Delta t} \mathbf{C}^n. \tag{5.15}$$

The code assembles the coefficients of Eq. (5.14) cell by cell and never forms the matrix of Eq. (5.15).

Pseudocode.

```
# Idea (Eq. 5.12): rate of change of the amount in cell j = inflow from inside + inflow from
# outside
# Backward Euler (Eq. 5.13): evaluate the right-hand side with the NEW values at level n+1
GIVEN old values C[j] (level n), dt, V[j], gW[j], gE[j]
   with gW[1] = 0 and gE[Nc] = G_R and the value just outside cell Nc equal to 0
FOR each cell j = 1 to Nc:
    1. v[j]    = V[j] / dt
    2. lower[j]= - gW[j]              # multiplies the new value of cell j-1 (Eq. 5.14)
    3. diag[j] = v[j] + gW[j] + gE[j]  # multiplies the new value of cell j (Eq. 5.14)
    4. upper[j]= - gE[j]              # multiplies the new value of cell j+1 (Eq. 5.14)
    5. rhs[j]  = v[j] * C[j]          # Eq. (5.14): old amount per unit time
# The equations lower[j]*Cnew[j-1] + diag[j]*Cnew[j] + upper[j]*Cnew[j+1] = rhs[j]
# are Eq. (5.14), one per cell.
```

Implementation detail: Eq. (5.15) is the same system written as one matrix. The code assembles the coefficients of Eq. (5.14) cell by cell and never forms that matrix.

Rust.

```rust
// src/numerical/finite_volume.rs, lines 162 to 182
pub fn backward_euler_step(mesh: &Mesh, old: &[f64], dt: f64) -> Result<Vec<f64>> {
    let n = mesh.cells.len();
    if old.len() != n || dt <= 0.0 || !dt.is_finite() {
        bail!("invalid state or timestep");
    }
    let mut v = vec![0.0; n];
    let mut gw = vec![0.0; n];
    let mut ge = vec![0.0; n];
    let mut rhs = vec![0.0; n];
    for i in 0..n {
        gw[i] = if i == 0 { 0.0 } else { mesh.conductance[i - 1] };
        ge[i] = if i + 1 == n {
            mesh.outer_conductance
        } else {
            mesh.conductance[i]
        };
        v[i] = mesh.cells[i].volume / dt;
        rhs[i] = v[i] * old[i];
    }
    solve_diffusion_system(&v, &gw, &ge, &rhs)
}
```

Mapping of symbols to the excerpt: V_j is `mesh.cells[i].volume`; Δt is `dt` (parameter); v_j is `v[i]`; g_j^W is `gw[i]`; g_j^E is `ge[i]`; $G_{j-1/2}$ is `mesh.conductance[i - 1]`; $G_{j+1/2}$ is `mesh.conductance[i]`; G_R is `mesh.outer_conductance`; C_j^n is `old[i]`; rhs_j is `rhs[i]`; C_j^{n+1} is elements of the returned vector; N_c is `n`.

The matrix is never formed; the three diagonals are carried by `gw`, `ge` and `v` (the diagonal entry is `v[i] + gw[i] + ge[i]`, assembled inside the solver as `e + ge`, see row B.13).

3.13 Tridiagonal solve, forward sweep without subtraction

Operation B.13. Classification: IMPLEMENTED. Equations: (5.17), (5.18), (5.19). Rust function: `solve_diffusion_system`. Pinned lines: src/numerical/finite_volume.rs, lines 132 to 160.

Final equation.

$$d_1 = v_1 + g_1^E, \qquad d_j = v_j + g_j^W + g_j^E - \frac{\left(g_j^W\right)^2}{d_{j-1}} \quad (j \geq 2). \tag{5.17}$$

$$e_1 = v_1, \qquad e_j = v_j + \frac{g_j^W \, e_{j-1}}{e_{j-1} + g_j^W} \quad (j \geq 2), \qquad d_j = e_j + g_j^E. \tag{5.18}$$

$$\begin{aligned} y_1 &= \frac{\mathrm{rhs}_1}{d_1}, \qquad y_j = \frac{\mathrm{rhs}_j + g_j^W \, y_{j-1}}{d_j} \quad (j = 2, \dots, N_c), \\ C_{N_c}^{n+1} &= y_{N_c}, \quad C_j^{n+1} = y_j + \frac{g_j^E}{d_j} \, C_{j+1}^{n+1} \quad (j = N_c - 1, \dots, 1). \end{aligned} \tag{5.19}$$

Eq. (5.17) is the textbook pivot (DERIVED NOT IMPLEMENTED). The code uses Eqs. (5.18) and (5.19).

Pseudocode.

```
# Goal: solve the chain of equations of B.12 from cell 1 outward
1. e[1] = v[1]                                          # Eq. (5.18)
2. d[1] = e[1] + gE[1]                                  # Eq. (5.18): d[j] = e[j] + gE[j]
3. y[1] = rhs[1] / d[1]                                 # Eq. (5.19)
4. FOR j = 2 to Nc:
       # Eq. (5.18): only additions of positive numbers
       4a. e[j] = v[j] + gW[j] * e[j-1] / (e[j-1] + gW[j])
       4b. d[j] = e[j] + gE[j]                          # Eq. (5.18)
       # Eq. (5.19): fold cell j-1 into cell j
       4c. y[j] = (rhs[j] + gW[j] * y[j-1]) / d[j]
```

Implementation detail: the textbook pivot of Eq. (5.17) is `d[j] = v[j] + gW[j] + gE[j] - gW[j]^2 / d[j-1]`. With `d[j-1] = e[j-1] + gW[j]` (because `gE[j-1] = gW[j]`) it has the same value as Eq. (5.18), but it contains a subtraction. The code uses the form without subtraction.

Rust.

```rust
// src/numerical/finite_volume.rs, lines 132 to 133 and 137 to 153
fn solve_diffusion_system(v: &[f64], gw: &[f64], ge: &[f64], rhs: &[f64]) -> Result<Vec<f64>> {
    let n = v.len();
    // ...
    let mut den = vec![0.0; n];
    let mut dp = vec![0.0; n];
    let mut e_prev = 0.0;
    for i in 0..n {
        let e = if i == 0 {
            v[0]
        } else {
            v[i] + gw[i] * e_prev / (e_prev + gw[i])
        };
        den[i] = e + ge[i];
        if den[i] <= 0.0 || !den[i].is_finite() {
            bail!("singular or non-finite pivot at cell {i}");
        }
        let carry = if i == 0 { 0.0 } else { gw[i] * dp[i - 1] };
        dp[i] = (rhs[i] + carry) / den[i];
        e_prev = e;
    }
```

Mapping of symbols to the excerpt: e_j is `e` (local), carried to the next cell as `e_prev`; e_{j-1} is `e_prev`; d_j is `den[i]`; y_j is `dp[i]`; $g_j^W y_{j-1}$ is `carry`; v_j is `v[i]`; g_j^W, g_j^E is `gw[i]`, `ge[i]`; rhs_j is `rhs[i]`; N_c is `n`.

Cell `i = 0` takes the explicit branch because $0 \cdot 0/(0+0)$ would be NaN (lines 141 to 143 and 150). The guard on `den[i]` is at lines 147 to 149. Lines 134 to 136 (length check) are omitted from the excerpt.

3.14 Back substitution

Operation B.14. Classification: IMPLEMENTED. Equations: (5.19). Rust function: `solve_diffusion_system`. Pinned lines: src/numerical/finite_volume.rs, lines 154 to 158.

Final equation.

$$\begin{aligned} y_1 &= \frac{\mathrm{rhs}_1}{d_1}, \qquad y_j = \frac{\mathrm{rhs}_j + g_j^W\, y_{j-1}}{d_j} \quad (j = 2, \dots, N_c), \\ C_{N_c}^{n+1} &= y_{N_c}, \quad C_j^{n+1} = y_j + \frac{g_j^E}{d_j}\, C_{j+1}^{n+1} \quad (j = N_c - 1, \dots, 1). \end{aligned} \tag{5.19}$$

Pseudocode.

```
# The values y[j] and d[j] come from B.13
# Eq. (5.19): the last cell is finished first
1. Cnew[Nc] = y[Nc]
2. FOR j = Nc-1 down to 1:
       # Eq. (5.19): correct cell j using the finished
       # neighbour on its outer side
       Cnew[j] = y[j] + (gE[j] / d[j]) * Cnew[j+1]
3. RETURN Cnew[1..Nc]                              # these are C at level n+1
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 154 to 158
    let mut x = vec![0.0; n];
    x[n - 1] = dp[n - 1];
    for i in (0..n - 1).rev() {
        x[i] = dp[i] + ge[i] / den[i] * x[i + 1];
    }
```

Mapping of symbols to the excerpt: $C_{N_c}^{n+1}$ is `x[n - 1]`; C_j^{n+1} is `x[i]`; y_j is `dp[i]`; d_j is `den[i]`; g_j^E is `ge[i]`; N_c is `n`.

The loop runs `for i in (0..n - 1).rev()`, that is from $j = N_c - 1$ down to $j = 1$ in the 1-based notation.

3.15 Release rate

Operation B.15. Classification: IMPLEMENTED. Equations: (3.7), (5.11), (5.22). Rust function: `run`. Pinned lines: src/numerical/finite_volume.rs, line 202; src/numerical/finite_volume.rs, line 210.

Final equation.

$$\Phi_R(t) = 4\pi R^2 \, J_r(R,t) = -4\pi R^2 D_5 \, \frac{\partial c_5}{\partial r}(R,t). \tag{3.7}$$

$$G_R = \frac{A(R)}{(R - r_{N_c})/D_5 + 0} = \frac{4\pi R^2 D_5}{R - r_{N_c}}, \qquad \Phi_R \approx G_R \, (C_{N_c} - 0) = G_R \, C_{N_c}. \tag{5.11}$$

$$\Phi_R^{n+1} = G_R \, C_{N_c}^{n+1}, \qquad M_{\text{rel}}^{n+1} = M_{\text{rel}}^{n} + \Delta t \, \Phi_R^{n+1}, \qquad M_{\text{rel}}^{0} = 0. \tag{5.22}$$

In the code the rate is $\Phi_R = G_R C_{N_c}$ in mol s^{-1}.

Pseudocode.

```
# Meaning (Eq. 3.7): release rate = surface area x outward flux at r = R
GIVEN  Cnew[Nc] from B.14 and G_R from B.7
1. Phi_R = G_R * Cnew[Nc]            # Eqs. (5.11), (5.22): flow through the surface, in mol/s
# Eq. (5.22): rate x time step = amount released in this step
2. Mrel_new = Mrel_old + dt * Phi_R
3. Mrel at level 0 is 0              # Eq. (5.22)
```

Implementation detail: the code does not use the plain sum of step 2. It uses the compensated sum of row B.16 (Eq. (5.23)).

Rust.

```rust
// src/numerical/finite_volume.rs, lines 202 and 210
            release_rate: mesh.outer_conductance * c[c.len() - 1],
        // ...
        let release_rate = mesh.outer_conductance * next[next.len() - 1];
```

Mapping of symbols to the excerpt: Φ_R^{n+1} is `release_rate` (local in the loop); Φ_R^0 is field `release_rate` of the first row; G_R is `mesh.outer_conductance`; $C_{N_c}^{n+1}$ is `next[next.len() - 1]`; $C_{N_c}^0$ is `c[c.len() - 1]`.

Line 202 is inside the struct literal of the first row (lines 196 to 206); line 210 is inside the loop (lines 208 to 234).

3.16 Cumulative release with Kahan compensation

Operation B.16. Classification: IMPLEMENTED. Equations: (5.22), (5.23). Rust function: `run`. Pinned lines: src/numerical/finite_volume.rs, lines 211 to 217.

Final equation.

$$\Phi_R^{n+1} = G_R\, C_{N_c}^{n+1}, \qquad M_{\mathrm{rel}}^{n+1} = M_{\mathrm{rel}}^{n} + \Delta t\, \Phi_R^{n+1}, \qquad M_{\mathrm{rel}}^{0} = 0. \tag{5.22}$$

$$\begin{aligned} \upsilon &= \Delta t\, \Phi_R^{n+1} - \kappa, \\ \Sigma &= M_{\mathrm{rel}}^{n} + \upsilon, \\ \kappa &\leftarrow (\Sigma - M_{\mathrm{rel}}^{n}) - \upsilon, \\ M_{\mathrm{rel}}^{n+1} &\leftarrow \Sigma. \end{aligned} \tag{5.23}$$

The plain recurrence of Eq. (5.22) is replaced by the compensated form of Eq. (5.23).

Pseudocode.

```
# Same sum as Eq. (5.22), but a small leftover is carried from step to step (Eq. 5.23)
GIVEN  Phi_R of this step, dt, Mrel (running total), kappa (the carried leftover)
1. upsilon = dt * Phi_R - kappa        # Eq. (5.23): add back what was lost last time
2. Sigma   = Mrel + upsilon            # Eq. (5.23): the new total
3. kappa   = (Sigma - Mrel) - upsilon  # Eq. (5.23): what the addition just lost
4. Mrel    = Sigma                     # Eq. (5.23)
```

Implementation detail: Eq. (5.23) does not give the starting value of `kappa`. The usual starting value is 0, together with `Mrel = 0` from Eq. (5.22).

Rust.

```rust
// src/numerical/finite_volume.rs, lines 211 to 217
        // Kahan compensated accumulation prevents long-run roundoff in the
        // independently accumulated boundary-release conservation check.
        let increment = dt * release_rate;
        let y = increment - release_compensation;
        let t = released + y;
        release_compensation = (t - released) - y;
        released = t;
```

Mapping of symbols to the excerpt: increment $\Delta t\, \Phi_R^{n+1}$ is `increment`; υ is `y`; Σ is `t` (a local sum, unrelated to time); κ is `release_compensation`; M_{rel}^{n} is `released`; M_{rel}^{n+1} is `released` after line 217.

3.17 Balance residual

Operation B.17. Classification: IMPLEMENTED. Equations: (5.24), (6.1). Rust function: `run`. Pinned lines: src/numerical/finite_volume.rs, lines 219 to 220.

Final equation.

$$\varepsilon^n = M^n + M_{\mathrm{rel}}^{n} - M^0, \qquad F^n = \frac{M_{\mathrm{rel}}^{n}}{M^0}. \tag{5.24}$$

$$M^{n+1} + \Delta t\, G_R C_{N_c}^{n+1} = M^n. \tag{6.1}$$

Eq. (6.1) is the property that makes $\varepsilon^n = 0$ in exact arithmetic; the code evaluates ε^n of Eq. (5.24).

Pseudocode.

```
# Material that is still inside + material that has left - material at the start should be 0
GIVEN  M0, and at level n: M (from B.10) and Mrel (from B.16)
1. eps = M + Mrel - M0                    # Eq. (5.24)
2. F   = Mrel / M0                        # Eq. (5.24)

# Why eps is 0 in exact arithmetic (Eq. 6.1): add Eq. (5.13) over all cells.
# The flow from cell j to cell j+1 appears once with a plus sign and once with a minus sign,
# so
# internal terms cancel. gW[1] = 0 removes the centre. Only the surface term is left:
#     M_new + dt * G_R * Cnew[Nc] = M_old  # Eq. (6.1)
```

Implementation detail: the code only evaluates `eps` of Eq. (5.24) and reports it. It does not force Eq. (6.1).

Rust.

```
// src/numerical/finite_volume.rs, lines 219 to 220
        let inv = inventory(mesh, &c);
        let residual = inv + released - initial;
```

Mapping of symbols to the excerpt: ε^n is `residual`; stored in field `balance_residual`; M^n is `inv`; M^n_{rel} is `released`; M^0 is `initial`.

The identity $M^{n+1} + \Delta t\, G_R C^{n+1}_{N_c} = M^n$ of Eq. (6.1) is not computed; only its residual is. The row at $t = 0$ stores 0.0 (line 203).

3.18 Release fraction

Operation B.18. Classification: IMPLEMENTED. Equations: (3.9), (5.24). Rust function: `run`. Pinned lines: src/numerical/finite_volume.rs, line 186; src/numerical/finite_volume.rs, line 227.

Final equation.

$$M_{\mathrm{rel}}(t) = \int_0^t \Phi_R(t')\, dt', \qquad F(t) = \frac{M_{\mathrm{rel}}(t)}{M^0} = 1 - \frac{M(t)}{M^0}. \tag{3.9}$$

$$\varepsilon^n = M^n + M^n_{\mathrm{rel}} - M^0, \qquad F^n = \frac{M^n_{\mathrm{rel}}}{M^0}. \tag{5.24}$$

Pseudocode.

```
# Meaning (Eq. 3.9): fraction released = released amount / starting amount = 1 - (amount left)
# / (starting amount)
GIVEN  Mrel at level n and M0
1. F[n] = Mrel / M0                 # Eq. (5.24)
2. Equivalent form: F = 1 - M / M0  # Eq. (3.9), holds when eps = 0
```

Rust.

```
// src/numerical/finite_volume.rs, lines 186 and 227
    let initial = inventory(mesh, &c);
                  // ...
                  release_fraction: released / initial,
```

Mapping of symbols to the excerpt: F^n is field `release_fraction` of a row; M^n_{rel} is `released`; M^0 is `initial`; $\Phi_R(t)$, $\int \Phi_R\, dt'$ is not coded as an integral; the sum of row B.16 replaces it.

3.19 Output rows and minimum concentration

Operation B.19. Classification: IMPLEMENTED. Equations: (5.25). Rust function: `run`. Pinned lines: src/numerical/finite_volume.rs, lines 196 to 206; src/numerical/finite_volume.rs, lines 221 to 233.

Final equation.

$$n \bmod n_{\mathrm{out}} = 0 \quad \text{or} \quad n = N_t, \qquad t_n = n\,\Delta t, \tag{5.25}$$

Pseudocode.

```
FOR n = 1 to Nt:
    1. t_n = n * dt                        # Eq. (5.25)
    2. IF (n mod n_out = 0) OR (n = Nt):  # Eq. (5.25): every n_out-th step, plus the last
           save one output row for time t_n
```

Implementation detail: Eq. (5.25) is the only equation of this row. The condition `n mod n_out = 0` also holds for `n = 0`; the equation does not say whether a row for level 0 is written. The minimum concentration named in the title has no equation. Treat it as one more value saved in each output row.

Rust.

```rust
// src/numerical/finite_volume.rs, lines 196 to 206 and 221 to 233
    rows.push((
        0.0,
        StepDiagnostics {
            inventory: initial,
            cumulative_release: 0.0,
            release_fraction: 0.0,
            release_rate: mesh.outer_conductance * c[c.len() - 1],
            balance_residual: 0.0,
            min_concentration: c.iter().copied().fold(f64::INFINITY, f64::min),
        },
    ));
        // ...
        if step % cfg.time.output_every_steps == 0 || step == steps {
            rows.push((
                step as f64 * dt,
                StepDiagnostics {
                    inventory: inv,
                    cumulative_release: released,
                    release_fraction: released / initial,
                    release_rate,
                    balance_residual: residual,
                    min_concentration: c.iter().copied().fold(f64::INFINITY, f64::min),
                },
            ));
        }
```

Mapping of symbols to the excerpt: n is `step`; n_{out} is `cfg.time.output_every_steps`; N_t is `steps`; t_n is first element of a row tuple, `step as f64 * dt`; Δt is `dt`; $\min_j C_j^n$ is field `min_concentration`.

The first part of the excerpt is the row at $t = 0$, the second part is the output rule inside the loop.

3.20 Explicit-step bound (diagnostic, printed once)

Operation B.20. Classification: IMPLEMENTED (diagnostic only). Equations: (6.6). Rust function: `explicit_dt_bound`. Pinned lines: src/numerical/finite_volume.rs, lines 238 to 252; src/main.rs, line 78.

Final equation.

$$\Delta t_{\mathrm{FE}} = \min_j \frac{V_j}{g_j^W + g_j^E}, \tag{6.6}$$

Pseudocode.

```
1. FOR each cell j = 1 to Nc:
       ratio[j] = V[j] / (gW[j] + gE[j])  # Eq. (6.6): how long cell j takes to empty at its
                                          # present outflow rate
2. dt_FE = smallest ratio[j]              # Eq. (6.6)
3. Print dt_FE once                       # diagnostic: the run does not use it to change dt
```

Rust.

```rust
// src/numerical/finite_volume.rs, lines 238 to 252
pub fn explicit_dt_bound(mesh: &Mesh) -> f64 {
    mesh.cells
        .iter()
        .enumerate()
        .map(|(i, cell)| {
            let gw = if i == 0 { 0.0 } else { mesh.conductance[i - 1] };
            let ge = if i + 1 == mesh.cells.len() {
                mesh.outer_conductance
            } else {
                mesh.conductance[i]
            };
            cell.volume / (gw + ge)
        })
        .fold(f64::INFINITY, f64::min)
}
```

```rust
// src/main.rs, line 78
    println!("explicit_dt_bound_s={:.12e}", fv::explicit_dt_bound(&mesh));
```

Mapping of symbols to the excerpt: Δt_{FE} is return value of `explicit_dt_bound`; V_j is `cell.volume`; g_j^W is `gw`; g_j^E is `ge`; $\min_j$ is `.fold(f64::INFINITY, f64::min)`; printed key is `explicit_dt_bound_s`.

3.21 Forward Euler scheme and its L^2 limit

Operation B.21. Classification: DERIVED NOT IMPLEMENTED. Equations: (6.7). Rust function: none.

Final equation.

$$\Delta t \le \frac{2}{\mu_{\max}}, \tag{6.7}$$

No Rust function computes this limit.

Pseudocode.

```
NOT IMPLEMENTED. No Rust function computes this limit. Description of the equation only:
1. Scale the conductance matrix:  S = V^(-1/2) * A * V^(-1/2)       # symbol table of B.21
2. mu_max = largest eigenvalue of S
3. Forward Euler is stable in the L2 sense when dt <= 2 / mu_max    # Eq. (6.7)
```

Rust. No Rust function computes this row.

No function computes Eq. (6.7). A search of `src/numerical/finite_volume.rs` for the words forward, explicit, eigenvalue and Gershgorin finds only `explicit_dt_bound` (line 238, row B.20) and its unit test (lines 392 and 393).

3.22 Source term, decay, $D(T)$, Robin outer face

Operation B.22. Classification: FUTURE IMPROVEMENT. Equations: (2.6), (3.10). Rust function: none.

Final equation.

$$\begin{aligned}\frac{\partial c}{\partial t} &= -\frac{1}{r^2}\frac{\partial}{\partial r}\left(-r^2 D\,\frac{\partial c}{\partial r}\right) + S \\ &= \frac{1}{r^2}\frac{\partial}{\partial r}\left(r^2 D\,\frac{\partial c}{\partial r}\right) + S\end{aligned} \tag{2.6}$$

$$-D_5\,\frac{\partial c_5}{\partial r}(R,t) = h_{\mathrm{m}}\left[c_5(R,t) - c_\infty\right]. \tag{3.10}$$

v1.0.0 sets $S = 0$ and uses the perfect-sink face of Eq. (3.4). No Rust code contains a source, a decay term, a temperature-dependent diffusivity or a Robin face for the layered particle.

Pseudocode.

```
NOT IMPLEMENTED. No Rust code contains these. What v1.0.0 does instead:
# the equation then has only the diffusion term
1. Source S = 0 in Eq. (2.6)
2. Outer face uses the perfect sink c5(R, t) = 0 of Eq. (3.4), not Eq. (3.10)

What the equations would add in a later version (description of the equations only):
3. Source: add S to the rate of change of c                          # Eq. (2.6)
# Eq. (3.10): -D5 * dc5/dr = h_m * (c5 - c_inf)
4. Robin outer face: flux out of the surface = h_m * (c5(R, t) - c_inf)
```

Rust.

```
// src/numerical/finite_volume.rs, lines 178 to 179
        v[i] = mesh.cells[i].volume / dt;
        rhs[i] = v[i] * old[i];
```

Mapping of symbols to the excerpt: S: no variable; `rhs[i]` is `v[i] * old[i]` with no source term; $D(T)$: no variable; `temperature_k` is read and checked only; Robin face (h_{m}, c_∞): no variable; the outer face is Dirichlet through `outer_conductance`.

FUTURE IMPROVEMENT, no Rust function. The excerpt, src/numerical/finite_volume.rs, lines 178 to 179, shows that the right-hand side has no source term. A search of `finite_volume.rs`, `main.rs` and `comparison.rs` for source, decay, robin, mass transfer, ambient and temperature finds only the fixture value `temperature_k: 1000.0` at `finite_volume.rs` line 263 (unit-test configuration).

4 Analytical solver (Code C)

Code C (`src/analytical/sturm_liouville.rs`) solves the same problem as a series. Separation of variables turns Eq. (2.7) into an eigenvalue problem on the five layers, which is the layered Sturm-Liouville problem of Gelbard (2003) with the partition factor set to 1. The code propagates the state (u, q) through the layers, counts the eigenvalues below a trial value with a Sturm count, isolates every eigenvalue by bisection on that count and forms the modal weights by quadrature. The release fraction is then the sum of Eq. (4.22). Table 4.1 lists the 21 operations C.1 to C.21. Rows C.1, C.20 and C.21 are not computed in v1.0.0.

Table 4.1 Operations of Code C in the order of the computation. Row C.1 is described in Section 4.1, row C.21 in Section 4.21.

No.	Final mathematical formulation	Pseudocode	Corresponding Rust implementation
C.1	$c = \phi(r)\, e^{-\Lambda t}$, Eq. (4.1); $\frac{1}{r^2}\frac{d}{dr}\left(r^2 D_\ell \frac{d\phi}{dr}\right) + \Lambda\phi = 0$, Eq. (4.2)	none; stated as the basis in the module comment	`module comment`, src/analytical/sturm_liouville.rs, lines 12 to 19. DERIVED NOT IMPLEMENTED (basis)
C.2	$u = r\phi$, $u'' + k_\ell^2 u = 0$, $k_\ell = \sqrt{\Lambda / D_\ell}$, Eqs. (4.3), (4.4)	`k[l] = sqrt(Lambda / D[l])` for each layer	`shoot`, `eigenvalue_count`, src/analytical/sturm_liouville.rs, line 141 and 1 more range (Section 4.2). IMPLEMENTED
C.3	$u_{\text{out}} = \cos(k_\ell L_\ell)\, u_{\text{in}} + \frac{\sin(k_\ell L_\ell)}{D_\ell k_\ell}\, q_{\text{in}}$, $q_{\text{out}} = -D_\ell k_\ell \sin(k_\ell L_\ell)\, u_{\text{in}} + \cos(k_\ell L_\ell)\, q_{\text{in}}$, Eq. (4.6)	`u_out = cs u_in + sn / (D k) q_in; q_out = -D k sn u_in + cs q_in`	`transfer_layer`, src/analytical/sturm_liouville.rs, lines 86 to 92. IMPLEMENTED
C.4	$u^+ = u^-$, $q^+ = q^- + (D_{\ell+1} - D_\ell)\, u(R_\ell)/R_\ell$, Eqs. (4.7), (4.8)	`u_plus = u; q_plus = q + (D[l+1] - D[l]) u / R[l]`	`interface_state`, src/analytical/sturm_liouville.rs, lines 98 to 100. IMPLEMENTED
C.5	$(u, q)(0) = (0, 1)$, Eq. (4.9); $g(\Lambda) = u(R; \Lambda)$, Eq. (4.10)	start at `(0, 1)`; for each layer transfer, then apply the interface rule; `g = u at R`	`shoot`, src/analytical/sturm_liouville.rs, lines 132 to 162. IMPLEMENTED
C.6	$\mathcal{N}(\Lambda) = \sum_{\ell=1}^{5} \left(\left\lfloor \frac{\theta_{0,\ell} + k_\ell L_\ell}{\pi} \right\rfloor - \left\lfloor \frac{\theta_{0,\ell}}{\pi} \right\rfloor \right)$, Eq. (4.13)	`theta0 = atan2(a, b); count += floor((theta0 + k L) / pi) - floor(theta0 / pi)` per layer	`eigenvalue_count`, src/analytical/sturm_liouville.rs, lines 166 to 192. IMPLEMENTED
C.7	$\Lambda_{\text{cov}} = \Lambda_{\max}$ if $\mathcal{N}(\Lambda_{\max}) \le K_{\max}$, else the lower end of a geometric search, Eq. (4.16)	`IF count(Lambda_max) <= K_max: Lambda_cov = Lambda_max; ELSE` geometric search	`eigenvalues`, src/analytical/sturm_liouville.rs, lines 202 to 217. IMPLEMENTED
C.8	$\Lambda_{\text{lo}} = 10^{-6} \left(\sum_\ell L_\ell / \sqrt{D_\ell} \right)^{-2}$, then shrunk by 0.01 until $\mathcal{N} = 0$, Eq. (4.14)	`Lambda_lo = 1e-6 S^(-2); WHILE count(Lambda_lo) != 0: Lambda_lo = 0.01 Lambda_lo`	`eigenvalues`, src/analytical/sturm_liouville.rs, lines 220 to 238. IMPLEMENTED
C.9	$m = \sqrt{ab}$ if $b > 4a$, else $(a + b)/2$; stop when $b - a \le 4 \times 10^{-15} b$, Eq. (4.15)	split a bracket at `m` while `N(b) - N(a) > 0`; stop when `b - a <= 4e-15 b`	`eigenvalues`, src/analytical/sturm_liouville.rs, lines 240 to 263. IMPLEMENTED

No.	Final mathematical formulation	Pseudocode	Corresponding Rust implementation
C.10	$0 = R_0 < R_1 < \cdots < R_5$, $L_\ell = R_\ell - R_{\ell-1}$, Eqs. (1.1), (1.2)	check `R[l] > R[l-1]` for each layer; `L[l] = R[l] - R[l-1]`	`validate_layers`, src/analytical/sturm_liouville.rs, lines 112 to 127. IMPLEMENTED
C.11	$u(s) = a_\ell \cos(k_\ell s) + b_\ell \sin(k_\ell s)$, $0 \le s \le L_\ell$, Eq. (4.5)	`a = u_in; b = q_in / (D k)`; carry `(u, q)` to the next layer	`ModalRelease::new`, src/analytical/sturm_liouville.rs, lines 338 to 344. IMPLEMENTED
C.12	$(u, q)(0) = (0, 1)$, Eq. (4.9); the scale cancels in w_n	start every mode at `(0, 1)`; no separate normalisation step	`shoot`, `ModalRelease::new`, src/analytical/sturm_liouville.rs, line 134 and 1 more range (Section 4.12). IMPLEMENTED (implicit)
C.13	$\int_{R_{\ell-1}}^{R_\ell} f\,dr \approx \sum_{i=1}^{N_{p,\ell}} \frac{h}{2} \sum_{m=1}^{16} \omega_m f(\mathrm{mid}_i + \frac{h}{2} x_m)$, Eq. (4.24)	`Np = ceil(k L / (pi/2)) + 1`; sum 16 weighted nodes on each panel	`gauss_legendre_16`, `integrate`, src/analytical/sturm_liouville.rs, lines 266 to 292 and 2 more ranges (Section 4.13). IMPLEMENTED
C.14	$I_{1,n} = \int_0^{R_1} r\,u_n\,dr$, $I_n = \int_0^{R} r\,u_n\,dr$, $N_n = \int_0^{R} u_n^2\,dr$, Eq. (4.20)	integrate `r u` over the kernel and over the particle; integrate `u^2` over the particle	`ModalRelease::new`, src/analytical/sturm_liouville.rs, lines 337 to 351. IMPLEMENTED
C.15	$w_n = \dfrac{3\,I_{1,n} I_n}{R_1^3 N_n}$, Eq. (4.22)	`w[n] = 3 I1[n] I[n] / (R[1]^3 N[n])`	`ModalRelease::new`, src/analytical/sturm_liouville.rs, line 325 and 1 more range (Section 4.15). IMPLEMENTED
C.16	$F(t) = 1 - \sum_{n=1}^{K} w_n e^{-\Lambda_n t}$, Eq. (4.22)	`remaining = sum of w[n] exp(-Lambda[n] t); F = 1 - remaining`	`remaining_fraction`, `release_fraction`, src/analytical/sturm_liouville.rs, lines 382 to 393. IMPLEMENTED
C.17	$\Lambda_{\max} = 45/t_{\min}$, $t_{\text{valid}} = 40/\Lambda_{\text{cov}}$, Eq. (4.26)	report `F_ex(t)` only if `t >= t_valid`	`exact_release`, `min_valid_time`, src/analytical/sturm_liouville.rs, lines 377 to 379 and 1 more range (Section 4.17). IMPLEMENTED
C.18	$\rho = \dfrac{\lvert q - D\,u/R_\ell \rvert}{\lvert q \rvert + \lvert D\,u/R_\ell \rvert}$, Eq. (4.27); $\varepsilon_F = \dfrac{2.2 \times 10^{-16}}{\min(\rho_{\min}, 1)}$, Eq. (4.28)	`rho_min` = smallest `rho` over modes and layers; `eps_F = 2.2e-16 / min(rho_min, 1)`	`ModalRelease::new`, `accuracy_floor`, src/analytical/sturm_liouville.rs, lines 330 to 336 and 1 more range (Section 4.18). IMPLEMENTED (heuristic, CLAIMED ONLY as a bound)
C.19	$z \cot z = 1 - \mathrm{Bi}$, Eq. (4.31)	`residual(z) = z cot(z) - (1 - Bi)`; find roots (unit test only)	`robin_residual`, `robin_roots`, src/analytical/sturm_liouville.rs, lines 34 to 36 and 1 more range (Section 4.19). IMPLEMENTED (unit test only)
C.20	$A_n = c_0 I_{1,n}/N_n$, Eq. (4.18); $c(r,t)$ and $M(t)$ series, Eqs. (4.19), (4.21)	none; Code C returns only `F(t)`	none. DERIVED NOT IMPLEMENTED

C.21	$\lim_{K\to\infty}\sum_{n\le K} w_n = 1$, Eq. (4.23); layered Robin condition, Eq. (3.10)	none; absent from v1.0.0	none. FUTURE IMPROVEMENT

Each operation below repeats the equation, the pseudocode and the Rust excerpt of its row in Table 4.1.

4.1 Separation of variables, shape equation

Operation C.1. Classification: DERIVED NOT IMPLEMENTED (basis). Equations: (4.1), (4.2). Rust function: `module comment`. Pinned lines: src/analytical/sturm_liouville.rs, lines 12 to 19.

Final equation.

$$c(r,t) = \phi(r)\, e^{-\Lambda t}, \tag{4.1}$$

$$\frac{1}{r^2}\frac{d}{dr}\left(r^2 D_\ell \frac{d\phi}{dr}\right) + \Lambda\phi = 0, \qquad R_{\ell-1} < r < R_\ell, \quad \ell = 1,\dots,5. \tag{4.2}$$

The code does not compute these equations. They are the basis of the modal problem.

Pseudocode.

```
NOT COMPUTED by the code. The module comment states it as the basis of the modal problem.
# Eq. (4.1): shape in r times decay in t
1. Look for solutions of the form c(r, t) = phi(r) * exp(-Lambda * t)
2. Put this form into the diffusion equation, divide out exp(-Lambda*t)
3. The shape must satisfy, inside each layer l = 1..5:
       (1/r^2) * d/dr( r^2 * D[l] * dphi/dr ) + Lambda * phi = 0       # Eq. (4.2)
4. Only some values of Lambda allow a shape that meets all the conditions: the eigenvalues
```

Rust.

```
// src/analytical/sturm_liouville.rs, lines 12 to 19
//! Modal formulation (u = r*phi, k_i^2 = Lambda/D_i):
//!   u_i'' + k_i^2 u_i = 0,
//!   u continuous and D (u' - u/r) continuous at every interface,
//!   u(0) = 0 (regular centre), u(R) = 0 (absorbing outer surface).
//! Orthogonality weight is r^2 for phi, i.e. weight 1 for u, so the release fraction is
//!   F(t) = 1 - sum_n w_n exp(-Lambda_n t),
//!   w_n  = 3 I1_n In_n / (R1^3 N_n),
//!   I1_n = int_0^R1 r u_n dr, In_n = int_0^R r u_n dr, N_n = int_0^R u_n^2 dr.
```

Mapping of symbols to the excerpt: Λ is `Lambda` in the comment; `lambda` in the code; D_ℓ is `D_i` in the comment; `l.diffusivity` in the code; $u = r\phi$ is `u`; k_ℓ^2 is `k_i^2` in the comment; $\phi(r)$: no variable; only u is stored.

DERIVED NOT IMPLEMENTED (basis). The module comment states the modal problem that the code solves; no statement computes the separation itself.

4.2 Transformation $u = r\phi$ and wavenumber

Operation C.2. Classification: IMPLEMENTED. Equations: (4.3), (4.4). Rust function: `shoot, eigenvalue_count`. Pinned lines: src/analytical/sturm_liouville.rs, line 141; src/analytical/sturm_liouville.rs, line 175.

Final equation.

$$\phi' = \frac{ru' - u}{r^2} = \frac{1}{r}\left(u' - \frac{u}{r}\right). \tag{4.3}$$

$$u'' + k_\ell^2\, u = 0, \qquad k_\ell = \sqrt{\frac{\Lambda}{D_\ell}}, \qquad R_{\ell-1} < r < R_\ell. \tag{4.4}$$

Pseudocode.

```
# removes the 1/r^2 factor of Eq. (4.2)
1. Define u = r * phi
2. Then dphi/dr = (r * du/dr - u) / r^2                          # Eq. (4.3)
# Eq. (4.4): a plain oscillation in r
3. Inside a layer the shape equation becomes  u'' + k^2 * u = 0
4. FOR each layer l = 1 to 5, for the trial value Lambda:
       # Eq. (4.4): oscillations are faster in a layer
       # with small D
       k[l] = sqrt(Lambda / D[l])
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, line 141
        let k = (lambda / l.diffusivity).sqrt();
```

```rust
// src/analytical/sturm_liouville.rs, line 175
        let k = (lambda / l.diffusivity).sqrt();
```

Mapping of symbols to the excerpt: k_ℓ is `k`; Λ is `lambda` (parameter); D_ℓ is `l.diffusivity`; $u = r\phi$ is `u`.

The first block is in `shoot` (line 141), the second in `eigenvalue_count` (line 175).

4.3 Layer transfer of the state (u, q)

Operation C.3. Classification: IMPLEMENTED. Equations: (4.5), (4.6). Rust function: `transfer_layer`. Pinned lines: src/analytical/sturm_liouville.rs, lines 86 to 92.

Final equation.

$$u(s) = a_\ell \cos(k_\ell s) + b_\ell \sin(k_\ell s), \qquad 0 \le s \le L_\ell. \tag{4.5}$$

$$\begin{pmatrix} u \\ q \end{pmatrix}_{\text{out}} = \mathbf{T}_\ell \begin{pmatrix} u \\ q \end{pmatrix}_{\text{in}}, \qquad \mathbf{T}_\ell = \begin{pmatrix} \cos(k_\ell L_\ell) & \dfrac{\sin(k_\ell L_\ell)}{D_\ell k_\ell} \\ -D_\ell k_\ell \sin(k_\ell L_\ell) & \cos(k_\ell L_\ell) \end{pmatrix}. \tag{4.6}$$

Pseudocode.

```
# State: u and q = D * du/dr. Inside a layer u(s) = a*cos(k*s) + b*sin(k*s), s = distance from
# the layer entrance.
GIVEN  (u_in, q_in) at the layer entrance, k = k[l], D = D[l], L = L[l]
1. cs = cos(k * L),  sn = sin(k * L)
2. u_out = cs * u_in + (sn / (D * k)) * q_in  # Eq. (4.6), first row of the matrix T
3. q_out = -(D * k * sn) * u_in + cs * q_in   # Eq. (4.6), second row of the matrix T
4. RETURN (u_out, q_out)                      # state at the layer exit
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 86 to 92
pub fn transfer_layer(u: f64, du_flux: f64, d: f64, k: f64, length: f64) -> (f64, f64) {
    let c = (k * length).cos();
    let s = (k * length).sin();
    let u2 = c * u + s * du_flux / (d * k);
    let q2 = -d * k * s * u + c * du_flux;
    (u2, q2)
}
```

Mapping of symbols to the excerpt: u_{in} is `u` (parameter); q_{in} is `du_flux` (parameter); D_ℓ is `d`; k_ℓ is `k`; L_ℓ is `length`; $\cos(k_\ell L_\ell)$, $\sin(k_\ell L_\ell)$ is `c`, `s`; u_{out} is `u2`, first element of the return value; q_{out} is `q2`, second element of the return value; $\mathbf{T}_\ell$ is not stored; its entries appear as `c`, `s / (d * k)`, `-d * k * s`, `c`.

The state q is the product $D\,du/dr$, so the parameter named `du_flux` already contains the factor D.

4.4 Interface rule

Operation C.4. Classification: IMPLEMENTED. Equations: (3.2), (3.3), (4.7), (4.8). Rust function: `interface_state`. Pinned lines: src/analytical/sturm_liouville.rs, lines 98 to 100.

Final equation.

$$c_\ell(R_\ell^-, t) = c_{\ell+1}(R_\ell^+, t), \qquad \ell = 1, \dots, 4, \quad t > 0. \tag{3.2}$$

$$D_\ell \frac{\partial c_\ell}{\partial r}(R_\ell^-, t) = D_{\ell+1} \frac{\partial c_{\ell+1}}{\partial r}(R_\ell^+, t), \qquad \ell = 1, \dots, 4, \quad t > 0. \tag{3.3}$$

$$u \text{ continuous}, \qquad D\left(u' - \frac{u}{r}\right) \text{ continuous at every } r = R_\ell. \tag{4.7}$$

$$q^+ = q^- + (D_{\ell+1} - D_\ell)\,\frac{u(R_\ell)}{R_\ell}. \tag{4.8}$$

Pseudocode.

```
# At r = R[l] the layers l and l+1 meet. Physics: same concentration and same flux on both
# sides (Eqs. 3.2, 3.3).
GIVEN  (u, q) just inside the interface, D[l], D[l+1], R[l]
1. u_plus = u                                          # Eq. (4.7): u is continuous
2. D * (du/dr - u/r) must be the same on both sides  # Eq. (4.7)
3. q_plus = q + (D[l+1] - D[l]) * u / R[l]           # Eq. (4.8): q jumps when D jumps
4. RETURN (u_plus, q_plus)                             # state at the start of layer l+1
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 98 to 100
pub fn interface_state(q_inner: f64, u: f64, d_inner: f64, d_outer: f64, radius: f64) -> f64 {
    q_inner + (d_outer - d_inner) * u / radius
}
```

Mapping of symbols to the excerpt: q^- is `q_inner`; q^+ is return value; u is `u`; D_ℓ is `d_inner`; $D_{\ell+1}$ is `d_outer`; R_ℓ is `radius`; the callers pass `left`, the outer radius of the layer just left.

Calls: `shoot`, lines 138 to 140; `eigenvalue_count`, lines 172 to 174. Concentration continuity of Eq. (3.2) needs no code because u is passed on unchanged.

4.5 Centre start and shooting, characteristic function

Operation C.5. Classification: IMPLEMENTED. Equations: (4.9), (4.10). Rust function: `shoot`. Pinned lines: src/analytical/sturm_liouville.rs, lines 132 to 162.

Final equation.

$$(u, q)\,(r = 0) = (0, 1). \tag{4.9}$$

$$g(\Lambda) = u(R; \Lambda), \qquad \text{eigenvalues: } g(\Lambda_n) = 0. \tag{4.10}$$

The value of g is computed and discarded (item I13); the eigenvalues come from the Sturm count.

Pseudocode.

```
GIVEN a trial Lambda
# Eq. (4.9): u = r*phi is 0 at the centre
1. (u, q) = (0, 1) at r = 0
2. FOR layer l = 1 to 5:
       2a. (u, q) = transfer across layer l           # row C.3
       2b. IF l < 5: (u, q) = interface rule at R[l]  # row C.4 (four interfaces, l = 1..4)
# Eq. (4.10): g is zero exactly at the eigenvalues
3. g(Lambda) = u at r = R
```

Implementation detail: the value of `g` is computed and then discarded (item I13 of Table 7.1). The eigenvalues come from the count of row C.6.

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 132 to 150 and 158 to 162
fn shoot(lambda: f64, layers: &[Layer], rescale: bool) -> (f64, Vec<LayerStart>) {
    let mut starts = Vec::with_capacity(layers.len());
    let (mut u, mut q) = (0.0, 1.0);
    let mut left = 0.0;
    let mut prev_d: Option<f64> = None;
    for l in layers {
        if let Some(pd) = prev_d {
            q = interface_state(q, u, pd, l.diffusivity, left);
        }
        let k = (lambda / l.diffusivity).sqrt();
        starts.push(LayerStart {
            u,
            q,
            k,
            d: l.diffusivity,
            left,
            right: l.outer_radius,
        });
        (u, q) = transfer_layer(u, q, l.diffusivity, k, l.outer_radius - left);
        // ...
        left = l.outer_radius;
        prev_d = Some(l.diffusivity);
    }
    (u, starts)
}
```

Mapping of symbols to the excerpt: $(u, q)(0) = (0, 1)$ is `(u, q)` initialised to `(0.0, 1.0)`; Λ is `lambda`; layer data is `layers` (slice of `Layer`), loop variable `l`; $R_{\ell-1}$ is `left`; $D_{\ell-1}$ is `pd`, from `prev_d`; state at the start of a layer is `starts`, element `LayerStart { u, q, k, d, left, right }`; $g(\Lambda) = u(R; \Lambda)$ is first element of the returned tuple.

In v1.0.0 the only call is `shoot(lam, layers, false)` at line 329; it discards $u(R)$ and uses the `starts`. The `rescale` branch (lines 151 to 157) is never executed and is omitted from the excerpt.

4.6 Phase form and zero count (Sturm count)

Operation C.6. Classification: IMPLEMENTED. Equations: (4.11), (4.12), (4.13). Rust function: `eigenvalue_count`. Pinned lines: src/analytical/sturm_liouville.rs, lines 166 to 192.

Final equation.

$$u(s) = H_\ell \sin(k_\ell s + \theta_0), \qquad \theta_0 = \operatorname{atan2}(a_\ell, b_\ell), \qquad a_\ell = u_{\text{in}}, \quad b_\ell = \frac{q_{\text{in}}}{D_\ell k_\ell}. \tag{4.11}$$

$$\text{zeros of } u \text{ in } 0 < s \le L_\ell = \left\lfloor \frac{\theta_0 + k_\ell L_\ell}{\pi} \right\rfloor - \left\lfloor \frac{\theta_0}{\pi} \right\rfloor. \tag{4.12}$$

$$\mathcal{N}(\Lambda) = \sum_{\ell=1}^{5} \left(\left\lfloor \frac{\theta_{0,\ell} + k_\ell L_\ell}{\pi} \right\rfloor - \left\lfloor \frac{\theta_{0,\ell}}{\pi} \right\rfloor \right) = \#\{n : \Lambda_n \le \Lambda\}. \tag{4.13}$$

Pseudocode.

```
GIVEN a trial Lambda
1. (u, q) = (0, 1) at r = 0                                  # row C.5
2. count = 0
3. FOR layer l = 1 to 5:
       3a. a = u_in                                          # Eq. (4.11)
       3b. b = q_in / (D[l] * k[l])                          # Eq. (4.11)
       # Eq. (4.11): u(s) = H * sin(k*s + theta0)
       3c. theta0 = atan2(a, b)
       3d. zeros_in_layer = floor((theta0 + k[l]*L[l]) / pi) - floor(theta0 / pi)
                                                                    # Eq. (4.12): the sine
                                                                    # crosses zero once each
                                                                    # time
                                                                    # its angle passes a
                                                                    # multiple of pi
       # Eq. (4.13): sum over the five layers
       3e. count = count + zeros_in_layer
       3f. carry (u, q) across the layer and the interface  # rows C.3 and C.4
# Eq. (4.13): count = number of eigenvalues <= Lambda
4. RETURN count
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 166 to 168, 171 to 182 and 191
pub fn eigenvalue_count(lambda: f64, layers: &[Layer]) -> usize {
    let mut n: i64 = 0;
    let (mut u, mut q) = (0.0, 1.0);
    // ...
    for l in layers {
        if let Some(pd) = prev_d {
            q = interface_state(q, u, pd, l.diffusivity, left);
        }
        let k = (lambda / l.diffusivity).sqrt();
        let a = u;
        let b = q / (l.diffusivity * k);
        // u(s) = A sin(k s + theta0); zeros at k s + theta0 = m pi for s in (0, L].
        let theta0 = a.atan2(b);
        let length = l.outer_radius - left;
        n += ((theta0 + k * length) / PI).floor() as i64 - (theta0 / PI).floor() as i64;
        (u, q) = transfer_layer(u, q, l.diffusivity, k, length);
    // ...
    n.max(0) as usize
```

Mapping of symbols to the excerpt: $\theta_{0,\ell}$ is `theta0`; a_ℓ is `a` (equal to `u` at the layer entrance); b_ℓ is `b`, which is `q / (l.diffusivity * k)`; k_ℓ is `k`; L_ℓ is `length`; $\mathcal{N}(\Lambda)$ is accumulator `n` (an `i64`), returned as `n.max(0) as usize`; π is `PI`; H_ℓ is not stored; the amplitude is divided out by `m`.

The rescale at lines 183 to 187 does not change the zero count because only the ratio $a : b$ enters θ_0.

4.7 Mode cap

Operation C.7. Classification: IMPLEMENTED. Equations: (4.16). Rust function: `eigenvalues`. Pinned lines: src/analytical/sturm_liouville.rs, lines 202 to 217.

Final equation.

$$\Lambda_{\mathrm{cov}} = \begin{cases} \Lambda_{\max}, & \mathcal{N}(\Lambda_{\max}) \le K_{\max}, \\ \text{lo of the geometric search, so that } \mathcal{N}(\Lambda_{\mathrm{cov}}) \le K_{\max}, & \text{otherwise.} \end{cases} \tag{4.16}$$

Pseudocode.

```
GIVEN  Lambda_max and the cap K_max = 20000
1. n_top = count(Lambda_max)                                    # row C.6
2. IF n_top <= K_max:
       # Eq. (4.16): every wanted mode fits
       Lambda_cov = Lambda_max
   ELSE:
       find Lambda_cov by a geometric search such that count(Lambda_cov) <= K_max
       Lambda_cov = the lower end ("lo") of the search bracket  # Eq. (4.16)
3. Only eigenvalues up to Lambda_cov are kept
```

Implementation detail: Eq. (4.16) does not say how the geometric search shrinks its bracket.

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 202 to 217
    let mut hi = lambda_max;
    if eigenvalue_count(hi, layers) > max_modes {
        let mut lo = hi * 1e-12;
        for _ in 0..200 {
            let mid = (lo * hi).sqrt();
            if eigenvalue_count(mid, layers) > max_modes {
                hi = mid;
            } else {
                lo = mid;
            }
            if hi - lo <= 1e-12 * hi {
                break;
            }
        }
        hi = lo;
    }
```

Mapping of symbols to the excerpt: $\Lambda_{\max}$ is `lambda_max` (parameter), copied to `hi`; Λ_{cov} is `hi` after the cap (line 216), returned as the second value; $K_{\max}$ is `max_modes` (parameter; the caller passes `MAX_MODES`); $\mathcal{N}(\hat{\Lambda})$ is `eigenvalue_count(hi, layers)` and `eigenvalue_count(mid, layers)`; bisection ends is `lo`, `hi`, `mid`.

The mode cap is never active in the main run (32 modes).

4.8 Lower bracket

Operation C.8. Classification: IMPLEMENTED. Equations: (4.14). Rust function: `eigenvalues`. Pinned lines: src/analytical/sturm_liouville.rs, lines 220 to 238.

Final equation.

$$\Lambda_{\text{lo}} = 10^{-6}\left(\sum_{\ell=1}^{5}\frac{L_\ell}{\sqrt{D_\ell}}\right)^{-2}, \quad \text{then } \Lambda_{\text{lo}} \leftarrow 0.01\,\Lambda_{\text{lo}} \text{ until } \mathcal{N}(\Lambda_{\text{lo}}) = 0. \tag{4.14}$$

Pseudocode.

```
1. S = sum over l = 1..5 of L[l] / sqrt(D[l])
2. Lambda_lo = 1e-6 * S^(-2)         # Eq. (4.14): a value far below the first eigenvalue
3. WHILE count(Lambda_lo) is not 0:
       Lambda_lo = 0.01 * Lambda_lo  # Eq. (4.14): shrink by a factor 100
4. Now no eigenvalue lies at or below Lambda_lo
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 220 to 238
    // Lower bound with zero eigenvalues below it.
    let phase_rate: f64 = {
        let mut left = 0.0;
        layers
            .iter()
            .map(|l| {
                let v = (l.outer_radius - left) / l.diffusivity.sqrt();
                left = l.outer_radius;
                v
            })
            .sum()
    };
    let mut lo = 1e-6 * (1.0 / phase_rate).powi(2);
    while eigenvalue_count(lo, layers) != 0 {
        lo *= 0.01;
        if lo < 1e-300 {
            bail!("could not bracket the lowest eigenvalue");
        }
    }
```

Mapping of symbols to the excerpt: Λ_{lo} is `lo`; $\sum_\ell L_\ell/\sqrt{D_\ell}$ is `phase_rate`; L_ℓ is `l.outer_radius - left`; D_ℓ is `l.diffusivity`; $\mathcal{N}(\Lambda)$ is `eigenvalue_count(lo, layers)`.

The factor 10^{-6} is the literal `1e-6` at line 232; the multiplier 0.01 is at line 234.

4.9 Root isolation by bisection on the count; stopping rule

Operation C.9. Classification: IMPLEMENTED. Equations: (4.13), (4.15). Rust function: `eigenvalues`. Pinned lines: src/analytical/sturm_liouville.rs, lines 240 to 263.

Final equation.

$$\mathcal{N}(\Lambda) = \sum_{\ell=1}^{5} \left(\left\lfloor \frac{\theta_{0,\ell} + k_\ell L_\ell}{\pi} \right\rfloor - \left\lfloor \frac{\theta_{0,\ell}}{\pi} \right\rfloor \right) = \#\{n : \Lambda_n \le \Lambda\}. \tag{4.13}$$

$$m = \begin{cases} \sqrt{ab}, & b > 4a, \\ (a+b)/2, & b \le 4a, \end{cases} \qquad \text{stop when } b - a \le 4 \times 10^{-15}\, b. \tag{4.15}$$

Pseudocode.

```
# The count N(Lambda) of row C.6 says how many eigenvalues lie at or below Lambda (Eq. 4.13).
# So N(b) - N(a) is the number of eigenvalues in the bracket (a, b].
1. Put the bracket (Lambda_lo, Lambda_cov) with counts (0, N(Lambda_cov)) on the stack
                                                                    # N(Lambda_lo) = 0 by row
                                                                    # C.8
2. WHILE the stack is not empty:
       2a. take a bracket (a, b) and its counts (na, nb) from the stack
       2b. IF nb - na = 0: drop it                # Eq. (4.13): no eigenvalue in (a, b]
       2c. IF b - a <= 4e-15 * b:                 # Eq. (4.15): stopping rule
               IF nb - na > 1: STOP with an error  # unresolved cluster of eigenvalues
               record (a + b) / 2 as an eigenvalue
           ELSE:
               # Eq. (4.15): wide bracket, split by geometric mean
               IF b > 4 * a:  m = sqrt(a * b)
               # Eq. (4.15): narrow bracket, split by arithmetic mean
               ELSE:          m = (a + b) / 2
               nm = N(m), kept between na and nb
               push (a, m, na, nm) and (m, b, nm, nb)
3. Sort the recorded eigenvalues in ascending order
```

Implementation detail: Eq. (4.15) gives the split point and the stopping rule. The reported value $(a + b)/2$, the error for a final bracket that still holds more than one eigenvalue, the clamp of $\mathcal{N}(m)$ to $[n_a, n_b]$ and the final sort are code choices that no equation states.

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 240 to 263
    let mut roots = Vec::with_capacity(n_hi);
    let mut stack = vec![(lo, hi, 0usize, n_hi)];
    while let Some((a, b, na, nb)) = stack.pop() {
        if nb <= na {
            continue;
        }
        if b - a <= 4e-15 * b {
            if nb - na > 1 {
                bail!("unresolved eigenvalue cluster near Lambda = {b:.6e}");
            }
            roots.push(0.5 * (a + b));
            continue;
        }
        let m = if b > 4.0 * a {
            (a * b).sqrt()
        } else {
            0.5 * (a + b)
        };
        let nm = eigenvalue_count(m, layers).clamp(na, nb);
        stack.push((a, m, na, nm));
        stack.push((m, b, nm, nb));
    }
    roots.sort_by(|x, y| x.partial_cmp(y).expect("finite roots"));
    Ok((roots, hi))
```

Mapping of symbols to the excerpt: a, b is `a`, `b`; n_a, n_b is `na`, `nb`; m is `m`; n_m is `nm`; Λ_n is elements of `roots`, each `0.5 * (a + b)`; stack of brackets is `stack`; $\mathcal{N}(\Lambda)$ is `eigenvalue_count(m, layers)`; stopping rule is `b - a <= 4e-15 * b`; Λ_{cov} is `hi`, returned with `roots`.

4.10 Input check of the layer list

Operation C.10. Classification: IMPLEMENTED. Equations: (1.1), (1.2). Rust function: `validate_layers`. Pinned lines: src/analytical/sturm_liouville.rs, lines 112 to 127.

Final equation.

$$0 = R_0 < R_1 < R_2 < R_3 < R_4 < R_5 = R \tag{1.1}$$

$$L_\ell = R_\ell - R_{\ell-1}, \qquad \ell = 1, \dots, 5 \tag{1.2}$$

Pseudocode.

```
1. R[0] = 0
2. FOR l = 1 to 5:
       2a. Check that R[l] > R[l-1]  # Eq. (1.1): each radius is larger than the one before
       2b. L[l] = R[l] - R[l-1]      # Eq. (1.2): thickness, positive by step 2a
3. The particle radius is R = R[5]   # Eq. (1.1)
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 112 to 127
pub fn validate_layers(layers: &[Layer]) -> Result<()> {
    if layers.len() < 2 {
        bail!("at least two layers are required");
    }
    let mut last = 0.0;
    for l in layers {
        if !(l.outer_radius.is_finite() && l.diffusivity.is_finite())
            || l.outer_radius <= last
            || l.diffusivity <= 0.0
        {
            bail!("layers need increasing finite radii and positive finite diffusivities");
        }
        last = l.outer_radius;
    }
    Ok(())
}
```

Mapping of symbols to the excerpt: R_ℓ is `l.outer_radius`; $R_{\ell-1}$ is `last` (starts at 0.0); D_ℓ is `l.diffusivity`; number of layers is `layers.len()`, at least 2 here; exactly 5 is checked in `Config::validate`.

4.11 Eigenfunction in each layer

Operation C.11. Classification: IMPLEMENTED. Equations: (4.5). Rust function: `ModalRelease::new`. Pinned lines: src/analytical/sturm_liouville.rs, lines 338 to 344.

Final equation.

$$u(s) = a_\ell \cos(k_\ell s) + b_\ell \sin(k_\ell s), \qquad 0 \le s \le L_\ell. \tag{4.5}$$

Pseudocode.

```
FOR each eigenvalue Lambda_n kept:
    # rows C.5 and C.12
    1. (u, q) = (0, 1) at r = 0
    2. FOR layer l = 1 to 5:
           # row C.2
           2a. k = sqrt(Lambda_n / D[l])
           # as in Eq. (4.11) of row C.6
           2b. a = u_in,  b = q_in / (D[l] * k)
           # Eq. (4.5)
           2c. In this layer: u(s) = a * cos(k * s) + b * sin(k * s),  0 <= s <= L[l]
           # rows C.3 and C.4
           2d. carry (u, q) to the next layer
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 338 to 344
            for (idx, st) in starts.iter().enumerate() {
                let (a, b) = (st.u, st.q / (st.d * st.k));
                let panels = ((st.k * (st.right - st.left)) / (0.5 * PI)).ceil() as usize + 1;
                let u_at = |r: f64| {
                    let s = r - st.left;
                    a * (st.k * s).cos() + b * (st.k * s).sin()
                };
```

Mapping of symbols to the excerpt: $u_n(r)$ is closure `u_at(r)`, return value; a_ℓ is `a`, which is `st.u`; b_ℓ is `b`, which is `st.q / (st.d * st.k)`; k_ℓ is `st.k`; s is `s`; $R_{\ell-1}$ is `st.left`; R_ℓ is `st.right`.

Here `st` is the `LayerStart` of the layer, from the unscaled shot of line 329 (one pass of `shoot` per eigenvalue).

4.12 Scale of the eigenfunction

Operation C.12. Classification: IMPLEMENTED (implicit). Equations: (4.9), (4.20). Rust function: `shoot`, `ModalRelease::new`. Pinned lines: src/analytical/sturm_liouville.rs, line 134; src/analytical/sturm_liouville.rs, line 352.

Final equation.

$$(u, q)\,(r = 0) = (0, 1). \tag{4.9}$$

$$I_{1,n} = \int_0^{R_1} r\,u_n\,dr, \qquad I_n = \int_0^{R} r\,u_n\,dr, \qquad N_n = \int_0^{R} u_n^2\,dr. \tag{4.20}$$

The scale is fixed by $q(0) = 1$ of Eq. (4.9) and divides out in the weight w_n of Eq. (4.22).

Pseudocode.

```
# Eq. (4.9): q(0) = 1 fixes the overall size
1. Start each eigenfunction with (u, q) = (0, 1) at r = 0
2. Every mode is built with this same rule, so its size is fixed
3. In the weight w[n] = 3 * I1[n] * I[n] / (R[1]^3 * N[n]), I1 and I each grow in proportion
   to the
   size of u and N grows with its square, so the size cancels  # Eqs. (4.20), (4.22)
```

Implementation detail: there is no separate normalisation step. The scale is set only by `q(0) = 1`.

Rust.

```rust
// src/analytical/sturm_liouville.rs, line 134
    let (mut u, mut q) = (0.0, 1.0);
```

```rust
// src/analytical/sturm_liouville.rs, line 352
            let w = 3.0 * i1 * moment / (r1.powi(3) * norm);
```

Mapping of symbols to the excerpt: $q(0) = 1$ is second element of the pair `(0.0, 1.0)`; w_n is `w`; N_n is `norm`; $I_{1,n}$ is `i1`; I_n is `moment`.

IMPLEMENTED (implicit): no normalisation statement exists. The scale of u_n is fixed by line 134 and cancels in line 352 because w_n is a ratio of two quadratic forms in u_n.

4.13 Quadrature

Operation C.13. Classification: IMPLEMENTED. Equations: (4.24). Rust function: `gauss_legendre_16, integrate`. Pinned lines: src/analytical/sturm_liouville.rs, lines 266 to 292; src/analytical/sturm_liouville.rs, lines 294 to 304; src/analytical/sturm_liouville.rs, line 340.

Final equation.

$$\int_{R_{\ell-1}}^{R_\ell} f(r)\,dr \approx \sum_{i=1}^{N_{\mathrm{p},\ell}} \frac{h}{2} \sum_{m=1}^{16} \omega_m\, f\!\left(\mathrm{mid}_i + \frac{h}{2} x_m\right), \qquad N_{\mathrm{p},\ell} = \left\lceil \frac{k_\ell L_\ell}{\pi/2} \right\rceil + 1, \tag{4.24}$$

Pseudocode.

```
# Goal: integral of f(r) over layer l, from R[l-1] to R[l]
# Gauss-Legendre rule, 16 points
GIVEN  16 nodes x[m] and weights omega[m] on [-1, 1]
# Eq. (4.24): one panel per quarter oscillation, plus one
1. Np = ceil( k[l] * L[l] / (pi/2) ) + 1
# width of each panel
2. h  = L[l] / Np
3. total = 0
4. FOR panel i = 1 to Np:
       4a. mid = midpoint of panel i
       4b. panel_sum = 0
       4c. FOR m = 1 to 16:
              panel_sum = panel_sum + omega[m] * f( mid + (h/2) * x[m] )  # Eq. (4.24)
       4d. total = total + (h/2) * panel_sum                              # Eq. (4.24)
# add the totals of the five layers to cover 0..R
5. RETURN total
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 266 and 270 to 290
fn gauss_legendre_16() -> ([f64; 16], [f64; 16]) {
    // ...
    for i in 0..n {
        let mut z = ((i as f64 + 0.75) / (n as f64 + 0.5) * PI).cos();
        let mut dp = 1.0;
        for _ in 0..100 {
            let (mut p_prev, mut p) = (1.0, z);
            for k in 2..=n {
                let kf = k as f64;
                let pk = ((2.0 * kf - 1.0) * z * p - (kf - 1.0) * p_prev) / kf;
                p_prev = p;
                p = pk;
            }
            dp = n as f64 * (z * p - p_prev) / (z * z - 1.0);
            let dz = p / dp;
            z -= dz;
            if dz.abs() < 1e-15 {
                break;
            }
        }
        x[i] = z;
        w[i] = 2.0 / ((1.0 - z * z) * dp * dp);
    }
```

```rust
// src/analytical/sturm_liouville.rs, lines 294 to 304
fn integrate(a: f64, b: f64, panels: usize, f: impl Fn(f64) -> f64) -> f64 {
    let (x, w) = gauss_legendre_16();
    let h = (b - a) / panels as f64;
    let mut total = 0.0;
    for p in 0..panels {
        let lo = a + p as f64 * h;
        let (mid, half) = (lo + 0.5 * h, 0.5 * h);
        total += half * (0..16).map(|i| w[i] * f(mid + half * x[i])).sum::<f64>();
    }
    total
}
```

```rust
// src/analytical/sturm_liouville.rs, line 340
                let panels = ((st.k * (st.right - st.left)) / (0.5 * PI)).ceil() as usize + 1;
```

Mapping of symbols to the excerpt: x_m is `x[i]`; ω_m is `w[i]`; Legendre recurrence P_k is `p_prev`, `p`, `pk`; P_{16}' is `dp`; Newton step is `dz`, with stop `dz.abs() < 1e-15`; h is `h`; $N_{\mathrm{p},\ell}$ is `panels`; mid_i is `mid`, with half-width `half`; f is `f`; L_ℓ, k_ℓ is `st.right - st.left`, `st.k`.

The first block is the node and weight generator (lines 266 to 292, shown without the declarations at lines 267 to 269 and the closing lines 291 and 292); the second is `integrate`; the third is the panel count. The nodes are rebuilt on every call of `integrate`.

4.14 Norm and moments

Operation C.14. Classification: IMPLEMENTED. Equations: (4.20). Rust function: `ModalRelease::new`. Pinned lines: src/analytical/sturm_liouville.rs, lines 337 to 351.

Final equation.

$$I_{1,n} = \int_0^{R_1} r\,u_n\,dr, \qquad I_n = \int_0^{R} r\,u_n\,dr, \qquad N_n = \int_0^{R} u_n^2\,dr. \tag{4.20}$$

Pseudocode.

```
FOR each mode n:
    1. I1[n] = integral over 0..R[1] of r * u_n dr  # Eq. (4.20): kernel only
    2. I[n]  = integral over 0..R    of r * u_n dr  # Eq. (4.20): whole particle
    3. N[n]  = integral over 0..R    of u_n^2 dr    # Eq. (4.20): the norm
    # each integral is the sum of the quadrature of row C.13 over the layers it covers
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 337 to 351
            let (mut norm, mut moment, mut i1) = (0.0, 0.0, 0.0);
            for (idx, st) in starts.iter().enumerate() {
                let (a, b) = (st.u, st.q / (st.d * st.k));
                let panels = ((st.k * (st.right - st.left)) / (0.5 * PI)).ceil() as usize + 1;
                let u_at = |r: f64| {
                    let s = r - st.left;
                    a * (st.k * s).cos() + b * (st.k * s).sin()
                };
                norm += integrate(st.left, st.right, panels, |r| u_at(r).powi(2));
                let m = integrate(st.left, st.right, panels, |r| r * u_at(r));
                moment += m;
                if idx == 0 {
                    i1 = m;
                }
            }
```

Mapping of symbols to the excerpt: N_n is `norm`; I_n is `moment`; $I_{1,n}$ is `i1`; u_n is closure `u_at`; r is the closure argument `r`; layer index is `idx` (0 for the kernel, so `idx == 0` means $\ell = 1$); integral over a layer is `integrate(st.left, st.right, panels, ...)`.

4.15 Modal weight

Operation C.15. Classification: IMPLEMENTED. Equations: (4.22). Rust function: `ModalRelease::new`. Pinned lines: src/analytical/sturm_liouville.rs, line 325; src/analytical/sturm_liouville.rs, lines 352 to 356.

Final equation.

$$F(t) = 1 - \sum_{n=1}^{K} w_n \, e^{-\Lambda_n t}, \qquad w_n = \frac{3\, I_{1,n}\, I_n}{R_1^3\, N_n}, \tag{4.22}$$

Pseudocode.

```
FOR each mode n = 1 to K:
    1. w[n] = 3 * I1[n] * I[n] / (R[1]^3 * N[n])                  # Eq. (4.22)
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 325 and 352 to 356
        let r1 = layers[0].outer_radius;
            // ...
            let w = 3.0 * i1 * moment / (r1.powi(3) * norm);
            if !w.is_finite() {
                bail!("non-finite modal weight at Lambda = {lam:.6e}");
            }
            weights.push(w);
```

Mapping of symbols to the excerpt: w_n is `w`, pushed onto `weights`; R_1 is `r1`, which is `layers[0].outer_radius`; $I_{1,n}$, I_n, N_n is `i1`, `moment`, `norm`; K is `lams.len()`.

Line 325 is the definition of `r1`; lines 353 to 355 are the finiteness check that aborts with an error.

4.16 Time evolution and release fraction

Operation C.16. Classification: IMPLEMENTED. Equations: (4.22). Rust function: `remaining_fraction`, `release_fraction`. Pinned lines: src/analytical/sturm_liouville.rs, lines 382 to 393.

Final equation.

$$F(t) = 1 - \sum_{n=1}^{K} w_n \, e^{-\Lambda_n t}, \qquad w_n = \frac{3\, I_{1,n}\, I_n}{R_1^3\, N_n}, \tag{4.22}$$

Pseudocode.

```
GIVEN  time t, weights w[1..K], eigenvalues Lambda[1..K]
# Eq. (4.22): each mode fades at its own rate
1. remaining = sum over n = 1..K of w[n] * exp(-Lambda[n] * t)
# Eq. (4.22): released = 1 - what is left
2. F = 1 - remaining
3. RETURN F
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 382 to 393
    pub fn remaining_fraction(&self, t: f64) -> f64 {
        self.eigenvalues
            .iter()
            .zip(&self.weights)
            .map(|(l, w)| w * (-l * t).exp())
            .sum()
    }

    /// Cumulative released fraction F(t) = 1 - sum_n w_n exp(-Lambda_n t).
    pub fn release_fraction(&self, t: f64) -> f64 {
        1.0 - self.remaining_fraction(t)
    }
```

Mapping of symbols to the excerpt: $F(t)$ is return value of `release_fraction`; $\sum_n w_n e^{-\Lambda_n t}$ is return value of `remaining_fraction`; Λ_n is `l`, from `self.eigenvalues`; w_n is `w`, from `self.weights`; t is `t`; K is the common length of the two zipped vectors.

There is no clamp of F to $[0, 1]$ (row C.21).

4.17 Truncation and validity time

Operation C.17. Classification: IMPLEMENTED. Equations: (4.26). Rust function: `exact_release`, `min_valid_time`. Pinned lines: src/analytical/sturm_liouville.rs, lines 377 to 379; src/analytical/sturm_liouville.rs, lines 399 to 418.

Final equation.

$$\Lambda_{\max} = \frac{45}{t_{\min}}, \qquad t_{\text{valid}} = \frac{40}{\Lambda_{\text{cov}}}, \qquad F^{\text{ex}}(t) \text{ is reported only for } t \geq t_{\text{valid}}. \tag{4.26}$$

Pseudocode.

```
GIVEN  t_min = the smallest positive time requested
# Eq. (4.26): upper limit of the eigenvalues wanted
1. Lambda_max = 45 / t_min
2. Find Lambda_cov (row C.7) and keep the modes up to Lambda_cov
# Eq. (4.26): earliest time the series is trusted
3. t_valid = 40 / Lambda_cov
4. For a time t:  IF t >= t_valid: report F_ex(t) from row C.16  # Eq. (4.26)
                  # F_ex is reported only for t >= t_valid
                  ELSE: report no value
```

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 377 to 379
    pub fn min_valid_time(&self) -> f64 {
        40.0 / self.lambda_covered
    }
```

```rust
// src/analytical/sturm_liouville.rs, lines 399 to 418
pub fn exact_release(
    layers: &[Layer],
    times: &[f64],
    max_modes: usize,
) -> Result<Vec<Option<f64>>> {
    let t_min = times
        .iter()
        .copied()
        .filter(|t| *t > 0.0)
        .fold(f64::INFINITY, f64::min);
    if !t_min.is_finite() {
        bail!("need at least one positive time");
    }
    let series = ModalRelease::new(layers, 45.0 / t_min, max_modes)?;
    let valid = series.min_valid_time();
    Ok(times
        .iter()
        .map(|&t| (t >= valid).then(|| series.release_fraction(t)))
        .collect())
}
```

Mapping of symbols to the excerpt: $t_{\min}$ is `t_min`; $\Lambda_{\max}$ is the argument `45.0 / t_min`; Λ_{cov} is `self.lambda_covered`; t_{valid} is return value of `min_valid_time`; local `valid`; $F^{\mathrm{ex}}(t)$ is `series.release_fraction(t)`, wrapped in `Option<f64>`; $K_{\max}$ is `max_modes`.

The literals 45 and 40 are at lines 412 and 378. `Some(F)` is returned only for `t >= valid` (line 416).

4.18 Flux ratio and accuracy floor

Operation C.18. Classification: IMPLEMENTED (heuristic, CLAIMED ONLY as a bound). Equations: (4.27), (4.28). Rust function: `ModalRelease::new, accuracy_floor`. Pinned lines: src/analytical/sturm_liouville.rs, lines 330 to 336; src/analytical/sturm_liouville.rs, lines 371 to 373.

Final equation.

$$\rho = \frac{|q - D\,u/R_\ell|}{|q| + |D\,u/R_\ell|}, \qquad \rho_{\min} = \min_{n,\,\ell} \rho, \tag{4.27}$$

$$\varepsilon_F = \frac{2.2 \times 10^{-16}}{\min(\rho_{\min}, 1)}. \tag{4.28}$$

Pseudocode.

```
rho_min = infinity
FOR each mode n and each interface l = 1 to 4:
    # outer side of the interface (item I15 of Table 7.1)
    1. take u, the post-jump q = q+ and D = D[l+1] at r = R[l]
    # Eq. (4.27): near 0 when the two terms nearly cancel,
    # which is where digits are lost
    2. rho = |q - D*u/R[l]| / ( |q| + |D*u/R[l]| )
    # Eq. (4.27): minimum over modes and layers
    3. rho_min = smaller of rho_min and rho
# Eq. (4.28): machine rounding size, enlarged by cancellation
4. eps_F = 2.2e-16 / min(rho_min, 1)
```

Implementation detail: this is a rule of thumb. The bound itself is CLAIMED ONLY.

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 330 to 336
            for st in starts.iter().skip(1) {
                let scale = st.q.abs() + (st.d * st.u / st.left).abs();
                if scale > 0.0 {
                    let flux = (st.q - st.d * st.u / st.left).abs();
                    min_flux_ratio = min_flux_ratio.min(flux / scale);
                }
            }
```

```rust
// src/analytical/sturm_liouville.rs, lines 371 to 373
    pub fn accuracy_floor(&self) -> f64 {
        2.2e-16 / self.min_flux_ratio.min(1.0)
    }
```

Mapping of symbols to the excerpt: ρ is `flux / scale`; $q = q^+$ (post-jump value) is `st.q`; $D = D_{\ell+1}$ is `st.d`; u is `st.u`; R_ℓ is `st.left` (the inner radius of layer $\ell + 1$); $\rho_{\min}$ is `min_flux_ratio`; ε_F is return value of `accuracy_floor`; the interface loop is `starts.iter().skip(1)` (starts 1 to 4, the four interfaces).

The literal `2.2e-16` at line 372 is a rounded machine epsilon, and the floor is a heuristic (CLAIMED ONLY as a bound).

4.19 Robin root equation (Case B reference)

Operation C.19. Classification: IMPLEMENTED (unit test only). Equations: (4.31). Rust function: `robin_residual`, `robin_roots`. Pinned lines: src/analytical/sturm_liouville.rs, lines 34 to 36; src/analytical/sturm_liouville.rs, lines 40 to 79.

Final equation.

$$z \cot z = 1 - \mathrm{Bi}. \tag{4.31}$$

Pseudocode.

```
GIVEN  Biot number Bi = h_m * R / D
1. residual(z) = z * cot(z) - (1 - Bi)       # Eq. (4.31) moved to one side
2. A root is a value z with residual(z) = 0  # Eq. (4.31): z * cot(z) = 1 - Bi
3. Find the roots z                          # used in a unit test as a Case B reference only
```

Implementation detail: Eq. (4.31) does not say how the roots are located.

Rust.

```rust
// src/analytical/sturm_liouville.rs, lines 34 to 36
pub fn robin_residual(z: f64, bi: f64) -> f64 {
    z / z.tan() - (1.0 - bi)
}
```

```rust
// src/analytical/sturm_liouville.rs, lines 43 to 48, 52, 56 to 66 and 67 to 68
    for n in 0..(count + 4) {
```

```
        let lo = n as f64 * PI + eps;
        let hi = (n as f64 + 1.0) * PI - eps;
        let segments = 128;
        let mut x0 = lo;
        let mut f0 = robin_residual(x0, bi);
            // ...
            if f0.is_finite() && f1.is_finite() && f0 * f1 < 0.0 {
                // ...
                for _ in 0..80 {
                    let m = 0.5 * (a + b);
                    let fm = robin_residual(m, bi);
                    if fa * fm <= 0.0 {
                        b = m;
                    } else {
                        a = m;
                        fa = fm;
                    }
                }
                let root = 0.5 * (a + b);
                // ...
                if root > 1e-6 && roots.last().is_none_or(|x: &f64| (root - *x).abs() > 1e-6) {
                    roots.push(root);
```

Mapping of symbols to the excerpt: z is `z`; Bi is `bi`; residual $z\cot z-(1-\mathrm{Bi})$ is return value of `robin_residual`; scan intervals is `lo`, `hi`, `x0`, `x1`, `segments` (= 128); bisection is `a`, `b`, `m`, `fa`, `fm`, 80 iterations; accepted root is `root`.

Used only by the unit test at lines 447 to 454; neither `main` nor `examples/verify.rs` calls it. The second block shows part of `robin_roots`: the loop head, the sign-change test, the bisection and the acceptance test.

4.20 Orthogonality, expansion coefficient A_n, series for $c(r,t)$, inventory series, closed-form integrals

Operation C.20. Classification: DERIVED NOT IMPLEMENTED. Equations: (4.17), (4.18), (4.19), (4.21), (4.25). Rust function: none.

Final equation.

$$\int_0^R r^2\phi_m\phi_n\,dr=\int_0^R u_m\,u_n\,dr=0,\qquad m\neq n. \tag{4.17}$$

$$A_n=\frac{c_0\int_0^{R_1} r\,u_n\,dr}{\int_0^R u_n^2\,dr}=\frac{c_0\,I_{1,n}}{N_n}, \tag{4.18}$$

$$c(r,t)=\sum_{n=1}^{\infty}A_n\,\frac{u_n(r)}{r}\,e^{-\Lambda_n t}. \tag{4.19}$$

$$M(t)=\sum_n A_n e^{-\Lambda_n t}\int_0^R\frac{u_n}{r}\,4\pi r^2\,dr=4\pi\sum_n A_n\,I_n\,e^{-\Lambda_n t}. \tag{4.21}$$

$$\int_0^L u^2\,ds = \frac{(a^2+b^2)L}{2} + \frac{(a^2-b^2)\sin(2kL)}{4k} + \frac{ab\,(1-\cos(2kL))}{2k},$$

$$\int_0^L (R_{\ell-1}+s)\,u\,ds = \frac{R_{\ell-1}}{k}\left[a\sin(kL) + b\,(1-\cos(kL))\right] + a\left[\frac{L\sin(kL)}{k} + \frac{\cos(kL)-1}{k^2}\right] + b\left[\frac{\sin(kL)}{k^2} - \frac{L\cos(kL)}{k}\right]. \tag{4.25}$$

No Rust function computes these equations.

Pseudocode.

```
NOT IMPLEMENTED. No Rust function computes these. Description of the equations only:
# Eq. (4.17)
1. Different modes are orthogonal:  integral of u_m * u_n dr = 0 for m not equal to n
# Eq. (4.18): how much of mode n the start state holds
2. Expansion coefficient: A[n] = c0 * I1[n] / N[n]
# Eq. (4.19)
3. Concentration: c(r, t) = sum over n of A[n] * u_n(r)/r * exp(-Lambda[n] * t)
# Eq. (4.21)
4. Inventory: M(t) = 4 * pi * sum over n of A[n] * I[n] * exp(-Lambda[n] * t)
5. Integrals over one layer could be written in closed form (formulas of Eq. (4.25), with a,
   b, k, L of that layer)
   instead of by the quadrature of row C.13
```

Rust. No Rust function computes this row.

DERIVED NOT IMPLEMENTED. A search of `sturm_liouville.rs` for orthogon, expansion, tail, partition and completeness finds only comments (lines 7, 8, 16 and 17). The weight of row C.15 absorbs A_n.

4.21 Completeness proof, tail bound, clamp of negative F, partition coefficients, Robin release for the layered particle

Operation C.21. Classification: FUTURE IMPROVEMENT. Equations: (4.23), (3.10). Rust function: none.

Final equation.

$$\lim_{K\to\infty} \sum_{n=1}^{K} w_n = 1. \tag{4.23}$$

$$-D_5\,\frac{\partial c_5}{\partial r}(R,t) = h_{\mathrm{m}}\left[c_5(R,t) - c_\infty\right]. \tag{3.10}$$

Eq. (4.23) is CLAIMED ONLY as a theorem for the layered sphere. The other items have no equation of their own.

Pseudocode.

```
NOT IMPLEMENTED. Description of the equations only:
# Eq. (4.23)
1. Completeness: the weights add to 1 when all modes are kept:
   limit of sum_{n=1..K} w[n] = 1 as K grows
   (CLAIMED ONLY as a theorem for the layered sphere; this means F(0) = 0)
# Eq. (3.10)
2. Robin release for the layered particle: -D5 * dc5/dr (R, t) = h_m * (c5(R, t) - c_inf)
3. Tail bound, clamp of negative F, partition coefficients: no equation, so no step is written
```

Rust. No Rust function computes this row.

FUTURE IMPROVEMENT, no Rust function. The word clamp occurs once, at line 258, where it limits the integer count `nm` to $[n_a, n_b]$; it is unrelated to a clamp of F.

5 Program flow and comparison (Code A)

Code A is the part of the program that is neither Code B nor Code C. It consists of `src/main.rs` (the driver), `src/config.rs` (reading and validation of the input), `src/postprocessing/comparison.rs` (the comparison of the two methods and the refinement study) and `src/postprocessing/plots.rs` (the figures). Table 5.1 follows one run of the program from the input file to the last printed line.

Table 5.1 Program flow of one run.

Step	What happens	Rows	Rust
1	Read the input file, reject placeholders and validate the values	A.1, A.2	`Config::load` at src/main.rs, line 37
2	Build the mesh	B.1 to B.7	`fv::build_mesh` at src/main.rs, line 38
3	Run the Backward Euler time stepping and collect the output rows	B.8 to B.19	`fv::run` at src/main.rs, line 39
4	Write `timeseries.csv` and `final_profile.csv`, then print the report of the finite-volume run, which includes Δt_{FE} of row B.20	A.4, A.5, B.20	src/main.rs, lines 29 to 87
5	Stop here if the flag `--fv-only` was given	none	src/main.rs, line 99
6	Compare with the series: build the layer list, evaluate F^{ex} for every output time with $t_n \geq t_{\mathrm{valid}}$, write the comparison file and the figure	A.3, A.6, C.1 to C.19	`compare::compare_release` at src/main.rs, line 104
7	Print a warning if the largest difference is below $3\varepsilon_F$	A.7	src/main.rs, lines 124 to 128
8	If the comparison fails, print `analytical_cross_check=SKIPPED` and end with exit status 0, which also skips the study (item I25)	none	src/main.rs, lines 135 to 138
9	Stop here if the flag `--no-convergence` was given, or if the estimated cost exceeds the limit `STUDY_COST_LIMIT`	A.12	src/main.rs, lines 141 to 149
10	Run the refinement study with Richardson extrapolation and observed orders, write `convergence_study.csv` and the figure	A.8 to A.11	`compare::convergence_study` at src/main.rs, line 151

Table 5.2 lists the 15 operations A.1 to A.15. Rows A.1 and A.14 have no equation.

Row A.15 is absent from v1.0.0.

Table 5.2 Operations of Code A in the order of the list A.1 to A.15. A.1 is Section 5.1, A.15 is Section 5.15.

No.	Final mathematical formulation	Pseudocode	Corresponding Rust implementation
A.1	no equation (reading and parsing step)	read the TOML text; stop if a placeholder remains; parse	`Config::load`, `unfilled_placeholders`, src/config.rs, lines 62 to 77 and 1 more range (Section 5.1). IMPLEMENTED
A.2	$0 = R_0 < R_1 < \cdots < R_5 = R$, $L_\ell = R_\ell - R_{\ell-1}$, Eqs. (1.1), (1.2); other checks have no equation	check five regions, increasing radii, `D[l] > 0`, cells, times, `c0`, `T`	`Config::validate`, src/config.rs, lines 79 to 113. IMPLEMENTED (T read, used in no formula)
A.3	$R_1, \dots, R_5$ in order, Eq. (1.1)	build the list of five layers; hand it to Code C	`layers_from_config`, src/postprocessing/comparison.rs, lines 15 to 23. IMPLEMENTED
A.4	output rule n mod $n_{\text{out}} = 0$ or $n = N_t$, Eq. (5.25)	load, mesh, run; write `timeseries.csv` and `final_profile.csv`	`main`, src/main.rs, lines 29 to 73. IMPLEMENTED
A.5	Δt_{FE}, Eq. (6.6); ε^n and F^n, Eq. (5.24)	print `dt_FE`, `eps` and `F` of the final row	`main`, src/main.rs, lines 75 to 87. IMPLEMENTED
A.6	$e(t_n) = \lvert F^{\text{FV}}(t_n) - F^{\text{ex}}(t_n)\rvert$ for $t_n \geq t_{\text{valid}}$, Eq. (7.1)	skip rows with `t_n < t_valid`; `e = abs(F_FV - F_ex)`	`compare_release`, src/postprocessing/comparison.rs, lines 47 to 91. IMPLEMENTED
A.7	$\max_n e(t_n) < 3\,\varepsilon_F$ gives a warning, Eq. (7.2)	`IF e_max < 3 eps_F`: print the warning	`main`, src/main.rs, lines 124 to 128. IMPLEMENTED
A.8	$F_{\text{RE}} = \frac{1}{3}\,(8F(\Delta t/4) - 6F(\Delta t/2) + F(\Delta t))$, Eq. (6.10)	run with `dt`, `dt/2`, `dt/4`; combine with weights 8, -6, 1 over 3	`time_extrapolated_release`, src/postprocessing/comparison.rs, lines 137 to 157. IMPLEMENTED
A.9	F_{RE}, Eq. (6.10); $p_k = \log_2(\text{err}_k/\text{err}_{k+1})$, Eq. (6.11)	run spatial and temporal refinements; record `err[k]`; compute `p[k]`	`convergence_study`, src/postprocessing/comparison.rs, lines 178 to 221 and 1 more range (Section 5.9). IMPLEMENTED
A.10	$p_k = \log_2 \dfrac{\text{err}_k}{\text{err}_{k+1}}$, Eq. (6.11)	`p[k] = log2(err[k] / err[k+1])` for neighbouring levels	`ConvergenceStudy::orders`, src/postprocessing/comparison.rs, lines 112 to 118. IMPLEMENTED
A.11	$\text{err}_k > 3\,\varepsilon_F$ for every k, Eq. (6.12)	`resolved = TRUE`; set `FALSE` if any `err[k] <= 3 eps_F`	`ConvergenceStudy::resolved`, src/postprocessing/comparison.rs, lines 120 to 126. IMPLEMENTED
A.12	$W = \dfrac{t_{\text{end}}}{\Delta t_{\text{in}}} N_c$; W_{study} with factor 1.75, Eq. (6.14)	`W_study = 1.75 spatial_sum + temporal_sum`; skip the study above a limit	`run_cost`, `study_cost`, src/postprocessing/comparison.rs, lines 159 to 176 and 2 more ranges (Section 5.12). IMPLEMENTED (factor 1.75 should be 7, item K1)

No.	Final mathematical formulation	Pseudocode	Corresponding Rust implementation
A.13	$\Lambda_{\max} = 45/t_{\min}$, $t_{\text{valid}} = 40/\Lambda_{\text{cov}}$, Eq. (4.26)	build the series for one time `t`; return `F_ex(t)` if `t >= t_valid`	`exact_release_at`, src/postprocessing/comparison.rs, lines 223 to 227. IMPLEMENTED (not called by `main`)
A.14	no equation (presentation)	write result files; draw SVG figures	`main`, `plots`, src/main.rs, lines 89 to 97 and 5 more ranges (Section 5.14). IMPLEMENTED (presentation, no equation)
A.15	none; same-mesh temporal reference, relative error and an integer check on $t_{\text{end}}/\Delta t_{\text{in}}$ are absent	none	none. FUTURE IMPROVEMENT

Each operation below repeats the equation, the pseudocode and the Rust excerpt of its row in Table 5.2.

5.1 Read input, reject unfilled placeholders

Operation A.1. Classification: IMPLEMENTED. Equations: none. Rust function: `Config::load, unfilled_placeholders`. Pinned lines: src/config.rs, lines 62 to 77; src/config.rs, lines 43 to 59.

Final equation.

No equation. The operation reads the TOML file, rejects any value that is still the placeholder, and parses the text (assumptions A1 to A8 of Table 1.2).

Pseudocode.

```
No equation. The steps follow the row text:
1. Read the TOML input file as text
2. IF any value is still the placeholder: stop with an error
3. Parse the text into the configuration values
```

Rust.

```rust
// src/config.rs, lines 62 to 77
    pub fn load(path: impl AsRef<Path>) -> Result<Self> {
        let raw = fs::read_to_string(path.as_ref())
            .with_context(|| format!("reading {}", path.as_ref().display()))?;
        let missing = unfilled_placeholders(&raw);
        if !missing.is_empty() {
            bail!(
                "{} literature value(s) in {} are still {PLACEHOLDER}:\n  {}",
                missing.len(),
                path.as_ref().display(),
                missing.join("\n  ")
            );
        }
        let cfg: Self = toml::from_str(&raw).context("parsing TOML")?;
        cfg.validate()?;
        Ok(cfg)
    }
```

```rust
// src/config.rs, lines 45 to 59
fn unfilled_placeholders(raw: &str) -> Vec<String> {
    let mut last_instruction = String::new();
    let mut found = Vec::new();
    for (i, line) in raw.lines().enumerate() {
        let trimmed = line.trim_start();
        if trimmed.starts_with('#') {
```

```
            if trimmed.starts_with("# INSERT") && trimmed.ends_with("BELOW") {
                last_instruction = trimmed.trim_start_matches('#').trim().to_string();
            }
        } else if trimmed.contains(PLACEHOLDER) {
            found.push(format!("line {}: {}", i + 1, last_instruction));
        }
    }
    found
}
```

Mapping of symbols to the excerpt: input path is `path`; file text is `raw`; list of unfilled lines is `missing`; placeholder text is `PLACEHOLDER` (`"INSERT_VALUE_HERE"`); parsed configuration is `cfg`; instruction text is `last_instruction`.

No equation. The first block is `Config::load`, the second `unfilled_placeholders`.

5.2 Validate input (five regions, radii, D_ℓ, cell counts, times, c_0, T)

Operation A.2. Classification: IMPLEMENTED (T read, used in no formula). Equations: (1.1), (1.2). Rust function: `Config::validate`. Pinned lines: src/config.rs, lines 79 to 113.

Final equation.

$$0 = R_0 < R_1 < R_2 < R_3 < R_4 < R_5 = R \tag{1.1}$$

$$L_\ell = R_\ell - R_{\ell-1}, \qquad \ell = 1, \dots, 5 \tag{1.2}$$

The other checks (positive D_ℓ, at least one cell per layer, positive finite times, $c_0 \geq 0$) are inequalities stated in words in Part VII and carry no equation number.

Pseudocode.

```
1. There are five regions                                   # layers 1..5
2. Check 0 = R[0] < R[1] < ... < R[5] = R                   # Eq. (1.1)
3. L[l] = R[l] - R[l-1] for l = 1..5                        # Eq. (1.2)
4. Checks stated in words (no equation number):
       D[l] > 0 for every layer
       at least one cell in every layer
       times positive and finite
       c0 >= 0
5. T is read but enters no formula
```

Rust.

```rust
// src/config.rs, lines 79 to 99
    pub fn validate(&self) -> Result<()> {
        if self.regions.len() != 5 {
            bail!("exactly five physical regions are required");
        }
        if self.species.trim().is_empty() {
            bail!("species must not be empty");
        }
        if !(self.temperature_k.is_finite() && self.temperature_k > 0.0) {
            bail!("temperature_k must be a positive finite number (kelvin)");
        }
        let mut last = 0.0;
        for r in &self.regions {
            if !(r.outer_radius_m.is_finite() && r.diffusivity_m2_s.is_finite())
                || r.outer_radius_m <= last
                || r.diffusivity_m2_s <= 0.0
                || r.cells == 0
            {
                bail!("invalid region {}", r.name);
            }
            last = r.outer_radius_m;
        }
```

```rust
// src/config.rs, lines 100 to 113
        let t = &self.time;
        if !(t.dt_s.is_finite() && t.end_time_s.is_finite())
            || t.dt_s <= 0.0
            || t.end_time_s <= 0.0
            || t.output_every_steps == 0
        {
            bail!("invalid time controls");
        }
        let c0 = self.initial.kernel_concentration_mol_m3;
        if !c0.is_finite() || c0 < 0.0 {
            bail!("initial concentration must be nonnegative and finite");
        }
        Ok(())
    }
```

Mapping of symbols to the excerpt: R_ℓ is `r.outer_radius_m`; $R_{\ell-1}$ is `last` (starts at 0.0); D_ℓ is `r.diffusivity_m2_s`; n_ℓ is `r.cells`; Δt_{in} is `t.dt_s`; t_{end} is `t.end_time_s`; n_{out} is `t.output_every_steps`; c_0 is `c0`; T is `self.temperature_k` (checked only); number of regions is `self.regions.len()`.

The excerpt is split in two blocks only to stay within about 25 lines each; together they cover lines 79 to 113.

5.3 Layer list handed to Code C

Operation A.3. Classification: IMPLEMENTED. Equations: (1.1). Rust function: `layers_from_config`. Pinned lines: src/postprocessing/comparison.rs, lines 15 to 23.

Final equation.

$$0 = R_0 < R_1 < R_2 < R_3 < R_4 < R_5 = R \tag{1.1}$$

Pseudocode.

```
1. Build a list of the five layers in order, layer 1 first  # radii ordered as in Eq. (1.1)
2. Each entry holds the outer radius R[l] of that layer     # Eq. (1.1)
3. Hand the list to Code C
```

Implementation detail: Code C also needs `D[l]` for each layer (Eq. (4.4)). No equation of this row shows it.

Rust.

```rust
// src/postprocessing/comparison.rs, lines 15 to 23
pub fn layers_from_config(cfg: &Config) -> Vec<Layer> {
    cfg.regions
        .iter()
        .map(|r| Layer {
            outer_radius: r.outer_radius_m,
            diffusivity: r.diffusivity_m2_s,
        })
        .collect()
}
```

Mapping of symbols to the excerpt: R_ℓ is field `outer_radius` of `Layer`, from `r.outer_radius_m`; D_ℓ is field `diffusivity` of `Layer`, from `r.diffusivity_m2_s`; list of layers is return value `Vec<Layer>`.

5.4 Main run: load, mesh, run, write `timeseries.csv` and `final_profile.csv`

Operation A.4. Classification: IMPLEMENTED. Equations: (5.25). Rust function: `main`. Pinned lines: src/main.rs, lines 29 to 73.

Final equation.

$$n \bmod n_{\text{out}} = 0 \quad \text{or} \quad n = N_t, \qquad t_n = n\,\Delta t, \tag{5.25}$$

Pseudocode.

```
1. Load the configuration                                          # row A.1
2. Build the mesh                                                  # rows B.1 to B.7
3. Run the time stepping                                           # rows B.11 to B.19
4. FOR each saved output row n (n mod n_out = 0, or n = Nt):       # Eq. (5.25)
       write the row for t_n = n * dt to timeseries.csv
5. Write the final concentration profile to final_profile.csv
```

Implementation detail: Eq. (5.25) covers only the choice of rows in step 4. Writing `final_profile.csv` has no equation, and the contents of that file are taken from the file name only.

Rust.

```rust
// src/main.rs, lines 37 to 44
    let cfg = Config::load(&path)?;
    let mesh = fv::build_mesh(&cfg)?;
    let (rows, final_c) = fv::run(&cfg, &mesh)?;
    fs::create_dir_all(&cfg.output.directory)?;
    let out = Path::new(&cfg.output.directory);

    let ts_path = out.join("timeseries.csv");
    let mut w = csv::Writer::from_path(&ts_path)?;
```

```rust
// src/main.rs, lines 54 to 73
    for (t, d) in &rows {
        w.serialize((
            t,
            d.inventory,
            d.cumulative_release,
            d.release_fraction,
            d.release_rate,
            d.balance_residual,
            d.min_concentration,
        ))?;
    }
    w.flush()?;

    let profile_path = out.join("final_profile.csv");
    let mut p = csv::Writer::from_path(&profile_path)?;
    p.write_record(["cell", "material", "radius_m", "concentration_mol_m3"])?;
```

```
    for (i, (cell, c)) in mesh.cells.iter().zip(&final_c).enumerate() {
        p.serialize((i, cell.material, cell.centroid, c))?;
    }
    p.flush()?;
```

Mapping of symbols to the excerpt: configuration is `cfg`; mesh is `mesh`; time rows and final state is `rows`, `final_c`; $(t_n,$ diagnostics$)$ is the tuple `(t, d)` of each row; n, n_{out}, N_t is inside `fv::run`, `step`, `output_every_steps`, `steps`; output files is `ts_path`, `profile_path`.

The output rule of Eq. (5.25) is evaluated in `fv::run` (row B.19); `main` writes the rows it receives. The excerpt omits lines 29 to 36 (argument handling) and 45 to 53 (header row).

5.5 Console report of the FV run

Operation A.5. Classification: IMPLEMENTED. Equations: (6.6), (5.24). Rust function: `main`. Pinned lines: src/main.rs, lines 75 to 87.

Final equation.

$$\Delta t_{\text{FE}} = \min_j \frac{V_j}{g_j^W + g_j^E}, \tag{6.6}$$

$$\varepsilon^n = M^n + M^n_{\text{rel}} - M^0, \qquad F^n = \frac{M^n_{\text{rel}}}{M^0}. \tag{5.24}$$

Pseudocode.

```
1. Print dt_FE = min over j of V[j] / (gW[j] + gE[j])          # Eq. (6.6): printed once
2. Print the balance residual  eps = M + Mrel - M0             # Eq. (5.24)
3. Print the release fraction  F = Mrel / M0                   # Eq. (5.24)
```

Implementation detail: the equations of this row do not say at which level n the values of step 2 and step 3 are printed.

Rust.

```
// src/main.rs, lines 75 to 87
    let last = &rows.last().expect("initial row").1;
    println!("case={}", cfg.case_name);
    println!("cells={}", mesh.cells.len());
    println!("explicit_dt_bound_s={:.12e}", fv::explicit_dt_bound(&mesh));
    println!("final_release_fraction={:.12e}", last.release_fraction);
    println!("final_inventory_mol={:.12e}", last.inventory);
    println!(
        "max_final_balance_residual_mol={:.12e}",
        last.balance_residual.abs()
    );
    println!("min_final_concentration={:.12e}", last.min_concentration);
    println!("timeseries={}", ts_path.display());
    println!("profile={}", profile_path.display());
```

Mapping of symbols to the excerpt: last stored row is `last`; Δt_{FE} is `fv::explicit_dt_bound(&mesh)`, printed as `explicit_dt_bound_s`; F^{N_t} is `last.release_fraction`, printed as `final_release_fraction`; M^{N_t} is `last.inventory`, printed as `final_inventory_mol`; $|\varepsilon^{N_t}|$ is `last.balance_residual.abs()`, printed as `max_final_balance_residual_mol`.

The key `max_final_balance_residual_mol` suggests a maximum over all steps; the code prints the residual of the last stored row only.

5.6 FV against series comparison

Operation A.6. Classification: IMPLEMENTED. Equations: (7.1). Rust function: `compare_release`. Pinned lines: src/postprocessing/comparison.rs, lines 47 to 91.

Final equation.

$$e(t_n) = \left| F^{\mathrm{FV}}(t_n) - F^{\mathrm{ex}}(t_n) \right|, \qquad t_n \geq t_{\mathrm{valid}}, \qquad F^{\mathrm{FV}}(t_n) = F^n, \tag{7.1}$$

Pseudocode.

```
FOR each output time t_n:
    # Eq. (7.1): compare only where the series is valid
    1. IF t_n < t_valid: skip this time
    2. F_FV = F[n] from the finite-volume run  # Eq. (7.1): F_FV(t_n) = F^n
    3. F_ex = series value at t_n              # row C.16
    4. e(t_n) = | F_FV - F_ex |                # Eq. (7.1)
```

Rust.

```rust
// src/postprocessing/comparison.rs, lines 48 to 49 and 58 to 75
pub fn compare_release(cfg: &Config, fv_rows: &fv::TimeSeries) -> Result<ReleaseComparison> {
    let layers = layers_from_config(cfg);
    // ...
    let t_min = times.iter().copied().fold(f64::INFINITY, f64::min);
    let series = ModalRelease::new(&layers, 45.0 / t_min, MAX_MODES)?;
    let valid = series.min_valid_time();
    let mut rows = Vec::new();
    let (mut max_err, mut final_err) = (0.0f64, f64::NAN);
    for (t, d) in fv_rows.iter().filter(|(t, _)| *t > 0.0) {
        let exact = (*t >= valid).then(|| series.release_fraction(*t));
        if let Some(e) = exact {
            let err = (d.release_fraction - e).abs();
            max_err = max_err.max(err);
            final_err = err;
        }
        rows.push(CompareRow {
            time_s: *t,
            fv: d.release_fraction,
            exact,
        });
    }
```

Mapping of symbols to the excerpt: t_n is `t`; t_{min} is `t_min`; t_{valid} is `valid`; $F^{\mathrm{FV}}(t_n) = F^n$ is `d.release_fraction`; $F^{\mathrm{ex}}(t_n)$ is `exact` (an `Option<f64>`), `e` inside `if let`; $e(t_n)$ is `err`; maximum and last error is `max_err`, `final_err`; series is `series`.

Lines 50 to 57 (the list of compared times) are omitted from the excerpt.

5.7 Floor warning

Operation A.7. Classification: IMPLEMENTED. Equations: (7.2). Rust function: `main`. Pinned lines: src/main.rs, lines 124 to 128.

Final equation.

$$\begin{aligned} &\max_{n:\ t_n \geq t_{\mathrm{valid}}} e(t_n) < 3\,\varepsilon_F \\ &\quad \Longrightarrow \quad \text{print a warning that the differences are not resolved,} \end{aligned} \tag{7.2}$$

Pseudocode.

```
1. e_max = largest e(t_n) over all output times with t_n >= t_valid  # Eq. (7.2)
# Eq. (7.2): the differences are as small as the
# series itself can resolve
2. IF e_max < 3 * eps_F:
       print a warning that the differences are not resolved
```

Rust.

```rust
// src/main.rs, lines 124 to 128
            if cmp.max_abs_error < 3.0 * cmp.accuracy_floor {
                println!(
                    "WARNING: FV-analytical differences are within 3x of the analytical series accuracy
                    ↪  floor; they are not resolved."
                );
            }
```

Mapping of symbols to the excerpt: $\max_n e(t_n)$ is `cmp.max_abs_error`; ε_F is `cmp.accuracy_floor`; factor 3 is the literal `3.0`; t_{valid} is `cmp.min_valid_time_s`, printed as `analytical_valid_from_s`.

5.8 Richardson time extrapolation

Operation A.8. Classification: IMPLEMENTED. Equations: (6.9), (6.10). Rust function: `time_extrapolated_release`. Pinned lines: src/postprocessing/comparison.rs, lines 137 to 157.

Final equation.

$$F(\Delta t) = F_0 + \chi_1 \Delta t + \chi_2 \Delta t^2 + \chi_3 \Delta t^3 + \ldots \tag{6.9}$$

$$F_{\text{RE}} = \frac{1}{3}\left(8F(\Delta t/4) - 6F(\Delta t/2) + F(\Delta t)\right) = F_0 + \frac{1}{8}\chi_3 \Delta t^3 + \ldots \tag{6.10}$$

Pseudocode.

```
# Model of the error (Eq. 6.9): F(dt) = F0 + chi1*dt + chi2*dt^2 + chi3*dt^3 + ...
1. Run the finite-volume model with step dt      and read F(dt)
2. Run it again with step dt/2                   and read F(dt/2)
3. Run it again with step dt/4                   and read F(dt/4)
4. F_RE = ( 8 * F(dt/4) - 6 * F(dt/2) + F(dt) ) / 3               # Eq. (6.10)

# Why these weights (checked by arithmetic):
#   constant term : (8 - 6 + 1) / 3 = 1            so F0 is kept
#   chi1 term     : 8/4 - 6/2 + 1 = 0               removed
#   chi2 term     : 8/16 - 6/4 + 1 = 0              removed
#   chi3 term     : (8/64 - 6/8 + 1) / 3 = 1/8     left over, as in Eq. (6.10)
```

Rust.

```rust
// src/postprocessing/comparison.rs, lines 137 to 157
fn with_dt(cfg: &Config, dt: f64) -> Config {
    let mut c = cfg.clone();
    c.time.dt_s = dt;
    c.time.output_every_steps = usize::MAX;
    c
}

fn final_release(cfg: &Config) -> Result<f64> {
    let mesh = fv::build_mesh(cfg)?;
    let (rows, _) = fv::run(cfg, &mesh)?;
    Ok(rows.last().expect("initial row").1.release_fraction)
}

/// Release fraction with the Backward-Euler time error removed by Richardson
/// extrapolation, (8 F(dt/4) - 6 F(dt/2) + F(dt)) / 3. The error expansion of Backward
/// Euler is a1 dt + a2 dt^2 + ..., so this cancels both leading terms.
/// Used to isolate the spatial error in refinement studies.
pub fn time_extrapolated_release(cfg: &Config) -> Result<f64> {
    let f = |m: f64| final_release(&with_dt(cfg, cfg.time.dt_s * m));
    Ok((8.0 * f(0.25)? - 6.0 * f(0.5)? + f(1.0)?) / 3.0)
}
```

Mapping of symbols to the excerpt: Δt is `cfg.time.dt_s`; step factor 1, $\frac{1}{2}$, $\frac{1}{4}$ is the closure argument `m`, called as `f(1.0)`, `f(0.5)`, `f(0.25)`; $F(\Delta t/4)$, $F(\Delta t/2)$, $F(\Delta t)$ is `f(0.25)?`, `f(0.5)?`, `f(1.0)?`; F_{RE} is return value of `time_extrapolated_release`; one run to F is `final_release(&with_dt(cfg, cfg.time.dt_s * m))`; configuration with a new step is `with_dt`, which sets `output_every_steps` to `usize::MAX`; last release fraction is `final_release`, `rows.last()...release_fraction`.

The excerpt is one contiguous span that contains the two helpers and the function; the formula itself is at line 156.

5.9 Spatial and temporal refinement study

Operation A.9. Classification: IMPLEMENTED. Equations: (6.10), (6.11). Rust function: `convergence_study`. Pinned lines: src/postprocessing/comparison.rs, lines 178 to 221; src/postprocessing/comparison.rs, lines 129 to 135.

Final equation.

$$F_{\mathrm{RE}} = \frac{1}{3}\left(8F(\Delta t/4) - 6F(\Delta t/2) + F(\Delta t)\right) = F_0 + \frac{1}{8}\chi_3 \Delta t^3 + \ldots \tag{6.10}$$

$$p_k = \log_2 \frac{\mathrm{err}_k}{\mathrm{err}_{k+1}}, \tag{6.11}$$

Pseudocode.

```
1. FOR each spatial refinement factor lambda_x (listed in row A.12):
       run the model with that refinement, record err[k]
2. FOR each temporal refinement factor lambda_t (listed in row A.12):
       run the model with that refinement, record err[k]
3. The time-extrapolated value F_RE of Eq. (6.10) is computed by row A.8
4. FOR each level k: p[k] = log2( err[k] / err[k+1] )              # Eq. (6.11), see row A.10
```

Implementation detail: Eqs. (6.10) and (6.11) do not say which reference value `err[k]` is measured against, nor where `F_RE` is used in the study. The symbol table calls `err[k]` the absolute error at refinement level `k`.

Rust.

```rust
// src/postprocessing/comparison.rs, lines 129 to 135
fn scaled_cells(cfg: &Config, factor: f64) -> Config {
    let mut c = cfg.clone();
    for r in &mut c.regions {
        r.cells = ((r.cells as f64 * factor).round() as usize).max(1);
    }
    c
}
```

```rust
// src/postprocessing/comparison.rs, lines 183 to 190 and 192 to 202
pub fn convergence_study(cfg: &Config) -> Result<ConvergenceStudy> {
    let layers = layers_from_config(cfg);
    let t_end = cfg.time.end_time_s;
    let series = ModalRelease::new(&layers, 45.0 / t_end, MAX_MODES)?;
    if t_end < series.min_valid_time() {
        bail!("exact series does not resolve the end time");
    }
    let exact_final = series.release_fraction(t_end);
    // ...
    let mut spatial = Vec::new();
    for f in [0.5, 1.0, 2.0, 4.0, 8.0] {
        let c = scaled_cells(cfg, f);
        let extrapolated = time_extrapolated_release(&c)?;
        spatial.push(ConvergencePoint {
            cells: c.regions.iter().map(|r| r.cells).sum(),
            dt_s: cfg.time.dt_s,
            fv_release: extrapolated,
            abs_error: (extrapolated - exact_final).abs(),
        });
    }
```

```rust
// src/postprocessing/comparison.rs, lines 204 to 214
    let mut temporal = Vec::new();
    for m in [4.0, 2.0, 1.0, 0.5, 0.25] {
        let c = with_dt(cfg, cfg.time.dt_s * m);
        let f = final_release(&c)?;
        temporal.push(ConvergencePoint {
            cells: c.regions.iter().map(|r| r.cells).sum(),
            dt_s: cfg.time.dt_s * m,
            fv_release: f,
            abs_error: (f - exact_final).abs(),
        });
    }
```

Mapping of symbols to the excerpt: λ_x is `f` (loop variable over `[0.5, 1.0, 2.0, 4.0, 8.0]`) and `factor` in `scaled_cells`; λ_t is `m` (loop variable over `[4.0, 2.0, 1.0, 0.5, 0.25]`); F_{RE} is `extrapolated`; $F^{\mathrm{ex}}(t_{\mathrm{end}})$ is `exact_final`; t_{end} is `t_end`; err_k is field `abs_error` of `ConvergencePoint`; $F^{\mathrm{FV}}(t_{\mathrm{end}})$, temporal is `f`, stored as `fv_release`; N_c of a point is field `cells`; step of a point is field `dt_s`.

The first block is `scaled_cells`, the second and third are parts of `convergence_study`. The observed orders p_k of Eq. (6.11) are computed in row A.10.

5.10 Observed order

Operation A.10. Classification: IMPLEMENTED. Equations: (6.11). Rust function: `ConvergenceStudy::orders`. Pinned lines: src/postprocessing/comparison.rs, lines 112 to 118.

Final equation.

$$p_k = \log_2 \frac{\mathrm{err}_k}{\mathrm{err}_{k+1}}, \tag{6.11}$$

Pseudocode.

```
FOR each pair of neighbouring levels k and k+1:
    1. p[k] = log2( err[k] / err[k+1] )  # Eq. (6.11): if halving the step divides the
                                         # error by 2^p, this gives p
```

Rust.

```rust
// src/postprocessing/comparison.rs, lines 112 to 118
    /// Observed order between successive refinements (each refinement halves h or dt).
    pub fn orders(points: &[ConvergencePoint]) -> Vec<f64> {
        points
            .windows(2)
            .map(|w| (w[0].abs_error / w[1].abs_error).log2())
            .collect()
    }
```

Mapping of symbols to the excerpt: p_k is element of the returned `Vec<f64>`; err_k is `w[0].abs_error`; err_{k+1} is `w[1].abs_error`; consecutive points is `points.windows(2)`; $\log_2$ is `.log2()`.

There is no guard against `w[1].abs_error` equal to zero.

5.11 Resolution test

Operation A.11. Classification: IMPLEMENTED. Equations: (6.12). Rust function: `ConvergenceStudy::resolved`. Pinned lines: src/postprocessing/comparison.rs, lines 120 to 126.

Final equation.

$$\mathrm{err}_k > 3\,\varepsilon_F \quad \text{for every } k, \tag{6.12}$$

Pseudocode.

```
1. resolved = TRUE
2. FOR every level k:
       # Eq. (6.12): errors below this size cannot be
       # told apart from the series floor
       IF NOT ( err[k] > 3 * eps_F ):  resolved = FALSE
3. RETURN resolved                                    # TRUE only if every level passes
```

Rust.

```rust
// src/postprocessing/comparison.rs, lines 120 to 126
    /// True when every error is well above the accuracy floor of the exact series, so
    /// observed orders mean something.
    pub fn resolved(&self, points: &[ConvergencePoint]) -> bool {
        points
            .iter()
            .all(|p| p.abs_error > 3.0 * self.accuracy_floor)
    }
```

Mapping of symbols to the excerpt: err_k is `p.abs_error`; ε_F is `self.accuracy_floor`; factor 3 is the literal `3.0`; “for every k” is `.all(...)`.

5.12 Study cost and guard

Operation A.12. Classification: IMPLEMENTED (factor 1.75 should be 7, item K1). Equations: (6.14). Rust function: `run_cost`, `study_cost`. Pinned lines: src/postprocessing/comparison.rs, lines 159 to 176; src/main.rs, line 21; src/main.rs, lines 141 to 150.

Final equation.

$$W = \frac{t_{\mathrm{end}}}{\Delta t_{\mathrm{in}}} N_c, \qquad W_{\mathrm{study}} = 1.75 \sum_{\lambda_{\mathrm{x}} \in \{0.5,1,2,4,8\}} W(\lambda_{\mathrm{x}}) + \sum_{\lambda_{\mathrm{t}} \in \{4,2,1,1/2,1/4\}} W(\lambda_{\mathrm{t}}), \quad (6.14)$$

Pseudocode.

```
# Cost of one run, in cell-steps:
# Eq. (6.14): number of steps x number of cells
1. W = ( t_end / dt_in ) * Nc
# Cost of the whole study:
2. spatial_sum  = sum of W(lambda_x) for lambda_x in {0.5, 1, 2, 4, 8}
3. temporal_sum = sum of W(lambda_t) for lambda_t in {4, 2, 1, 1/2, 1/4}
4. W_study = 1.75 * spatial_sum + temporal_sum  # Eq. (6.14)
```

Implementation detail: the factor 1.75 is what the code uses. Item K1 of Table 7.1 records that it should be 7. The limit that the guard compares `W_study` with is not given by Eq. (6.14), and neither is how `Nc` and `dt` change with a factor `lambda`.

Rust.

```rust
// src/postprocessing/comparison.rs, lines 159 to 176
/// Rough work estimate (cell-steps) for one FV run.
pub fn run_cost(cfg: &Config) -> f64 {
    let cells: usize = cfg.regions.iter().map(|r| r.cells).sum();
    cfg.time.end_time_s / cfg.time.dt_s * cells as f64
}

/// Estimated cell-steps for the whole refinement study.
pub fn study_cost(cfg: &Config) -> f64 {
    let spatial: f64 = [0.5, 1.0, 2.0, 4.0, 8.0]
        .iter()
        .map(|f| run_cost(&scaled_cells(cfg, *f)) * 1.75)
        .sum();
    let temporal: f64 = [4.0, 2.0, 1.0, 0.5, 0.25]
        .iter()
        .map(|m| run_cost(&with_dt(cfg, cfg.time.dt_s * m)))
        .sum();
    spatial + temporal
}
```

```rust
// src/main.rs, lines 20 to 21 and 141 to 150
/// Skip the refinement study when it would need more than this many cell-steps.
const STUDY_COST_LIMIT: f64 = 4.0e9;
    // ...
    if no_study {
        return Ok(());
    }
    let cost = compare::study_cost(&cfg);
    if cost > STUDY_COST_LIMIT {
        println!(
            "convergence_study=SKIPPED (estimated {cost:.2e} cell-steps; reduce end_time/dt or use fewer
            ↪  cells)"
        );
        return Ok(());
    }
```

Mapping of symbols to the excerpt: W is return value of `run_cost`; $t_{\mathrm{end}}/\Delta t_{\mathrm{in}}$ is `cfg.time.end_time_s / cfg.time.dt_s`; N_c is `cells`, sum of `r.cells` over the regions; λ_{x} is `f` (closure argument); λ_{t} is `m` (closure argument); factor 1.75 is the literal `1.75`; W_{study}

is return value `spatial + temporal`; W_{lim} is `STUDY_COST_LIMIT`; W_{study} as used in `main` is `cost`.

The factor 1.75 should be 7 because each spatial point runs three time-extrapolation runs of cost 4, 2 and 1 times one run (known defect K1; PROVED by counting).

5.13 One exact value (helper for tests and tools)

Operation A.13. Classification: IMPLEMENTED (not called by `main`). Equations: (4.26). Rust function: `exact_release_at`. Pinned lines: src/postprocessing/comparison.rs, lines 223 to 227.

Final equation.

$$\Lambda_{\max} = \frac{45}{t_{\min}}, \qquad t_{\text{valid}} = \frac{40}{\Lambda_{\text{cov}}}, \qquad F^{\text{ex}}(t) \text{ is reported only for } t \geq t_{\text{valid}}. \tag{4.26}$$

Pseudocode.

```
GIVEN one time t
1. t_min = t                                                    # the only time requested
2. Lambda_max = 45 / t_min                                      # Eq. (4.26)
3. Find Lambda_cov and the modes (rows C.7 to C.15)
4. t_valid = 40 / Lambda_cov                                    # Eq. (4.26)
5. IF t >= t_valid: RETURN F_ex(t)                              # Eq. (4.26)
   ELSE: RETURN no value
```

Rust.

```rust
// src/postprocessing/comparison.rs, lines 223 to 227
/// Exact release fraction at one time for a configuration (helper for tests and tools).
pub fn exact_release_at(cfg: &Config, t: f64) -> Result<f64> {
    let v = analytical::exact_release(&layers_from_config(cfg), &[t], MAX_MODES)?;
    v[0].ok_or_else(|| anyhow::anyhow!("exact series does not resolve t = {t:e} s"))
}
```

Mapping of symbols to the excerpt: t is `t`; $F^{\text{ex}}(t)$ is `v[0]`, an `Option<f64>`, turned into an error when `None`; $\Lambda_{\max} = 45/t$ is computed inside `exact_release` (row C.17); $K_{\max}$ is `MAX_MODES`.

Not called by `main`; used by `tests/finite_volume_vs_analytical.rs` through `exact_release_at`.

5.14 Output files and figures (`comparison`, `convergence_study`, SVG plots)

Operation A.14. Classification: IMPLEMENTED (presentation, no equation). Equations: none. Rust function: `main, plots`. Pinned lines: src/main.rs, lines 89 to 97; src/main.rs, lines 104 to 134; src/main.rs, lines 151 to 172; src/postprocessing/plots.rs, lines 29 to 112; src/postprocessing/plots.rs, lines 189 to 215; src/postprocessing/plots.rs, lines 217 to 273.

Final equation.

No equation.

Pseudocode.

```
No equation. The steps follow the row text:
1. Write the comparison results and the convergence study results to files
2. Draw the figures as SVG plots from those results
```

Rust.

```rust
// src/main.rs, lines 89 to 97
    let radii: Vec<f64> = mesh.cells.iter().map(|c| c.centroid).collect();
    let layers: Vec<(String, f64)> = cfg
        .regions
        .iter()
        .map(|r| (r.name.clone(), r.outer_radius_m))
        .collect();
    let fig = out.join("final_profile.svg");
    plots::profile(&fig, &cfg.case_name, &radii, &final_c, &layers)?;
    println!("figure={}", fig.display());
```

```rust
// src/main.rs, lines 104 to 118
    match compare::compare_release(&cfg, &rows) {
        Ok(cmp) => {
            let cmp_path = out.join("comparison_fv_vs_analytical.csv");
            let mut c = csv::Writer::from_path(&cmp_path)?;
            c.write_record(["time_s", "release_fv", "release_analytical", "abs_error"])?;
            for r in &cmp.rows {
                match r.exact {
                    Some(e) => c.serialize((r.time_s, r.fv, e, (r.fv - e).abs()))?,
                    None => {
                        let (t, f) = (r.time_s.to_string(), r.fv.to_string());
                        c.write_record([t.as_str(), f.as_str(), "", ""])?
                    }
                }
            }
            c.flush()?;
```

```rust
// src/main.rs, lines 151 to 168
    match compare::convergence_study(&cfg) {
        Ok(study) => {
            let study_path = out.join("convergence_study.csv");
            let mut s = csv::Writer::from_path(&study_path)?;
            s.write_record([
                "study",
                "total_cells",
                "dt_s",
                "fv_release",
                "abs_error_vs_exact",
            ])?;
            for (name, pts) in [("spatial", &study.spatial), ("temporal", &study.temporal)] {
                for q in pts {
                    s.serialize((name, q.cells, q.dt_s, q.fv_release, q.abs_error))?;
                }
            }
            s.flush()?;
            let fig = out.join("convergence.svg");
```

```rust
// src/postprocessing/plots.rs, lines 189 to 204
/// Spatial (left) and temporal (right) error against the exact series.
pub fn convergence(path: &Path, case: &str, study: &ConvergenceStudy) -> Result<()> {
    let root = SVGBackend::new(path, (1100, 480)).into_drawing_area();
    root.fill(&WHITE).map_err(pe)?;
    let (left, right) = root.split_horizontally(550);
    let pts = |v: &[ConvergencePoint], x: fn(&ConvergencePoint) -> f64| -> Vec<(f64, f64)> {
        v.iter().map(|p| (x(p), p.abs_error)).collect()
    };
    loglog_panel(
        &left,
        &format!("Spatial refinement ({case})"),
        "total FV cells",
        &pts(&study.spatial, |p| p.cells as f64),
        -2.0,
        study.accuracy_floor,
    )?;
```

Mapping of symbols to the excerpt: profile radii is `radii` (volume centroids); layer boundaries is `layers`, outer radii of the regions; profile figure is `plots::profile`; comparison table is `cmp.rows`, written to `comparison_fv_vs_analytical.csv`; comparison figure is `plots::release_comparison`; study table is `study.spatial`, `study.temporal`, written to `convergence_study.csv`; convergence figure is `plots::convergence`; reference slopes is the arguments `-2.0` and `1.0`.

Presentation only, no equation. The excerpts show the four places where a figure or a table is produced; lines 119 to 134 and 169 to 172 of `main.rs` and the bodies of the plot functions are not excerpted.

5.15 Temporal order against a same-mesh reference; relative error; check that $t_{\text{end}}/\Delta t_{\text{in}}$ is an integer

Operation A.15. Classification: FUTURE IMPROVEMENT. Equations: none. Rust function: none.

Final equation.

No equation and no Rust code.

Pseudocode.

```
NOT IMPLEMENTED. No equation and no Rust code. Nothing is written here.
```

Rust. No Rust function computes this row.

FUTURE IMPROVEMENT, no Rust function. A search of `comparison.rs`, `main.rs` and `config.rs` for same-mesh, relative, integer, fract and rem_euclid finds nothing.

6 Numerical properties

Table 6.1 lists the numerical properties that apply to the implemented methods of v1.0.0, with the final equation or condition, a one-sentence explanation and the way the property is implemented or verified. The status of each property is PROVED, DEMONSTRATED NUMERICALLY or CLAIMED ONLY, as defined in the title block. The proofs are in Part VI of the Full Derivation (Sections 6.1 to 6.11) and, for Code C, in Part IV (Sections 4.6 to 4.13). Numbers marked calculated come from the synthetic five-layer input. Item numbers such as I9 and K1 refer to Table 7.1, and items O10 and O13 to Table 7.5 of the Full Derivation. Check scripts such as `p6_properties` are listed in Appendix A of the Full Derivation. They are Python re-implementations, so a passing script confirms the algebra or the replica and does not by itself prove the Rust code correct.

Table 6.1 Numerical properties of the implemented v1.0.0 methods. Every link points to the tag `v1.0.0`.

No.	Mathematical property	Final equation or condition	Explanation	How implemented or verified
1	Discrete conservation	$M^{n+1} + \Delta t\, G_R C_{N_c}^{n+1} = M^n$, Eq. (6.1); $\varepsilon^n = 0$ in exact arithmetic	Summing the cell equations (5.13) over all cells, every interior flow leaves one cell and enters the next. The condition $g_1^W = 0$ closes the centre, so only the outer face remains.	PROVED (Section 6.1). The residual ε^n is evaluated at src/numerical/finite_volume.rs, lines 219 to 220. Main run: relative residual near 2.8×10^{-13}, DEMONSTRATED NUMERICALLY (Section 6.8.4). Asserted below 10^{-12} in examples/verify.rs, lines 199 to 223.

No.	Mathematical property	Final equation or condition	Explanation	How implemented or verified
2	Monotone inventory and bounded release	$M^{n+1} \le M^n$ and $0 \le F^n \le 1$ in exact arithmetic	Follows from Eq. (6.1) and $G_R C_{N_c}^{n+1} \ge 0$.	PROVED (Section 6.1). `run` records $\min_j C_j$ at src/numerical/finite_volume.rs, line 204 and src/numerical/finite_volume.rs, line 230. The checks are assertions at examples/verify.rs, lines 215 to 216.
3	M-matrix	$(\mathbf{V}/\Delta t + \mathbf{A})^{-1} \ge 0$ entrywise, Eq. (6.2)	The matrix has non-positive off-diagonal entries and a positive diagonal that exceeds the sum of the absolute off-diagonal entries by v_j.	PROVED (Section 6.2; Varga, 2000). The matrix is never formed; its entries are the coefficients of row B.12 at src/numerical/finite_volume.rs, lines 162 to 182.
4	Positivity and maximum principle	$\mathbf{C}^n \ge 0 \Rightarrow 0 \le C_j^{n+1} \le \max_k C_k^n$ for every $\Delta t > 0$, Eq. (6.3)	A nonnegative inverse applied to a nonnegative right-hand side gives nonnegative values, and the row margin gives the upper bound.	PROVED (Section 6.2). Test at src/numerical/finite_volume.rs, line 310 uses a looser tolerance than the property (item I22). Assertion at examples/verify.rs, line 215.
5	Positivity in floating point	$y_j \ge 0$ and $C_j^{n+1} \ge 0$ exactly, Eq. (6.4)	The elimination of Eq. (5.18) and the sweeps of Eq. (5.19) use only sums, products and quotients of nonnegative numbers, so no subtraction can cancel.	PROVED (IEEE rounding model, Section 6.3.3). Implemented in `solve_diffusion_system` at src/numerical/finite_volume.rs, lines 132 to 160.
6	Pivots and no pivoting	$v_j \le e_j \le v_j + g_j^W$, $d_j > 0$, multipliers in $(0, 1)$	Every pivot is bounded away from zero by the row margin, so Gaussian elimination needs no row exchange.	PROVED (Section 6.3.1). No pivoting in src/numerical/finite_volume.rs, lines 132 to 160.
7	Cost of one step	13 operations per cell, $O(N_c)$ work and memory	Assembly, one forward sweep and one back substitution each touch every cell once.	PROVED by counting (Section 6.3.2), src/numerical/finite_volume.rs, lines 140 to 158 and src/numerical/finite_volume.rs, lines 178 to 179.
8	A-stability and L-stability of Backward Euler	$\lvert\mathcal{R}(\zeta)\rvert \le 1$ for $\operatorname{Re}\zeta \le 0$ with $\mathcal{R}(\zeta) = 1/(1-\zeta)$, and $\mathcal{R} \to 0$ as $\zeta \to -\infty$	Backward Euler damps every decaying mode for every step size.	PROVED (established textbook result, Hairer & Wanner, 1996; Section 6.4.1).
9	Symmetric positive definite conductance matrix	$\mathbf{A} = \mathbf{A}^{\mathsf{T}}$ and $\mathbf{x}^{\mathsf{T}}\mathbf{A}\mathbf{x} > 0$	The conductances $G_{j+1/2}$ and G_R are positive and enter symmetrically.	PROVED (Section 6.4.2). Smallest scaled eigenvalue $\mu_{\min} = 3.2733 \times 10^{-6}\ \mathrm{s}^{-1}$ (calculated).
10	Unconditional stability	$\lVert\mathbf{C}^{n+1}\rVert_V \le \dfrac{1}{1 + \Delta t\, \mu_{\min}} \lVert\mathbf{C}^n\rVert_V$ for every $\Delta t > 0$, Eq. (6.5)	Every step shrinks the volume-weighted norm, so the step size does not limit stability.	PROVED (Section 6.4.3). Holds through the choice of Backward Euler in row B.12.

No.	Mathematical property	Final equation or condition	Explanation	How implemented or verified
11	Forward Euler positivity bound	$\Delta t_{\mathrm{FE}} = \min_j V_j/(g_j^W + g_j^E)$, Eq. (6.6)	The largest explicit step that keeps every cell nonnegative.	PROVED. IMPLEMENTED as a diagnostic only, src/numerical/finite_volume.rs, lines 238 to 252, printed once. Value 7.812×10^{-3} s (calculated).
12	Forward Euler L^2 bound	$\Delta t \le 2/\mu_{\max}$, Eq. (6.7); always $\Delta t_{\mathrm{FE}} \le 2/\mu_{\max}$	Stability limit of the explicit scheme in the volume-weighted norm.	PROVED. DERIVED NOT IMPLEMENTED (row B.21). Value 7.824×10^{-3} s (calculated).
13	Stable and still inaccurate at large Fourier number	$\mathrm{Fo} = D\,\Delta t/w^2$, Eq. (6.13), up to 1.28×10^4	A large step stays stable and positive and loses first-order accuracy.	DEMONSTRATED NUMERICALLY (Section 6.4.5): 2.8 percent relative error at 2.2×10^4 s, from `tests/golden/v1.0/comparison_fv_vs_analytical.csv`.
14	Flux consistency at interior faces	$\tau = O(w^2)$	Central difference with centroid distances.	PROVED (Section 6.5.2, case a).
15	Flux consistency at interfaces and the outer face	$\tau = O(w)$; $\tau = 0$ if and only if $w_L/w_R = \sqrt{D_L/D_R}$	The two half-cell distances differ across an interface, so the error is first order unless the width ratio matches the diffusivity ratio.	PROVED (Section 6.5.2, cases b and c).
16	Flux consistency at the faces next to the centre	$\tau = O(w^3)$ at $r_f = w,\ 2w,\ 3w$	The Taylor expansion of the face flux at the three faces nearest the centre cancels the lower orders.	PROVED by Taylor expansion for three faces (Section 6.5.2, case d).
17	Local error of Backward Euler	$-\frac{1}{2}\Delta t^2\, y'' + O(\Delta t^3)$	Standard first-order local truncation error.	PROVED (Section 6.5.3).
18	Convergence of the semi-discrete scheme	$\|\eta\|_V = O(w^{3/2})$, Eq. (6.15)	Energy estimate that bounds the error by the accumulated face truncation errors.	PROVED for smooth solutions with uniform remainders assumed. CLAIMED ONLY for the discontinuous initial step of v1.0.0 (Section 6.10).
19	Observed spatial order of F	$F^{\mathrm{FV}} - F = O(w^2) + O(\Delta t)$, Eq. (6.8); $p_k = \log_2(\mathrm{err}_k/\mathrm{err}_{k+1})$, Eq. (6.11)	Refining the mesh by a factor 2 divides the error by about 4.	DEMONSTRATED NUMERICALLY: $p_k = 2.00$ (five layers), above the proved order $3/2$. Computed by `ConvergenceStudy::orders`, src/postprocessing/comparison.rs, lines 112 to 118.
20	Observed temporal order of F	$p_k \approx 1$, Eq. (6.8)	Refining the step by a factor 2 divides the error by about 2.	DEMONSTRATED NUMERICALLY: 0.999 to 1.000 against a same-mesh reference. The printed 0.98 to 0.89 includes the spatial error of the 200-cell mesh (item I9).

No.	Mathematical property	Final equation or condition	Explanation	How implemented or verified
21	Richardson cancellation	$F_{\text{RE}} = \frac{1}{3}(8F(\Delta t/4) - 6F(\Delta t/2) + F(\Delta t)) = F_0 + \frac{1}{8}\chi_3 \Delta t^3 + \ldots$, Eq. (6.10)	The weights remove the terms in Δt and Δt^2 of the error model, Eq. (6.9).	PROVED given Eq. (6.9). Implemented in `time_extrapolated_release`, src/postprocessing/comparison.rs, lines 137 to 157.
22	No cancellation in the elimination	$e_j = v_j + \frac{g_j^W e_{j-1}}{e_{j-1} + g_j^W}$, Eq. (5.18)	The pivot has the same value as the textbook form of Eq. (5.17) and contains no subtraction.	PROVED for the sign. CLAIMED ONLY for the constants of the worst-case bounds (Section 6.8.3).
23	Balance drift at very large Fourier number	drift of ε^n / M^0 at $\text{Fo} = 2.5 \times 10^5$	The subtraction-free form keeps the balance at round-off level.	DEMONSTRATED NUMERICALLY by Python replicas: 1.9×10^{-16} to 3.9×10^{-16}, against 9.5×10^{-14} for the textbook form. The Rust test asserts only a bound of 10^{-12}; it passed under `cargo test --release` in the acceptance check of 2026-11-06 (item O10).
24	Textbook pivot "loses about 6 digits"	code comments at src/numerical/finite_volume.rs, lines 126 to 131 and src/numerical/finite_volume.rs, line 377	The mechanism (cancellation in the textbook pivot) is real.	CLAIMED ONLY: the quantitative claim was not reproduced (items I5 and K4).
25	Round-off of the balance residual	$\lvert\varepsilon^n\rvert / M^0$ near 10^{-13}	Rounding in the sums of Eqs. (5.21) and (5.23) depends on the operation order.	DEMONSTRATED NUMERICALLY (main run 2.806×10^{-13}). No bound proved (Section 6.8.4).
26	Cost of the refinement study	$W = \frac{t_{\text{end}}}{\Delta t_{\text{in}}} N_c$ and W_{study} of Eq. (6.14)	The count of runs shows that the spatial part must use the factor 7, not 1.75.	PROVED by counting. The code uses 1.75 in `study_cost`, src/postprocessing/comparison.rs, lines 159 to 176 (item K1). The result is unaffected; only the skip decision is.
27	Real, positive, simple eigenvalues and orthogonal eigenfunctions	$\int_0^R u_m \, u_n \, dr = 0$ for $m \neq n$, Eq. (4.17)	The layered problem is a Sturm-Liouville problem with weight r^2 for ϕ.	PROVED (Section 4.8). DEMONSTRATED NUMERICALLY for 32 modes. The orthogonality itself is DERIVED NOT IMPLEMENTED (row C.20).

No.	Mathematical property	Final equation or condition	Explanation	How implemented or verified
28	Sturm count	$\mathcal{N}(\Lambda) = \#\{n : \Lambda_n \leq \Lambda\}$, Eq. (4.13)	The number of zeros of u in $(0, R]$ equals the number of eigenvalues at or below Λ.	PROVED in exact arithmetic with a quoted comparison theorem (Section 4.6). DEMONSTRATED NUMERICALLY for 95 modes. Implemented in `eigenvalue_count`, src/analytical/sturm_liouville.rs, lines 164 to 192.
29	Bisection on the count misses no eigenvalue	stopping rule $b - a \leq 4 \times 10^{-15} b$, Eq. (4.15)	A bracket is dropped only when the count shows that it holds no eigenvalue.	PROVED for an exact count. DEMONSTRATED NUMERICALLY for the float64 code. Implemented at src/analytical/sturm_liouville.rs, lines 240 to 263.
30	Accuracy of the eigenvalues	relative error about $\epsilon_{\mathrm{m}}/\rho_{\min}$	The shooting recursion loses digits where flux terms cancel at an interface.	DEMONSTRATED NUMERICALLY (synthetic case, Section 4.7).
31	Completeness of the modes	$\lim_{K\to\infty} \sum_{n\leq K} w_n = 1$, Eq. (4.23)	The weights add up to 1 when all modes are kept, which means $F(0) = 0$.	CLAIMED ONLY for the layered sphere. DEMONSTRATED NUMERICALLY: 32 modes give 1.000007275592, checked by `modal_completeness`, examples/verify.rs, lines 228 to 246. PROVED for the homogeneous sphere, Eq. (4.30).
32	Panel rule of the quadrature	$k_\ell h < \pi/2$ with $N_{\mathrm{p},\ell} = \lceil k_\ell L_\ell/(\pi/2)\rceil + 1$, Eq. (4.24)	Each 16-point panel integrates polynomials exactly to degree 31, and a panel is shorter than a quarter wave.	PROVED (Section 4.11). Implemented at src/analytical/sturm_liouville.rs, lines 266 to 304 and src/analytical/sturm_liouville.rs, line 340.
33	Quadrature error of the weights	about 10^{-13}	Round-off of the composite Gauss-Legendre sum.	DEMONSTRATED NUMERICALLY (synthetic case, Section 4.11).
34	Truncation of the series	each omitted mode is below e^{-40} of its weight for $t \geq t_{\mathrm{valid}}$, Eq. (6.16)	The gate $t_{\mathrm{valid}} = 40/\Lambda_{\mathrm{cov}}$ makes every omitted term $e^{-\Lambda_n t} \leq e^{-40}$.	PROVED per term. The sum of the omitted tail is DEMONSTRATED NUMERICALLY. Implemented at src/analytical/sturm_liouville.rs, lines 375 to 379.
35	Accuracy of the kept eigenvalues and weights	3.7×10^{-13} relative for Λ_n, 9.0×10^{-14} absolute for w_n (Rust), Eq. (6.17)	Error propagation from the eigenvalues and weights to F.	DEMONSTRATED NUMERICALLY by a one-off comparison of the library against a high-precision oracle (item O13).
36	Accuracy floor of the series	$\varepsilon_F = 2.2 \times 10^{-16} / \min(\rho_{\min}, 1) = 3.997 \times 10^{-12}$ (calculated), Eq. (4.28)	An estimate of the cancellation error of $1 - \sum_n w_n e^{-\Lambda_n t}$.	CLAIMED ONLY as a bound (heuristic, one case). Implemented at src/analytical/sturm_liouville.rs, lines 330 to 336 and src/analytical/sturm_liouville.rs, lines 366 to 373.

No.	Mathematical property	Final equation or condition	Explanation	How implemented or verified
37	Slightly negative early-time value	$F^{\mathrm{ex}}(2 \times 10^{3}\ \mathrm{s}) = -8.55 \times 10^{-14}$, true value 1.6×10^{-15}	The series is a difference of nearly equal numbers at early time, and the result lies inside ε_F.	DEMONSTRATED NUMERICALLY, golden file `tests/golden/v1.0/comparison_fv_vs_analytical.csv`. The code does not clamp it (item K9).
38	Nonnegative concentration and full release	$c \geq 0$ and $0 \leq 1 - F \leq (R/R_1)^{3/2} e^{-6 D_{\min} t / R^2}$	The continuous problem keeps c nonnegative and empties the particle.	PROVED (Section 3.5.5). DERIVED NOT IMPLEMENTED. The discrete counterparts are rows 2 and 4.

7 Known inconsistencies

Table 7.1 lists every place where a delivered Phase 1 document, a code comment, a test or the code itself disagrees with another of them, as found while writing the Full Derivation (Section 7.5 there, where the table is Table 7.3). The Rust code was not changed for any item. For each item the table gives the description, the side that is wrong, the recommended correction and a status. Section numbers and equation numbers in the table refer to the Full Derivation, and the item numbers I1 to I26 and K1 to K18 are the ones used there.

The Phase 1 derivation is docs/mathematics/phase1/TRISO_Finite_Volume_Derivation.md and the Phase 1 pseudocode is docs/mathematics/phase1/TRISO_Finite_Volume_Pseudocode.md. Line numbers given for them refer to the tag. The status codes are: A, handled in these documents, while editing the delivered Phase 1 file needs owner approval; B, handled in these documents and no approval is needed; C, a change to Rust code, a comment, a test or the manifest needs owner approval; D, handled in these documents, while a change to code or comments needs owner approval.

Table 7.1 Known inconsistencies between the delivered Phase 1 documents, code comments, tests and code (Table 7.3 of the Full Derivation).

ID	Description	Wrong side	Recommended correction	Status
I1	The Phase 1 derivation (line 348) defines $\dot{F}^n = G_R C_N^n / M^0$. The code computes and writes $\Phi_R^n = G_R C_{N_c}^n$ in mol s^{-1} (src/numerical/finite_volume.rs, line 202 and src/numerical/finite_volume.rs, line 210, column `release_rate_mol_s` at src/main.rs, line 50).	Phase 1 document	Define Φ_R^n as implemented (Eq. (5.22)); give $\dot{F} = \Phi_R / M^0$ as DERIVED NOT IMPLEMENTED.	A
I2	The Phase 1 derivation (line 348) and pseudocode (line 34) show the plain sum $M_{\mathrm{rel}}^{n+1} = M_{\mathrm{rel}}^n + \Delta t\, G_R C_N^{n+1}$. The code uses Kahan compensation (src/numerical/finite_volume.rs, lines 211 to 217). The effect over the synthetic run is $4.3 \times 10^{-16} M^0$ in a Python replica.	Phase 1 document (incomplete)	State the compensated recurrence, Eq. (5.23), and why it exists.	A

ID	Description	Wrong side	Recommended correction	Status
I3	The Phase 1 documents use the input step. The code uses $\Delta t = t_{\text{end}}/\lceil t_{\text{end}}/\Delta t_{\text{in}} \rceil$ (src/numerical/finite_volume.rs, lines 190 to 191), which equals the input for the synthetic case (1000 steps of 200 s).	Phase 1 document (incomplete)	State the rule, Eq. (5.16); see K13.	A
I4	The Phase 1 derivation, Section 5.1, does not say that cells are uniform inside a layer or that the last face of a layer is set to R_ℓ (src/numerical/finite_volume.rs, lines 55 to 62).	Phase 1 document (incomplete)	Give $w_\ell = L_\ell/n_\ell$ and the snapping rule, Eq. (5.2).	A
I5	The Phase 1 derivation (line 331) says the textbook elimination lets the balance drift to about 10^{-9}. Code comments at src/numerical/finite_volume.rs, lines 126 to 131 and src/numerical/finite_volume.rs, line 377 say it breaks the balance and loses about 6 digits. No test reproduces this. Python replicas of two cases give a drift of 4.6×10^{-14} to 9.5×10^{-14} for the textbook pivot and 3.9×10^{-16} to 1.2×10^{-15} for the subtraction-free form.	Phase 1 document and code comments (overstated); the code is correct	Describe the subtraction-free form as a no-cancellation and floating-point positivity guarantee; quote replica numbers as replica numbers (Section 6.8).	D
I6	The Phase 1 derivation (lines 333 to 341) and pseudocode (lines 23 to 29) suggest that a forward Euler check exists. Only the bound is computed (src/numerical/finite_volume.rs, lines 238 to 252, printed at src/main.rs, line 78).	Phase 1 documents	Present Δt_{FE} as an implemented diagnostic. The scheme and Eq. (6.7) are DERIVED NOT IMPLEMENTED.	A
I7	The pseudocode (line 71) checks $C \geq 0$ and $0 \leq F \leq 1$ inside the time loop. `run` only records $\min_j C_j$ (src/numerical/finite_volume.rs, line 204 and src/numerical/finite_volume.rs, line 230). The checks are assertions in examples/verify.rs, lines 215 to 216 and src/numerical/finite_volume.rs, line 310.	Phase 1 pseudocode	Write "record $\min_j C_j^n$"; list the assertions under verification.	A
I8	The pseudocode (line 94) reports runtime, positivity and maximum conservation residual. The program prints only the final residual, under the key `max_final_balance_residual_mol` (src/main.rs, lines 81 to 84), and no runtime. The runtime is printed by examples/verify.rs, line 224.	Phase 1 pseudocode and the printed key name	Document it as the final residual. FUTURE: rename the key or print the maximum.	D
I9	The printed temporal orders 0.98 to 0.89 (src/main.rs, lines 189 to 193) understate first order because the 200-cell spatial error enters. Against the same-mesh reference the orders are 0.999 to 1.000 (Section 7.3.4).	Presentation without caveat	State both numbers and the reason. A same-mesh reference is FUTURE.	B

ID	Description	Wrong side	Recommended correction	Status
I10	The Phase 1 derivation (line 220) and pseudocode (line 84) use $t \geq 40/\Lambda_{\max}$. The code requests $\Lambda_{\max} = 45/t_{\min}$ (src/postprocessing/comparison.rs, line 59), may lower it to Λ_{cov} by the cap (src/analytical/sturm_liouville.rs, lines 202 to 217) and gates at $40/\Lambda_{\text{cov}}$ (src/analytical/sturm_liouville.rs, lines 377 to 379).	Phase 1 documents (incomplete)	Define $\Lambda_{\max}$, Λ_{cov} and $K_{\max} = 20\,000$ separately, Eq. (4.26).	A
I11	The Phase 1 derivation (line 211) writes $\sum_n w_n = 1$ beside the truncated series. Thirty-two modes give 1.000007275592.	Phase 1 document (imprecise)	Write $\lim_{K\to\infty} \sum_{n\leq K} w_n = 1$, checked numerically by `modal_completeness`, examples/verify.rs, lines 228 to 246, not proved (Eq. (4.23)).	A
I12	The Phase 1 derivation (line 198) writes $c = \sum_n A_n \phi_n e^{-\Lambda_n t}$ as part of the method. Code C never forms A_n or $c(r,t)$.	Phase 1 document	Label as DERIVED NOT IMPLEMENTED (Eqs. (4.18), (4.19)).	A
I13	The Phase 1 derivation (line 186) cites $\det \mathbf{M}(\Lambda) = 0$ and the pseudocode (lines 40 and 79) isolates roots of $g = u(R;\Lambda)$. The code evaluates g and discards it (src/analytical/sturm_liouville.rs, line 329); roots come from the Sturm count only.	Phase 1 documents	Present g and the determinant as equivalent characterisations that are DERIVED NOT IMPLEMENTED (Eq. (4.10)).	A
I14	The Phase 1 derivation (line 214) says integrals are computed layer by layer. The code uses composite 16-point Gauss-Legendre with $\lceil k_\ell L_\ell/(\pi/2)\rceil + 1$ panels (src/analytical/sturm_liouville.rs, line 340); closed forms exist and are unused.	Phase 1 document (incomplete)	Describe the rule and its exactness to degree 31 per panel (Eq. (4.24)); closed forms (4.25) are DERIVED NOT IMPLEMENTED.	A
I15	The Phase 1 derivation (line 221) defines ρ without saying on which side of an interface. The code uses the post-jump q and the outer layer's D (src/analytical/sturm_liouville.rs, lines 330 to 336): outer side 5.504×10^{-5} (floor 4.0×10^{-12}), inner side 1.231×10^{-4} (floor 1.8×10^{-12}).	Phase 1 document (incomplete)	State the side and call ε_F an estimate (Eqs. (4.27), (4.28)).	A
I16	The Phase 1 derivation (line 363) says the floor grows roughly in proportion to the diffusivity contrast. The code comment at src/analytical/sturm_liouville.rs, lines 369 to 370 says the true error stayed 10 to 20 times below the estimate. Neither is tested, and the second rests on one case (a factor 21).	Phase 1 document and code comment (unsupported)	Mark both as claims until a contrast sweep exists.	D

ID	Description	Wrong side	Recommended correction	Status
I17	The Phase 1 derivation (line 329) says the diagonal entry is larger than the sum of the other entries, “which are zero or negative”. Strict dominance compares with the sum of absolute values.	Phase 1 document, wording only (the conclusion is right)	Write $a_{jj} > \sum_{k \neq j} \lvert a_{jk} \rvert$ with margin $v_j = V_j / \Delta t$ (Section 6.2).	A
I18	The Phase 1 derivation (line 29) calls D_i a “constant speed”.	Phase 1 document	Write “constant diffusivity” with unit $\mathrm{m}^2\ \mathrm{s}^{-1}$.	A
I19	The Phase 1 derivation (line 188) counts the allowed fade rates “below a given Λ”. The count includes equality (src/analytical/sturm_liouville.rs, lines 164 to 165).	Phase 1 document	Use $\leq$ and “zeros in $(0, R]$” (Eq. (4.13)).	A
I20	The Phase 1 derivation cites labels such as TRISO-FV-114 and TRISO-ML-474 from a working document outside STRATO (Phase 1 README, line 14).	Phase 1 document	This derivation is self-contained and has its own numbering.	B
I21	The Phase 1 derivation and pseudocode (lines 14 and 20) carry a source term $S_j V_j$. The code has none (`rhs = v * old`, src/numerical/finite_volume.rs, line 179).	Phase 1 documents (more general than the code)	Derive with S in Part II, set $S = 0$ explicitly and mark the discrete source FUTURE.	B
I22	The Phase 1 derivation proves positivity in exact arithmetic only. The sweep also guarantees $C^{n+1} \geq 0$ in floating point. The unit test at src/numerical/finite_volume.rs, line 310 uses a looser tolerance than the property.	Phase 1 document (incomplete)	State the floating-point property, Eq. (6.4).	A
I23	The Phase 1 derivation, Section 4.6, agrees with `robin_residual` and `robin_roots` (src/analytical/sturm_liouville.rs, lines 31 to 79) but does not say that only a unit test calls them (src/analytical/sturm_liouville.rs, lines 447 to 454).	Phase 1 document (incomplete)	Add that sentence.	A
I24	Notation clash. The Phase 1 derivation uses i for the layer and j for the cell and writes δ_P, δ_E and the subscripts P, E, W. Rust uses `i` for the cell, `mat` for the layer and `dl`, `dr`, `gw`, `ge`.	Notation	One notation table with a Rust-name column (Table 7.1 and the notation tables).	B
I25	If the cross-check fails, the program prints `analytical_cross_check=SKIPPED` and returns success, which also skips the convergence study (src/main.rs, lines 135 to 138). The pseudocode (lines 87 to 92) omits this branch.	Phase 1 pseudocode (incomplete); the exit status 0 is a code choice	Document the branch. FUTURE: a non-zero exit or a separate message for the skipped study.	D
I26	Working notes gave the explicit step bound as $7.812107958750 \times 10^{-3}$ s in some rows and ...751 in others. The golden `main_run_stdout.txt` prints `7.812107958751e-3`.	Working notes	Use the printed value, or 7.8121×10^{-3} s in prose.	B

ID	Description	Wrong side	Recommended correction	Status
K1	`study_cost` multiplies each spatial run cost by 1.75 (src/postprocessing/comparison.rs, line 169) while `time_extrapolated_release` costs $4 + 2 + 1 = 7$ runs. The spatial part is low by a factor 4.00 (synthetic: 5.425×10^6 against 2.17×10^7). Results are unaffected, only the skip decision.	Code	Use the factor 7 in a later release; the documents report the formula as implemented and flag it.	C
K2	The comment "each refinement halves h or dt" (src/postprocessing/comparison.rs, line 112) holds only if every n_ℓ is even (the scaling by 0.5 uses `round` at src/postprocessing/comparison.rs, line 132) and $t_{\text{end}}/\Delta t$ stays an integer. This is true for the synthetic input.	Comment (conditionally true)	State the condition. FUTURE: $p_k = \ln(\text{err}_k/\text{err}_{k+1})/\ln(h_k/h_{k+1})$.	D
K3	The Richardson comment (src/postprocessing/comparison.rs, lines 150 to 153) needs the three runs in the ratio $1 : \frac{1}{2} : \frac{1}{4}$, which holds only if $t_{\text{end}}/\Delta t_{\text{in}}$ is an integer; see K13.	Comment	As K2.	D
K4	The comments at src/numerical/finite_volume.rs, lines 126 to 131 and src/numerical/finite_volume.rs, line 377: the mechanism (cancellation in $b_j - a_j c'_{j-1}$) is real, the quantitative claim is not reproduced (item I5).	Comments	Soften them or add a test.	C
K5	The accuracy-floor comment at src/analytical/sturm_liouville.rs, lines 366 to 370; see I16.	Comment	Treat as a claim only.	C
K6	The comment at src/analytical/sturm_liouville.rs, lines 396 to 398 says the omitted modes retain e^{-45} at the earliest time. That holds only when the cap is not reached; with a cap, $\Lambda_{\text{cov}} < 45/t_{\text{min}}$ and early times return no value. The gate uses e^{-40} (src/analytical/sturm_liouville.rs, lines 375 to 376), conservative by e^{-5} when no cap acts.	Comment (incomplete)	State both constants and the cap case.	C
K7	src/lib.rs, line 13 points to `docs/mathematics/STRATO_Mathematical_Derivations.md`, which does not exist at the tag. The Phase 1 README (line 16) says it is being written for v2.0, while these documents live in `docs/mathematics/v1.0.0/`.	Code comment and path plan	Publish the derivation at the cited path or change the reference in a later release.	C

ID	Description	Wrong side	Recommended correction	Status
K8	The comment at src/analytical/sturm_liouville.rs, lines 38 to 39 says poles of cot z are skipped by scanning finite sign changes. The poles at $n\pi$ are excluded by the 10^{-8} offsets (src/analytical/sturm_liouville.rs, lines 44 to 45).	Comment (imprecise)	Rewrite it.	C
K9	`release_fraction` (src/analytical/sturm_liouville.rs, lines 391 to 393) can return a slightly negative F at early time (-8.55×10^{-14} at 2×10^{3} s, inside ε_F). No comment claims otherwise.	Code behaviour	FUTURE: flag or clamp.	C
K10	`homogeneous_uniform` (examples/verify.rs, lines 62 to 85) takes $\lfloor t_{\text{end}}/\Delta t \rfloor$ steps and an uncompensated sum, so it does not exercise `run`.	None (the documents must not cite it as a test of `run`)	State this in the documents.	B
K11	`temperature_k` is read and validated (src/config.rs, lines 86 to 88) but no formula uses it.	Scope statement	The documents must not suggest a temperature dependence.	B
K12	The comment at src/postprocessing/comparison.rs, line 95 documents `ConvergencePoint.cells` as total cells or a time step in seconds. The field always holds the total cell count (src/postprocessing/comparison.rs, line 197 and src/postprocessing/comparison.rs, line 209); the time step is in `dt_s`.	Code comment	Fix the comment; no effect on results.	C
K13	The step rule can change N_t by one for decimal inputs, because the quotient of two doubles may land just above an integer. With $\Delta t_{\text{in}} = 0.01$ and $t_{\text{end}} = 0.07$ the quotient is 7.000000000000001 and $N_t = 8$. The three Richardson runs then take 8, 15 and 29 steps instead of 7, 14 and 28. This was found with Python double arithmetic; the Rust program was not run on these inputs. It is absent from the synthetic input, where $2 \times 10^{5}/200 = 1000$ exactly.	Code	FUTURE: round the quotient to the nearest integer when it is within a tolerance of one, or reject non-integer ratios.	C
K14	src/analytical/sturm_liouville.rs, line 67 uses `Option::is_none_or`, stable since Rust 1.82. `Cargo.toml` declares `edition = "2021"` but no `rust-version`, and there is no toolchain file, so an older toolchain fails to compile.	Manifest	Add `rust-version = "1.82"` in a later release; the documents state the toolchain used.	C

ID	Description	Wrong side	Recommended correction	Status
K15	The comment at examples/verify.rs, lines 32 to 35 says the errors at Δt and $\Delta t/2$ differ by about a factor 2. The measured ratio is 1.811, because the 200-cell spatial error is included. The assertion window $[1.8, 2.2]$ is passed with a margin of 0.6 percent.	Comment and window	State the cause; widen or change the check in a later release.	C
K16	The comment at src/analytical/sturm_liouville.rs, lines 130 to 131 documents the `rescale` parameter of `shoot` as meant for counting. Counting has its own inline rescale (src/analytical/sturm_liouville.rs, lines 183 to 187) and `shoot` is called only with `false` (src/analytical/sturm_liouville.rs, line 329), so the branch at src/analytical/sturm_liouville.rs, lines 151 to 157 is dead.	Dead code with a misleading comment	Document it as unused or remove it later.	C
K17	The comments at src/analytical/sturm_liouville.rs, line 7 and src/analytical/sturm_liouville.rs, line 306 describe a five-layer series, while `validate_layers` accepts any $N \geq 2$ (src/analytical/sturm_liouville.rs, lines 112 to 127).	Comment (narrower than the code)	Say "multilayer, $N \geq 2$"; the configuration still fixes five regions.	C
K18	The test `analytical_series_matches_v1_golden` (tests/phase1_regression.rs, lines 71 to 88) compares the series with the golden file at 10^{-12}, below $\varepsilon_F = 4.0 \times 10^{-12}$. It passes on the machine that produced the golden file; behaviour on another platform is untested.	Test tolerance	Document it as platform dependent; consider 10^{-10}.	C

Three items bear directly on rows of this document: K1 (the factor 1.75 of row A.12 should be 7), I5 (the claim that the textbook pivot loses digits, row B.13) and K9 (a slightly negative F at early time, row C.16). The future improvements and the properties that stay open in v1.0.0 are collected in Table 7.4 of the Full Derivation, and the statements whose evidence is thinner than their section label suggests are listed in Table 7.5 there (items O1 to O15). In particular, the Rust unit tests and `examples/verify.rs` were not re-run while the Full Derivation was written. The 20 Rust tests were run afterwards, in the acceptance check of 2026-11-06 (`cargo test --release`), and all passed (item O10); `examples/verify.rs` was not re-run, and the pandoc to XeLaTeX build of the two documents was not run on the machine used (item O15).

8 References

The list follows the APA 7 style. It contains the works cited in this document. The Full Derivation lists the complete set and states how the bibliographic data were checked. The code, the benchmark and the tests of STRATO are cited by file and line range at tag `v1.0.0` of https://github.com/weewzr/STRATO and not through this list.

Gelbard, F. (2003). *Analytical modeling of fission product releases by diffusion from*

multicoated fuel particles (Report No. SAND2002-3966). Sandia National Laboratories. https://doi.org/10.2172/809605

Hairer, E., & Wanner, G. (1996). *Solving ordinary differential equations II: Stiff and differential-algebraic problems* (2nd rev. ed.). Springer. https://doi.org/10.1007/978-3-642-05221-7

Higham, N. J. (2002). *Accuracy and stability of numerical algorithms* (2nd ed.). SIAM. https://doi.org/10.1137/1.9780898718027

International Atomic Energy Agency. (1997). *Fuel performance and fission product behaviour in gas cooled reactors* (IAEA-TECDOC-978).

Kahan, W. (1965). Pracniques: Further remarks on reducing truncation errors. *Communications of the ACM, 8*(1), 40. https://doi.org/10.1145/363707.363723

Patankar, S. V. (1980). *Numerical heat transfer and fluid flow*. Hemisphere Publishing Corporation.

Varga, R. S. (2000). *Matrix iterative analysis* (2nd rev. and expanded ed.). Springer. https://doi.org/10.1007/978-3-642-05156-2